

INFORMATIQUE • TLE TI

MODULE 4 : PROJET

LEÇON N15

PROJET INTÉGRATEUR : CONCEPTION ET DÉVELOPPEMENT D'UNE APPLICATION INFORMATIQUE

DURÉE CONSEILLÉE : 10 H

DANS CETTE LEÇON

Cette leçon de synthèse guide l'élève dans la conduite complète d'un projet informatique : du cahier des charges à la publication d'une application web dynamique intégrant base de données et réseau local. Elle mobilise et articule toutes les compétences des modules Programmation, Réseaux/Internet/Sécurité et Systèmes d'information/Bases de données, et aboutit à la production d'un rapport de réalisation et d'un guide d'utilisation.

- À la fin de cette leçon, je sais rédiger un cahier des charges complet (besoins, fonctionnalités, contraintes, livrables) à partir d'une situation réelle
- À la fin de cette leçon, je sais planifier un projet informatique (découpage en tâches, diagramme de Gantt, répartition des rôles)
- À la fin de cette leçon, je sais modéliser un système d'information avec UML (cas d'utilisation, classes, séquence) et le traduire en base de données
- À la fin de cette leçon, je sais développer une application web dynamique connectée à une base de données
- À la fin de cette leçon, je sais configurer un petit réseau local pour héberger et publier mon application
- À la fin de cette leçon, je sais concevoir et exécuter un plan de tests (unitaires, d'intégration, de validation)



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Lancement du projet : cahier des charges et planification
2. Conception du système d'information avec UML et la base de données
3. Réalisation : développement de l'application web dynamique
4. Réseau et publication : héberger l'application sur un réseau local
5. Tests, validation et recette de l'application
6. Documentation : rapport de réalisation et guide d'utilisation
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais rédiger un cahier des charges complet (besoins, fonctionnalités, contraintes, livrables) à partir d'une situation réelle
- À la fin de cette leçon, je sais planifier un projet informatique (découpage en tâches, diagramme de Gantt, répartition des rôles)
- À la fin de cette leçon, je sais modéliser un système d'information avec UML (cas d'utilisation, classes, séquence) et le traduire en base de données
- À la fin de cette leçon, je sais développer une application web dynamique connectée à une base de données
- À la fin de cette leçon, je sais configurer un petit réseau local pour héberger et publier mon application
- À la fin de cette leçon, je sais concevoir et exécuter un plan de tests (unitaires, d'intégration, de validation)
- À la fin de cette leçon, je sais rédiger un rapport de réalisation de projet conforme à la structure attendue
- À la fin de cette leçon, je sais produire un guide d'utilisation clair destiné aux utilisateurs finaux
- À la fin de cette leçon, je sais intégrer de manière cohérente les compétences des trois modules du programme de Terminale TI

PRÉREQUIS

- Programmation web : HTML, CSS, PHP et requêtes SQL (modules 1 et 3)
- Conception de bases de données : MCD, MLD, création de tables sous MySQL
- Notions de réseau : adressage IP, configuration d'un poste, client/serveur
- Modélisation UML : diagrammes de cas d'utilisation et de classes
- Démarche algorithmique et structuration d'un programme

1 LANCEMENT DU PROJET : CAHIER DES CHARGES ET PLANIFICATION

Au lycée bilingue de Nkongsamba, c'est la fin du premier trimestre. Dans le secrétariat, trois enseignants remplissent à la main des centaines de bulletins ; une erreur de report oblige à tout recommencer. Dehors, des parents font la queue depuis 7 heures pour demander « comment se porte mon enfant en maths ». Les informations circulent par bouche-à-oreille, les moyennes sont recalculées à la calculatrice, et les relevés s'égarent entre les salles de classe. Excédé, le proviseur convoque les élèves de Terminale TI et leur lance un défi : « Vous avez huit semaines. Concevez une application web de gestion des notes, hébergée sur le réseau local du lycée, accessible aux enseignants depuis la salle informatique. Je veux un rapport de réalisation et un guide d'utilisation. »

Comment passer d'une idée à une application fonctionnelle, testée, documentée et publiée ? Quelles étapes suivre, quels outils mobiliser, comment répartir le travail dans l'équipe ? La réponse tient en deux mots : **démarche de projet**. C'est elle que cette leçon intégratrice met en œuvre, et elle commence ici, avant la moindre ligne de code.

1.1 Qu'est-ce qu'un projet informatique ?

DÉFINITION Projet informatique

Un **projet informatique** est un ensemble d'activités organisées en vue de produire un résultat logiciel ou matériel précis, dans un **délai** donné, avec des **ressources** limitées (humaines, matérielles, financières), et sanctionné par des **livrables** vérifiables.

Trois caractéristiques distinguent un projet d'une simple activité : il a un **objectif** unique et mesurable (une application de gestion des notes, pas « faire de l'informatique »), il est **temporaire** (il a un début et une fin définis, ici 8 semaines), et il consomme des **ressources** identifiées (une équipe de 4 élèves, un serveur, des logiciels libres).

DÉFINITION Livrable

Un **livrable** est un produit concret, vérifiable, remis au client à une étape clé du projet : cahier des charges validé, maquettes d'interface, modèle de base de données, application fonctionnelle, rapport de réalisation, guide d'utilisation. Chaque livrable fait l'objet d'une validation (acceptation ou refus) par le client.

PROPRIÉTÉ Caractéristiques fondamentales d'un projet (Triangle QCD)

Tout projet informatique est soumis à l'équilibre de trois contraintes interdépendantes, souvent appelées le triangle QCD :

- Qualité (fonctionnalités, performance, sécurité, maintenabilité) ;
- Coût (budget, ressources humaines, licences, matériel) ;
- Délai (date de livraison imposée, jalons intermédiaires).

Modifier l'un des sommets impacte obligatoirement les deux autres (ex. : réduire le délai sans augmenter le coût dégrade la qualité).

1.2 Recueillir et analyser les besoins

Avant d'écrire quoi que ce soit, il faut comprendre ce que veut réellement le client, au-delà de ce qu'il exprime spontanément. Trois techniques complémentaires s'imposent :

- **L'entretien direct** : rencontrer le proviseur, le censeur, les enseignants. Poser des questions ouvertes (« décrivez-moi comment vous remplissez un bulletin aujourd'hui, étape par étape ») puis fermées (« l'application doit-elle gérer les absences ? »). Prendre des notes structurées et les faire valider par l'interlocuteur.
- **L'observation sur le terrain** : passer une demi-journée au secrétariat pour observer les documents réels (bulletins manuscrits, registres de notes, formulaires), les gestes métiers, les temps d'attente, les reprises. On découvre ainsi des besoins implicites (ex. : gestion des notes par compétence, arrondi des moyennes) que le client n'a pas su formuler.
- **Le questionnaire ciblé** : distribuer un formulaire court aux enseignants (nombre de classes suivies, maîtrise de l'outil informatique, fonctionnalités souhaitées, horaires de disponibilité) pour recueillir des données quantifiables et prioriser.

L'analyse des besoins consiste ensuite à structurer ce qui a été recueilli en deux catégories distinctes :

DÉFINITION Besoins fonctionnels et non fonctionnels

- Les **besoins fonctionnels** décrivent ce que l'application doit faire : services rendus aux utilisateurs (saisir des notes, calculer des moyennes, éditer des bulletins, lister les élèves d'une classe). Ils se traduisent par des cas d'utilisation (UML).
- Les **besoins non fonctionnels** décrivent comment l'application doit le faire : contraintes de performance (temps de réponse < 2 s), de sécurité (mots de passe hachés, HTTPS en local, droits par profil), d'ergonomie (interface en français, responsive), de portabilité (fonctionnement sur le réseau local sans Internet), de légalité (protection des données élèves selon la loi camerounaise).

1.3 Le cahier des charges

DÉFINITION Cahier des charges (CDC)

Le **cahier des charges** est le document contractuel qui décrit de manière précise, complète et non ambiguë le besoin du client, les fonctionnalités attendues, les contraintes techniques/organisationnelles et les critères d'acceptation du projet. Il est signé par le client (le proviseur) et le chef de projet : toute modification ultérieure du périmètre doit faire l'objet d'un avenant négocié (gestion de la dérive des objectifs ou scope creep).

MÉTHODE Rédiger un cahier des charges en 6 rubriques obligatoires

1. Contexte et problème : décrire la situation actuelle (organisation manuelle, doublons, erreurs, lenteur) et la douleur ressentie par les utilisateurs.
2. Objectifs du projet : formuler des objectifs **SMART** (Spécifiques, Mesurables, Atteignables, Réalistes, Temporels). Ex. : « Générer un bulletin complet pour une classe de 60 élèves en moins de 2 minutes », « Éliminer tout calcul manuel de moyenne ».
3. Fonctionnalités attendues : lister chaque fonction avec un identifiant unique (F1, F2...), une description courte, un acteur déclencheur et une priorité **MoSCoW** (Must/-Should/Could/Won't : Indispensable / Souhaitable / Optionnel / Exclu).
4. Contraintes : techniques (matériel existant : 1 serveur Windows, 20 postes clients ; logiciels imposés : libres), budgétaires (0 FCFA), de délais (8 semaines calendaires), humaines (équipe de 4 élèves, 6 h/semaine), légales (données sensibles mineurs).
5. Livrables et planning : énumérer ce qui sera remis (CDC, diagrammes UML, script SQL, code source, exécutable, rapport, guide) avec les dates de remise (jalons).
6. Critères d'acceptation (recette) : définir des scénarios de test précis, reproductibles, que le client exécutera pour valider la conformité (ex. : « Scénario REC-01 : Je me connecte comme enseignant, je saisis les notes de 30 élèves en Mathématiques, je demande l'impression des bulletins : les 30 bulletins sortent corrects, sans erreur de calcul, en moins de 5 minutes »).

EXEMPLE Extrait du cahier des charges — Application « NotesLBN »

Contexte : Gestion manuelle des notes au Lycée Bilingue de Nkongsamba. Problèmes : lenteur, erreurs de calcul, perte de documents, absence de traçabilité, files d'attente parents. Délai : 8 semaines (semaines 1 à 8 du 2^e trimestre). Budget : 0 FCFA (stack 100% logiciels libres). Hébergement : Serveur local du lycée (intranet), aucune connexion Internet requise pour l'usage courant.

Fonctionnalités prioritaires (extrait) :

Id	Fonctionnalité	Priorité (MoSCoW)
F1	Authentification enseignants (login + mot de passe haché password_hash)	Must (Indispensable)
F2	Saisie / modification notes par classe, matière, séquence, type (DS, Devoir, Examen)	Must
F3	Calcul automatique moyennes (pondérées), rangs, appréciations, décision du conseil de classe	Must
F4	Génération bulletins individuels PDF (logo établissement, mentions légales)	Must
F5	Tableau de bord proviseur : statistiques de réussite par classe, matière, genre	Should (Souhaitable)
F6	Export données vers format officiel MINEDUB (CSV/Excel)	Could (Optionnel)

Critère d'acceptation principal (Recette finale) : Le proviseur, connecté en tant qu'administrateur, saisit les notes d'une classe de 60 élèves sur 5 matières et obtient les 60 bulletins imprimés conformes en moins de 30 minutes, sans aucune intervention manuelle de calcul.

ATTENTION**Coder avant de valider : l'erreur classique fatale**

L'erreur la plus fréquente des débutants est de commencer à programmer dès que l'idée semble claire. Résultat : après trois semaines de travail, le client déclare « ce n'est pas ce que je voulais » ou « il manque la gestion des absences » et tout est à refaire. Règle d'or : aucune ligne de code applicative n'est écrite avant la signature du cahier des charges par le client. Ce document est ton bouclier : il fige le périmètre (scope), protège l'équipe contre les demandes infinies non planifiées, et sert de référence unique pour les tests de recette.

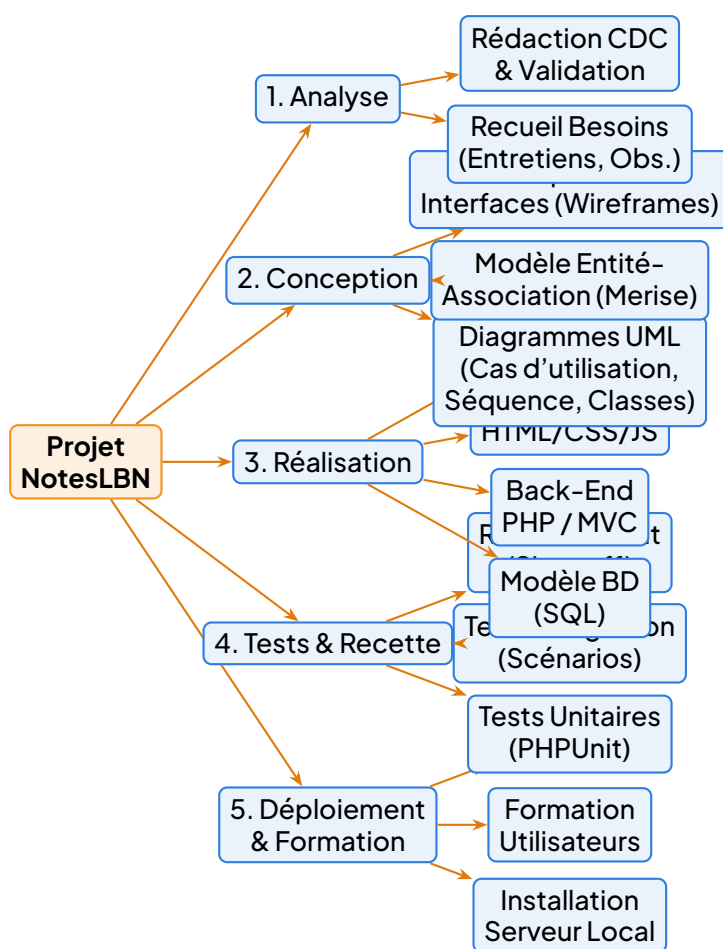
LES AVAIS-TU ?**Pourquoi les projets informatiques échouent-ils ? Le rapport CHAOS**

Les études historiques du Standish Group (rapports CHAOS, publiés depuis 1994) montrent que la majorité des échecs de projets informatiques dans le monde ne provient pas d'erreurs de programmation pure, mais d'une mauvaise définition des besoins : cahier des charges flou, client final jamais consulté, fonctionnalités qui changent en cours de route (scope creep), absence de critères de recette. Un code médiocre qui répond au bon besoin vaut mieux qu'un code parfait qui répond au mauvais. C'est pourquoi les géants du logiciel — et les startups camerounaises qui réussissent (ex. GiftedMom, Kiro'o Games, Maviance) — investissent autant de temps dans la **découverte produit** (Product Discovery) et

l'écoute client que dans le développement lui-même.

1.4 Découper le projet : phases, tâches et WBS (STP)

Un projet de 8 semaines semble immense si on le regarde en bloc. La solution : le **découper** hiérarchiquement. La **WBS** (Work Breakdown Structure), ou **STP** (Structure de Tâches du Projet) en français, organise le projet en **phases** (lots), chaque phase en **tâches** (paquets de travail), chaque tâche étant assez fine pour être confiée à une personne, estimée en durée (quelques jours max) et produire un livrable intermédiaire.



Découpage hiérarchique (WBS / STP) du projet NotesLBN en 5 phases et 14 tâches élémentaires. Chaque feuille de l'arbre est une tâche affectable et estimable.

1.5 Planifier : diagramme de Gantt, jalons, chemin critique et marges

Une fois les tâches listées, il faut répondre à trois questions : combien de temps dure chaque tâche ? dans quel ordre logique (dépendances) ? qui la fait ? Le **diagramme de Gantt** répond

visuellement : chaque tâche est une barre horizontale dont la position et la longueur indiquent ses dates de début et de fin au plus tôt.

MÉTHODE**Construire un diagramme de Gantt rigoureux**

1. Lister les tâches (feuilles de la WBS) avec leur durée estimée réaliste (en jours ouvrés).
2. Identifier les dépendances (précédence) : quelle tâche doit être terminée avant qu'une autre ne commence? (Fin-début : on ne teste pas du code qui n'existe pas; Début-début : on peut commencer la rédaction du guide dès que la maquette d'écran est figée).
3. Calculer les dates au plus tôt : une tâche sans prédécesseur démarre au jour 0 (ou 1); une tâche dépendante démarre à la date de fin au plus tôt de son prédécesseur le plus tardif.
4. Tracer les barres sur l'axe du temps et repérer les tâches pouvant se dérouler en parallèle (ex. : rédaction du guide d'utilisation pendant la phase de tests).
5. Poser les jalons (**jalons** ou milestones) : événements ponctuels (durée nulle) marquant la fin d'une phase ou la remise d'un livrable majeur (losanges sur le diagramme).
6. Identifier le chemin critique : la plus longue chaîne de tâches dépendantes qui détermine la durée minimale du projet. Tout retard sur une tâche du chemin critique reporte la date de fin du projet d'autant.
7. Calculer les marges (float) : pour les tâches non critiques, la marge totale = date au plus tard – date au plus tôt. Elle indique le retard absorbable sans impacter la fin du projet.

DÉFINITION**Jalon, Chemin critique, Marge**

- Un **jalon** (milestone) est un événement ponctuel (durée 0) marquant l'atteinte d'un objectif intermédiaire (ex. : « CDC validé », « Base de données créée », « Version testable livrée »).
- Le **chemin critique** est la séquence de tâches dépendantes dont la somme des durées est maximale ; il détermine la durée minimale incompressible du projet. Une tâche appartient au chemin critique si sa marge totale est nulle.
- La **marge totale** d'une tâche est le délai maximal dont dispose son démarrage pour être retardé sans retarder la fin du projet.

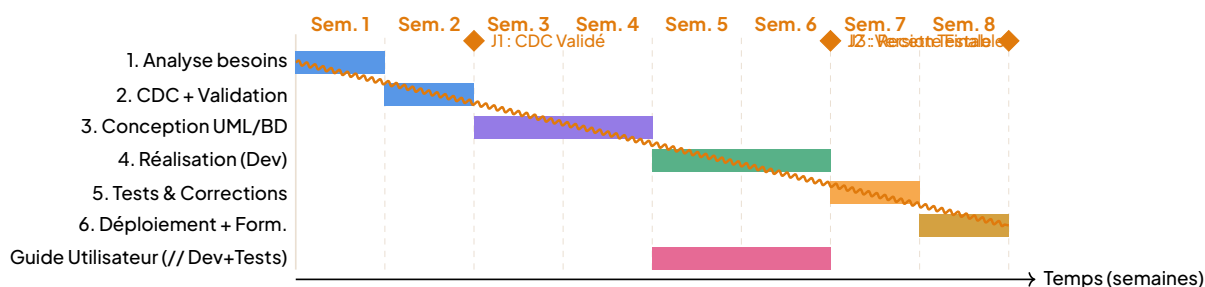


Diagramme de Gantt prévisionnel du projet NotesLBN (8 semaines). Les barres principales forment le chemin critique (marge 0). La tâche « Guide Utilisateur » (rose) s'exécute en parallèle du développement et des tests (marge > 0). Les losanges noirs sont les jalons de validation.

Dans cet exemple, les tâches principales s'enchaînent séquentiellement : le chemin critique couvre l'intégralité des 8 semaines. **Aucun retard n'est absorbable** sur ce chemin. D'où l'importance vitale de **suivre le planning chaque semaine** (réunion d'avancement de 15 min : « Qu'ai-je fait ? Que fais-je ? Quels blocages ? ») et de réagir dès le premier jour de dérive.

1.6 Répartir les rôles et choisir les outils (Stack technique)

Un projet se mène en équipe. Pour un groupe de 4 élèves, la répartition classique des responsabilités (inspirée des méthodes agiles légères) est :

Rôle principal	Responsabilités clés (non exhaustives)
Chef de projet (Scrum Master)	Interface unique client (proviseur), tenue du planning (Gantt), animation réunions hebdo, gestion des risques, arbitrage conflits, validation livrables avant remise.
Architecte / Développeur Back-End	Conception technique (BDD, API, sécurité), programmation PHP (MVC léger), requêtes SQL optimisées, gestion sessions/authentification, génération PDF (TCPDF/Dompdf).
Développeur Front-End / Intégrateur	Maquettes HTML/CSS (responsive, accessibilité), JavaScript (AJAX pour saisie dynamique), intégration templates, tests navigateurs, UX/UI.
Testeur / Documentaliste (QA)	Écriture scénarios de test (cas nominaux, limites, erreurs), exécution tests manuels/automatisés, rapport d'anomalies (ticketing), rédaction Rapport de réalisation + Guide d'utilisation , commentaires code (PHPDoc).

ATTENTION Séparation des pouvoirs : Développer ≠ Tester

Un même élève peut cumuler deux rôles (ex. : Chef de projet + Documentaliste), mais

le rôle de Testeur/QA doit impérativement être tenu par quelqu'un d'autre que le développeur principal du module testé. On ne trouve pas ses propres bugs (aveuglement de confirmation). C'est une règle professionnelle non négociable.

Enfin, les **outils et technologies** doivent découler logiquement du cahier des charges, jamais l'inverse. Contraintes NotesLBN : budget nul, hébergement local Windows, application web dynamique, base de données relationnelle. Conséquences techniques imposées :

- **Serveur local** : **WampServer** (Apache 2.4 + MySQL 8 + PHP 8.x) sur le PC serveur du lycée. Simple, gratuit, standard Windows.
- **SGBD** : **MySQL** (ou MariaDB) — standard, supporté par PHP nativement (`mysqli` / PDO).
- **Langages** : **HTML5**, **CSS3** (Flexbox/Grid, pas de framework lourd), **JavaScript vanilla** (ES6+, `fetch API`), **PHP 8** (POO, PDO, `password_hash/verify`).
- **Modélisation** : **MySQL Workbench** (Merise MLD/MPD, forward engineering) ou papier/crayon + `draw.io` pour UML.
- **Planification** : Tableur (LibreOffice Calc / Excel) pour le Gantt ; `GanttProject` (libre) si disponible.
- **Versionning (obligatoire)** : **Git** (dépôt local sur serveur ou clé USB partagée) — **pas de travail sans Git**. Branches : `main` (stable), `dev`, `feature/*`.

EXERCICE RÉSOLU 1**Calcul de chemin critique et marges sur un mini-projet**

Énoncé. Une équipe réalise un site vitrine pour un cybercafé de Bafoussam. Tâches, durées (jours) et dépendances :

- A — Recueil besoins (3 j) — Sans prédécesseur.
- B — Maquettes (2 j) — Après A.
- C — Développement (5 j) — Après B.
- D — Rédaction guide utilisateur (2 j) — Après B (peut commencer en // de C).
- E — Tests & Recette (2 j) — Après C et D.

Questions : 1) Durée minimale du projet ? 2) Chemin critique ? 3) Marge de la tâche D ?

SOLUTION

Solution détaillée. Calcul des dates au plus tôt (DP / FP) :

A : DP = 0, FP = 3

B : DP = 3, FP = 5

C : DP = 5, FP = 10

D : DP = 5, FP = 7

E : DP = $\max(\text{FP}_C, \text{FP}_D) = 10$, FP = 12

1) Durée minimale : 12 jours. (Fin de E au jour 12). 2) Chemin critique : $A \rightarrow B \rightarrow C \rightarrow E$ (somme = $3+2+5+2 = 12$ j). Marge totale = 0 pour A, B, C, E. 3) Marge de D : Date au plus tard de D = DP de E = 10. Marge = DLT – DPT = $10 - 5 = 5$ jours. D peut glisser de 5 jours sans retarder le projet. C'est une tâche non critique à surveiller pour ne pas qu'elle dérive au-delà de sa marge.

EXERCICE 1 Mise en situation : Projet NotesLBN — Compléments

Le projet NotesLBN intègre deux tâches supplémentaires :

- T7 — « Formation des enseignants utilisateurs » : durée 2 jours, ne peut commencer qu'après la fin des tests (tâche 5).
- T8 — « Rédaction du guide d'utilisation complet » : durée 4 jours, peut démarrer dès la fin du développement (tâche 4).

Le planning initial (Gantt ci-dessus) prévoit 1 semaine (5 jours) pour le déploiement+formation (semaine 8).

1. Ces deux tâches modifient-elles la date de fin du projet (semaine 8)? Justifie en calculant les dates au plus tôt.
2. Rédige les rubriques « Contexte et problème » et « Objectifs (SMART) » du cahier des charges de NotesLBN en 12 lignes maximum au total.

CORRIGÉ Exercice — Projet NotesLBN

1) Analyse d'ordonnancement : Reprenons les fins au plus tôt (FP) du Gantt principal (en jours, 5 j/sem) : T4 (Dev) FP = 30 j (fin sem 6). T5 (Tests) FP = 35 j (fin sem 7). T6 (Déploiement) FP = 40 j (fin sem 8).

- T8 (Guide, 4 j) : DP = FP(T4) = 30 j. FP = 34 j. Se termine pendant la semaine 7 (tests), bien avant la fin du projet. Marge = $40 - 34 = 6$ j. Ne retarde pas le projet.
- T7 (Formation, 2 j) : DP = FP(T5) = 35 j. FP = 37 j. Se termine pendant la semaine 8 (déploiement prévu 35–40 j). Ne retarde pas le projet si faite en parallèle du déploiement technique.

Conclusion : La date de fin (semaine 8) n'est pas modifiée si T7 et T6 s'exécutent en parallèle (ex. : matin déploiement technique, après-midi formation). Le chef de projet doit l'organiser ainsi.

2) Extrait CDC (exemple attendu) : Contexte : Au Lycée Bilingue de Nkongsamba, la gestion des notes est entièrement manuelle : saisie sur registres papier, calculs de moyennes à la calculatrice, rédaction manuscrite des bulletins. Cela engendre des erreurs fréquentes, une lenteur extrême (semaines pour une promotion), des pertes de documents et l'impossibilité pour les parents d'avoir un suivi régulier. Objectifs SMART : (1) Automatiser la saisie, le calcul pondéré des moyennes, les rangs et les décisions pour 100% des élèves. (2) Générer des bulletins officiels PDF conformes au format MINEDUB en < 2 min/-classe. (3) Garantir l'intégrité et la confidentialité des notes (authentification forte, droits par profil, pas d'accès Internet). (4) Livrer l'application opérationnelle sur le LAN du lycée pour la fin du 2^e trimestre (8 semaines).

À RETENIR**L'essentiel de la section — Lancement & Planification**

- Un projet informatique = Objectif SMART + Délai fixe + Ressources limitées + Livrables validés.
- Le **Cahier des Charges (CDC)** signé est le contrat : contexte, objectifs, fonctionnalités (priorisées MoSCoW), contraintes, livrables, critères de recette. Zero code avant signature.
- Besoins fonctionnels (quoi faire) vs non fonctionnels (comment : sécurité, perf, legal).
- **WBS / STP** : découpage hiérarchique en tâches affectables, estimables, livrables.
- **Diagramme de Gantt** : ordonnancement temporel, dépendances, jalons, chemin critique (marge 0 = surveillance quotidienne), marges (tampons).
- Rôles distincts (Chef, Dev Back, Dev Front, QA/Doc) — Séparation Dev/Test obligatoire.
- Stack technique imposée par le CDC : WampServer / MySQL / PHP 8 / HTML-CSS-JS / Git.

Prochaine étape : Conception du système d'information avec UML (Cas d'utilisation, Séquence, Classes) et modélisation de la base de données (Merise MCD/MLD/MPD).

2

CONCEPTION DU SYSTÈME D'INFORMATION AVEC UML ET LA BASE DE DONNÉES

Dans la section précédente, nous avons lancé notre projet intégrateur : l'application **NotesLy-**

cée, destinée à la gestion des notes d'un lycée camerounais. Nous avons rédigé un cahier des charges précis (qui fait quoi, quelles fonctionnalités, quelles contraintes) et planifié le travail. Nous savons donc désormais quoi faire. Mais entre le cahier des charges et le premier fichier PHP, il manque une étape décisive : la **conception**. C'est elle qui répond à la question comment le faire.

2.1 Le rôle de la conception : du « quoi faire » au « comment le faire »

Imagine un entrepreneur de Bafoussam qui recevrait une commande de construction d'une école. Se lancerait-il immédiatement dans le coulage du béton ? Certainement pas : il commencerait par dessiner des plans (plan de masse, plan des salles, plan électrique). En informatique, c'est exactement pareil : coder sans conception, c'est construire sans plan. On obtient des programmes incohérents, des bases de données redondantes, et des bugs découverts trop tard, quand ils coûtent très cher à corriger.

DÉFINITION Conception d'un système d'information

La **conception** est l'étape du cycle de vie d'un projet informatique qui transforme les besoins exprimés dans le cahier des charges (le quoi) en modèles précis décrivant la structure et le comportement de la future application (le comment). On utilise pour cela le langage de modélisation **UML** (Unified Modeling Language), un langage graphique normalisé, indépendant de tout langage de programmation.

UML propose de nombreux diagrammes. Pour notre projet NotesLycée, trois d'entre eux suffisent et se complètent :

- le **diagramme de cas d'utilisation** : qui utilise le système, et pour quoi faire (vue externe) ;
- le **diagramme de classes** : quelles données sont manipulées et comment elles sont liées (vue structurelle) ;
- le **diagramme de séquence** : comment les objets collaborent dans le temps pour réaliser un scénario (vue dynamique).

2.2 Le diagramme de cas d'utilisation : qui fait quoi ?

2.3 Acteurs et cas d'utilisation

DÉFINITION Acteur et cas d'utilisation

Un **acteur** est un rôle joué par une entité externe au système (personne ou autre système) qui interagit avec lui. Un **cas d'utilisation** (use case) est une fonctionnalité complète du système, rendant un service observable à un acteur. On le nomme par un verbe à l'infinitif : « Saisir une note », « Consulter un bulletin ».

Reprenons le cahier des charges de NotesLycée. Trois acteurs apparaissent clairement : l'**enseignant**

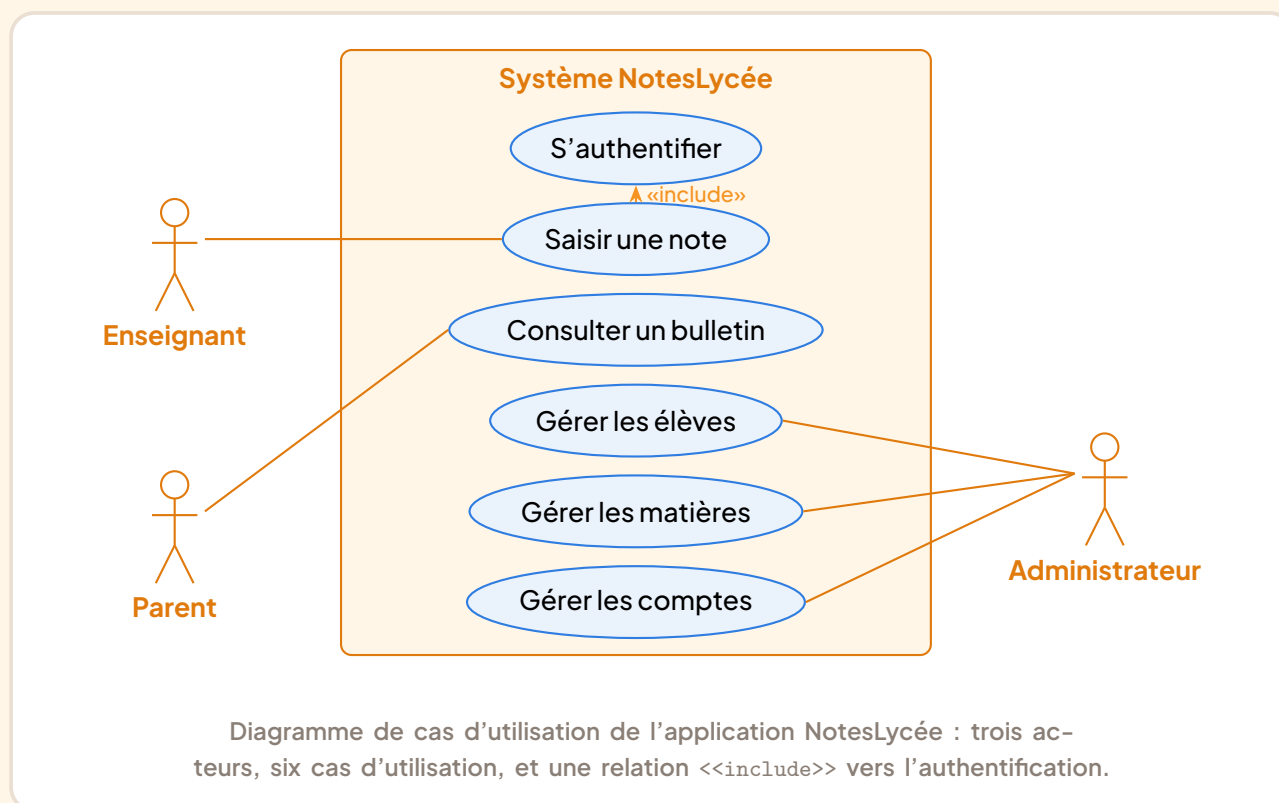
(il saisit les notes), l'**administrateur** (il gère les élèves, les classes, les matières et les comptes) et le **parent** (il consulte les résultats de son enfant en ligne). Chaque acteur est relié aux cas d'utilisation qu'il déclenche.

ATTENTION Acteur ≠ utilisateur interne du programme

L'erreur la plus fréquente est de confondre **acteur** et « utilisateur du programme ». Un acteur est un rôle externe au système, jamais une partie du logiciel. Ainsi, « la base de données » ou « le script PHP de vérification du mot de passe » ne sont pas des acteurs : ce sont des composants internes. Symétriquement, une même personne physique peut jouer plusieurs rôles (un enseignant peut aussi être parent d'élève) : on modélise alors deux acteurs distincts, car les rôles sont distincts.

MÉTHODE Construire un diagramme de cas d'utilisation en 4 étapes

1. Identifier les acteurs : relire le cahier des charges et repérer chaque rôle externe qui interagit avec le système (qui saisit ? qui consulte ? qui administre ?).
2. Identifier les cas d'utilisation : pour chaque acteur, lister les services qu'il attend du système, formulés par un verbe à l'infinitif.
3. Dessiner : placer les acteurs (bonshommes) à l'extérieur d'un rectangle représentant la frontière du système, et les cas d'utilisation (ellipses) à l'intérieur ; relier chaque acteur à ses cas.
4. Ajouter les relations : <<include>> si un cas en utilise obligatoirement un autre (ex. « Saisir une note » inclut « S'authentifier »), <<extend>> pour un cas optionnel, et la généralisation entre acteurs si un acteur hérite des droits d'un autre.



La relation <<include>> mérite une remarque : saisir une note exige d'être authentifié. Plutôt que de répéter « vérifier le mot de passe » dans chaque cas, on factorise ce comportement dans le cas « S'authentifier ». C'est le même réflexe que la factorisation de code en fonctions.

2.4 Le diagramme de classes : structurer les données

Le diagramme de cas d'utilisation dit qui fait quoi, mais pas sur quelles données. C'est le rôle du **diagramme de classes**. Une **classe** représente un ensemble d'objets partageant les mêmes **attributs** (propriétés) et les mêmes opérations. Les classes sont reliées par des **associations** portant des **multiplicités** (ou cardinalités) : combien d'objets d'une classe sont liés à un objet de l'autre classe.

Pour NotesLycée, le cahier des charges fait apparaître quatre entités centrales : **Élève**, **Classe**, **Matière** et **Note**. Réfléchissons aux liens :

- une classe regroupe plusieurs élèves, mais un élève appartient à une seule classe : association **1-n** ;
- une note concerne un seul élève et une seule matière, mais un élève reçoit plusieurs notes et une matière donne lieu à plusieurs notes : la classe **Note** est le point de rencontre de deux associations **1-n**.

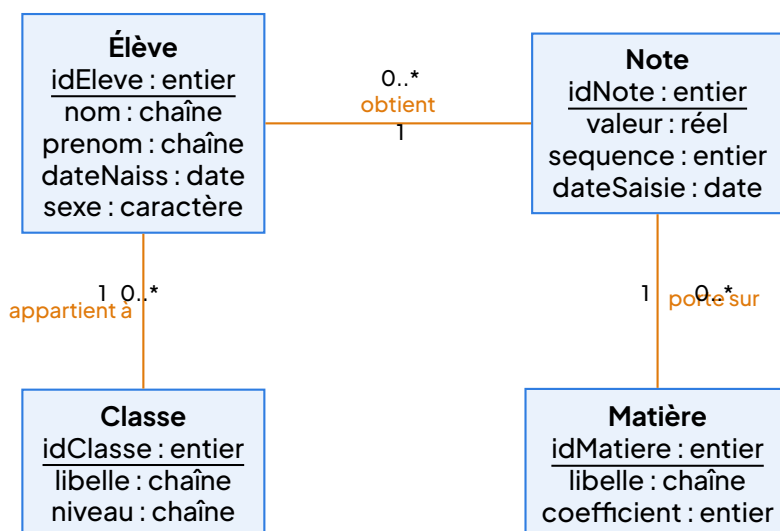


Diagramme de classes de NotesLycée : un élève obtient de 0 à plusieurs notes (0..*), chaque note concerne exactement un élève (1) et une matière (1); chaque élève appartient à une seule classe (1), une classe contient 0..* élèves.

ATTENTION Bien lire les multiplicités

La multiplicité se lit en face de la classe cible : « un élève obtient 0..* notes » car le 0..* est du côté de Note, et « chaque note est obtenue par exactement 1 élève » car le 1 est du côté d'Élève. Inverser les cardinalités est l'erreur classique du baccalauréat : prends l'habitude de formuler la phrase à voix haute dans les deux sens avant de valider ton diagramme.

2.5 Le diagramme de séquence : le scénario « saisir une note »

Le diagramme de **séquence** décrit, pour un scénario précis, l'ordre chronologique des messages échangés entre les acteurs et les objets du système. Le temps s'écoule de haut en bas; chaque participant possède une ligne de vie verticale.

Scénario : l'enseignant M. Kamdem saisit la note 15/20 de l'élève Ngo Bassa en Mathématiques, séquence 2.

MÉTHODE Étapes du scénario « Saisir une note »

1. L'enseignant soumet le formulaire de saisie (élève, matière, séquence, valeur) au Contrôleur PHP.
2. Le contrôleur vérifie la session (l'enseignant est-il authentifié ?) et valide les données ($0 \leq \text{valeur} \leq 20$, séquence entre 1 et 6).

3. Le contrôleur demande à l'objet Note de s'enregistrer : exécution d'une requête `INSERT` vers la Base MySQL.
4. La base confirme l'insertion (ou renvoie une erreur d'intégrité) ; le contrôleur affiche un message de succès (ou d'erreur) à l'enseignant.

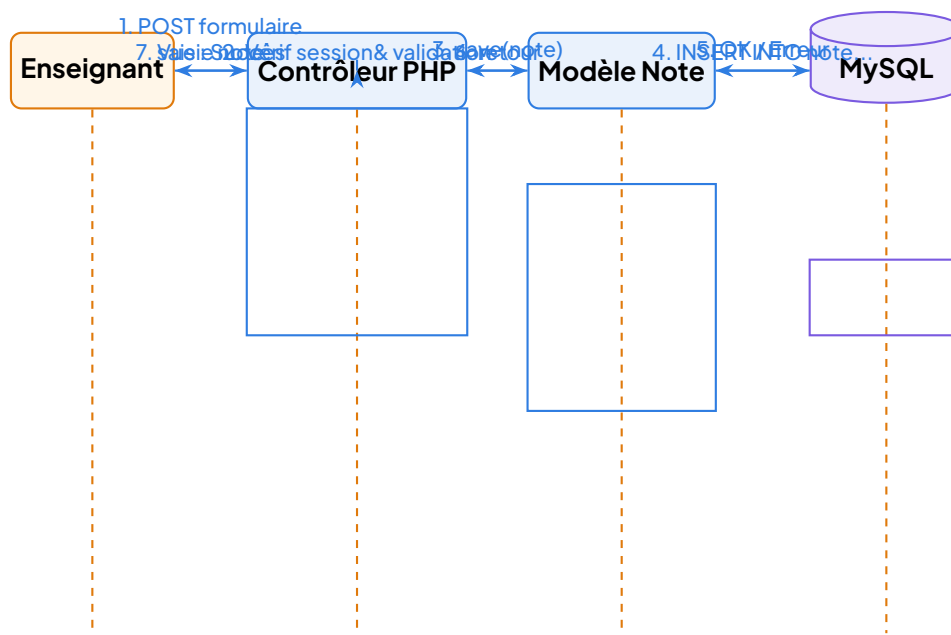


Diagramme de séquence : scénario « Saisir une note ». Les boîtes colorées sur les lignes de vie représentent les périodes d'activation (exécution).

Ce diagramme est précieux car il révèle les vérifications à programmer : sans lui, on oublie souvent le contrôle de session ou la validation de la plage de la note — deux failles classiques (accès non autorisé, notes incohérentes comme 47/20).

2.6 Du diagramme de classes au MCD puis au MLD

Nous avons maintenant un modèle objet. Or notre application utilisera une base de données relationnelle (MySQL). Il faut donc traduire le diagramme de classes en **MCD** (Modèle Conceptuel de Données : entités, attributs, relations, cardinalités — quasiment une transcription du diagramme de classes), puis en **MLD** (Modèle Logique de Données : tables et clés). Les règles de passage sont mécaniques.

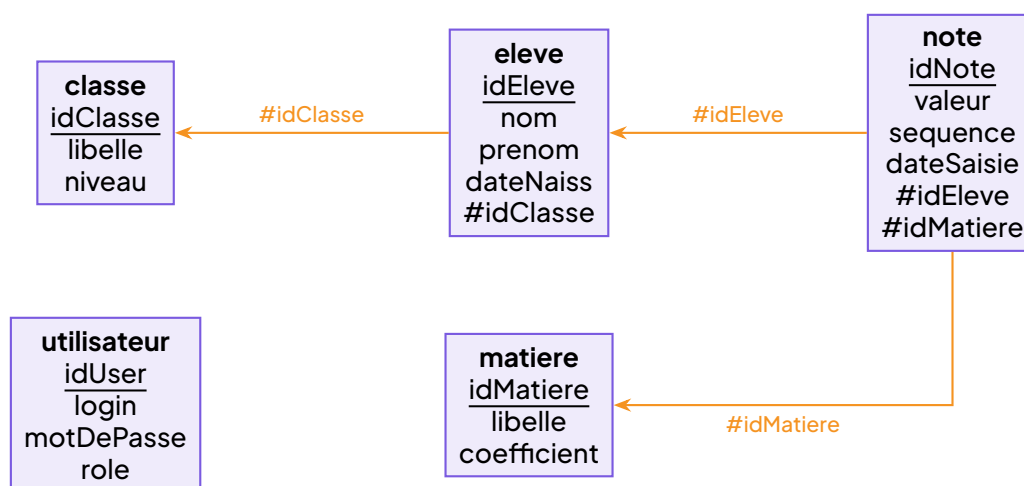
PROPRIÉTÉ Règles de passage du diagramme de classes au MLD (admises)

1. Toute classe devient une table ; chaque attribut devient une colonne ; l'attribut iden-

tifiant devient la **clé primaire** (soulignée).

2. Toute association de type 1-n se traduit par une **clé étrangère** : la clé primaire du côté « 1 » migre comme colonne dans la table du côté « n ».
3. Toute association de type n-n (plusieurs-plusieurs) devient une table associative dont la clé primaire est composée des clés primaires des deux tables reliées ; les éventuels attributs de l'association deviennent des colonnes de cette table.

Appliquons ces règles à NotesLycée. Les quatre classes donnent quatre tables. L'association 1-n entre **Classe** et **Élève** fait migrer `idClasse` dans la table `eleve`. La classe **Note**, déjà reliée en 1-n à **Élève** et à **Matière**, reçoit deux clés étrangères : `idEleve` et `idMatiere`. Ajoutons enfin une table `utilisateur` pour l'authentification (enseignants, administrateurs, parents). Nous obtenons le MLD à cinq tables :



MLD de la base `notes_lycee` : les clés primaires sont soulignées, les clés étrangères préfixées par # et matérialisées par des flèches vers la table référencée.

2.7 Création de la base MySQL : tables, clés et contraintes d'intégrité

Le MLD se traduit directement en SQL. Trois types de **contraintes d'intégrité** garantissent la cohérence des données :

- l'**intégrité d'entité** : la clé primaire (PRIMARY KEY) est unique et non nulle ;
- l'**intégrité référentielle** : une clé étrangère (FOREIGN KEY ... REFERENCES) ne peut référencer qu'une ligne existante ;
- l'**intégrité de domaine** : chaque colonne respecte son type et ses contraintes (NOT NULL, CHECK, UNIQUE).

EXEMPLE Script SQL complet de la base notes_lycee

```
-- Creation et selection de la base de donnees
CREATE DATABASE IF NOT EXISTS notes_lycee
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE notes_lycee;

-- Table des classes (aucune cle etrangere : a creer en premier)
CREATE TABLE classe (
  idClasse INT AUTO_INCREMENT PRIMARY KEY,
  libelle VARCHAR(20) NOT NULL, -- ex. 'Tle TI 1'
  niveau VARCHAR(10) NOT NULL -- ex. 'Terminale'
) ENGINE = InnoDB;

-- Table des eleves : cle etrangere vers classe (association 1-n)
CREATE TABLE eleve (
  idEleve INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  prenom VARCHAR(50) NOT NULL,
  dateNaiss DATE NOT NULL,
  sexe CHAR(1) NOT NULL CHECK (sexe IN ('M', 'F')),
  idClasse INT NOT NULL,
  FOREIGN KEY (idClasse) REFERENCES classe(idClasse)
) ENGINE = InnoDB;

-- Table des matieres
CREATE TABLE matiere (
  idMatiere INT AUTO_INCREMENT PRIMARY KEY,
  libelle VARCHAR(40) NOT NULL, -- ex. 'Mathematiques'
  coefficient INT NOT NULL CHECK (coefficient >= 1)
) ENGINE = InnoDB;

-- Table des notes : deux cles etrangeres (eleve, matiere)
CREATE TABLE note (
  idNote INT AUTO_INCREMENT PRIMARY KEY,
  valeur DECIMAL(4,2) NOT NULL CHECK (valeur BETWEEN 0 AND 20),
  sequence INT NOT NULL CHECK (sequence BETWEEN 1 AND 6),
  dateSaisie DATE NOT NULL,
  idEleve INT NOT NULL,
  idMatiere INT NOT NULL,
  FOREIGN KEY (idEleve) REFERENCES eleve(idEleve),
  FOREIGN KEY (idMatiere) REFERENCES matiere(idMatiere),
  UNIQUE (idEleve, idMatiere, sequence) -- une seule note par triplet
) ENGINE = InnoDB;

-- Table des utilisateurs (authentification)
CREATE TABLE utilisateur (
  idUser INT AUTO_INCREMENT PRIMARY KEY,
  login VARCHAR(30) NOT NULL UNIQUE,
  motDePasse VARCHAR(255) NOT NULL, -- empreinte hachee, jamais en clair
  role ENUM('admin', 'enseignant', 'parent') NOT NULL
) ENGINE = InnoDB;
```

ATTENTION Pièges fréquents en SQL

- **Ordre de création** : on crée d'abord les tables référencées (`classe`, `eleve`, `matiere`) avant celles qui contiennent les clés étrangères (`note`), sinon MySQL rejette la contrainte.
- **Moteur InnoDB** : sans `ENGINE = InnoDB`, les clés étrangères ne sont pas vérifiées sous MySQL.
- **Mots de passe** : ne jamais stocker un mot de passe en clair ; on stocke une empreinte produite par `password_hash()` en PHP, d'où le `VARCHAR(255)`.
- **Contrainte UNIQUE composée** : elle empêche la saisie en double d'une même note (même élève, même matière, même séquence) — un bug métier très fréquent.

2.8 Le dictionnaire des données : garder la cohérence avec le cahier des charges

Le **dictionnaire des données** est le tableau de référence qui décrit chaque donnée de la base : nom, type, taille, contrainte, signification. Il sert de contrat entre les concepteurs et les développeurs, et permet de vérifier que toute donnée exigée par le cahier des charges existe bien dans le MLD — et réciproquement, qu'aucune colonne n'est superflue.

Donnée	Type	Contrainte	Signification
idEleve	entier	PK, auto-incrément	Identifiant unique de l'élève
nom	chaîne(50)	NOT NULL	Nom de famille de l'élève
valeur	décimal(4,2)	0 à 20	Note sur 20, ex. 15,50
sequence	entier	1 à 6	Séquence d'évaluation
coefficient	entier	≥ 1	Coefficient de la matière
role	énuméré	admin / enseignant / parent	Profil de l'utilisateur

EXERCICE RÉSOLU 2 Vérifier la cohérence cahier des charges / MLD

Énoncé. Le cahier des charges de NotesLycée exige : « le parent consulte le bulletin de son enfant, avec la moyenne pondérée par séquence ». (a) Montrer que le MLD permet de calculer cette moyenne. (b) Écrire la requête SQL donnant la moyenne de la séquence 2 de l'élève d'identifiant 7.

SOLUTION

Solution détaillée. (a) La moyenne pondérée exige trois données : les `valeur` des notes (table `note`), la `sequence` (table `note`) et le `coefficient` de chaque matière (table `matiere`). La clé étrangère `note.idMatiere` permet la jointure entre les deux tables : toutes les données nécessaires sont présentes, le MLD est cohérent avec cette exigence. La formule

est :

$$\text{Moyenne} = \frac{\sum_i \text{valeur}_i \times \text{coefficient}_i}{\sum_i \text{coefficient}_i}$$

(b) La requête s'écrit avec une jointure et les fonctions d'agrégation SUM :

```
SELECT SUM(n.valeur * m.coefficient) / SUM(m.coefficient) AS moyenne
FROM note n
JOIN matiere m ON n.idMatiere = m.idMatiere
WHERE n.idEleve = 7 AND n.sequence = 2;
```

On vérifie mentalement : si l'élève a 12/20 en Maths (coef 4) et 16/20 en Physique (coef 2), la requête calcule $(12 \times 4 + 16 \times 2) / (4 + 2) = 80 / 6 \approx 13,33$. Le résultat est correct : le modèle répond au besoin du cahier des charges.

À RETENIR L'essentiel de la section

- La conception transforme le « quoi faire » (cahier des charges) en « comment le faire » (modèles UML).
- Le diagramme de cas d'utilisation identifie les acteurs (rôles externes) et les services du système ; le diagramme de classes structure les données ; le diagramme de séquence décrit un scénario dans le temps.
- Passage au MLD : une classe → une table ; une association 1-n → une clé étrangère ; une association n-n → une table associative.
- Les contraintes d'intégrité (clé primaire, clé étrangère, CHECK, UNIQUE) protègent la cohérence de la base dès sa création.
- Le dictionnaire des données garantit la traçabilité entre le cahier des charges et la base.

Notre système d'information est désormais entièrement modélisé : nous savons qui utilise NotesLycée, quelles données il manipule, comment elles sont reliées, et la base MySQL est prête. La prochaine section passera à la **réalisation** : développer les pages PHP dynamiques qui s'appuieront sur cette architecture.

3 RÉALISATION : DÉVELOPPEMENT DE L'APPLICATION WEB DYNAMIQUE

Dans la section précédente, nous avons conçu notre système d'information : diagrammes de cas d'utilisation et de classes UML, modèle entité-association, schéma relationnel et script SQL de création de la base `notes_college`. Tout est prêt sur le papier. Il est temps de passer à la **réalisation** : transformer cette conception en une application web dynamique qui fonctionne réel-

lement, accessible depuis le navigateur des enseignants et des parents de notre collège. C'est l'étape où l'on écrit le code — et où l'on découvre que bien organiser ce code est aussi important que bien l'écrire.

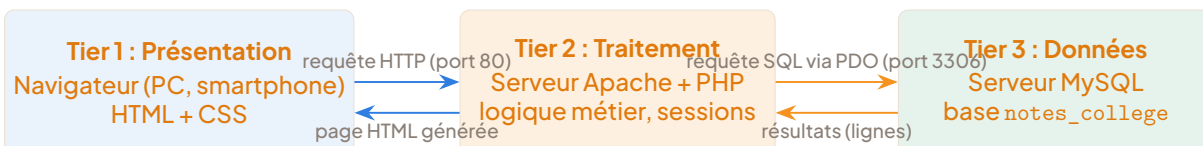
3.1 Architecture client/serveur et modèle 3 tiers

Intuition. Quand un parent consulte le bulletin de son enfant sur son téléphone, trois acteurs travaillent ensemble sans qu'il le sache : son navigateur affiche une belle page, un programme sur un serveur distant calcule les moyennes, et une base de données fournit les notes. Séparer ces trois rôles dans trois « couches » distinctes rend l'application plus claire, plus sûre et plus facile à faire évoluer.

DÉFINITION Application web dynamique et architecture 3 tiers

Une **application web dynamique** est une application accessible via un navigateur web, dont les pages sont générées à la demande par un programme exécuté sur le serveur, en fonction de l'utilisateur et des données du moment (contrairement à un site statique dont les pages sont figées). Elle suit généralement une **architecture 3 tiers** :

1. Tier présentation : ce que voit l'utilisateur — pages HTML mises en forme par CSS, exécutées dans le navigateur ;
2. Tier traitement : la logique métier — scripts PHP exécutés par le serveur web (Apache) : vérifier un mot de passe, calculer une moyenne, décider qui a le droit de voir quoi ;
3. Tier données : le stockage persistant — le SGBD MySQL qui conserve élèves, notes, utilisateurs.



Architecture 3 tiers : le navigateur ne parle jamais directement à MySQL ; PHP fait office d'intermédiaire contrôlé.

PROPRIÉTÉ Règle d'or de la communication entre tiers

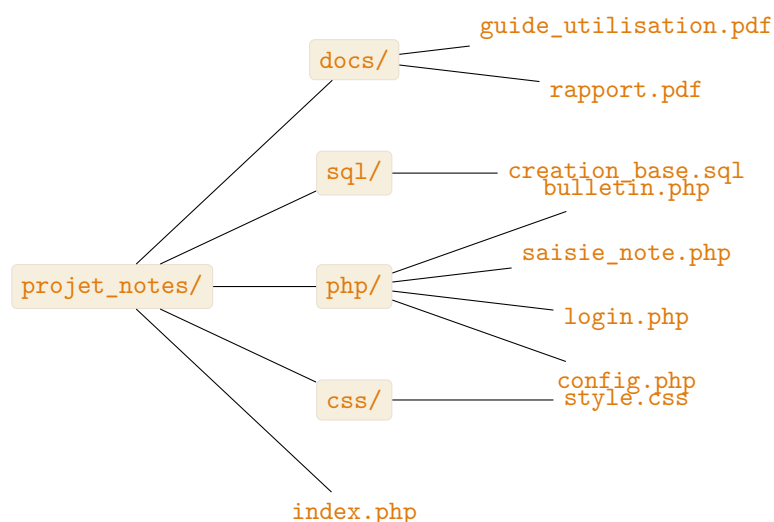
Le client (navigateur) ne communique jamais directement avec la base de données. Toute demande de données passe par le tier traitement (PHP), qui vérifie l'identité et les droits de l'utilisateur avant d'interroger MySQL. C'est ce qui empêche un visiteur de lire toute la base en tapant une URL astucieuse. En pratique, le navigateur dialogue avec Apache en HTTP (port 80 par défaut), et PHP dialogue avec MySQL sur son port propre (3306 par défaut) : ce sont deux protocoles et deux canaux distincts.

LESAVAIS-TU ? Le Cameroun fonctionne en 3 tiers

L'architecture 3 tiers est celle de la quasi-totalité des applications en ligne utilisées au Cameroun : les plateformes de résultats universitaires, les portails de `impots.cm`, et même les services de mobile money (MTN MoMo, Orange Money) : ton téléphone affiche l'interface (tier 1), des serveurs applicatifs vérifient ton code PIN et exécutent le transfert (tier 2), et des bases de données sécurisées enregistrent les soldes (tier 3). Maîtriser ce modèle, c'est comprendre l'architecture de l'économie numérique camerounaise.

3.2 Organisation du code : fichiers, pages dynamiques et sessions

Intuition. Un projet où tout le code est entassé dans un seul fichier devient vite ingérable, surtout en équipe. On sépare donc les fichiers par rôle, comme on sépare les rayons d'une bibliothèque : la mise en forme d'un côté, la logique de l'autre, la documentation à part.



Arborescence du projet : chaque dossier a un rôle précis, ce qui facilite le travail en équipe.

Le fichier `config.php` joue un rôle central : il contient la connexion à la base et est inclus en tête de chaque page grâce à `require`. Ainsi, si le mot de passe MySQL change, on ne modifie qu'un **seul** fichier. On utilise `require` (et non `include`) car la page ne peut pas fonctionner sans la base : si le fichier est absent, `require` provoque une erreur fatale qui arrête le script, là où `include` se contenterait d'un avertissement et laisserait la page s'exécuter dangereusement.

Reste un problème : le protocole HTTP est sans état — le serveur oublie tout entre deux pages. Comment se souvenir que l'utilisateur connecté est « Mme Ngo Bell, enseignante » ? Grâce aux **sessions** : au moment de la connexion, PHP crée un espace mémoire côté serveur, identifié par un cookie côté client. Chaque page protégée commence alors par :

```

<?php
session_start();           // ouvre (ou reprend) la session
// Barrière de sécurité : pas de session = retour au login
if (!isset($_SESSION['id_user'])) {

```

```
header('Location: login.php');
exit(); // indispensable : sans exit, le reste de la page
        // serait quand meme execute et envoie !
}
?>
```

ATTENTION Piège classique : oublier `exit()` après une redirection

La fonction `header('Location: ...')` ne fait qu'ajouter un en-tête HTTP à la réponse : elle n'arrête pas l'exécution du script. Sans `exit()`, le code qui suit (affichage des notes, par exemple) est exécuté et transmis au navigateur, même pour un visiteur non connecté. Toute redirection de sécurité doit donc être immédiatement suivie de `exit()` ;.

3.3 Connexion PHP–MySQL et requêtes préparées

Intuition. PHP et MySQL sont deux programmes distincts : il faut établir un « pont » entre eux. Nous utilisons l'extension **PDO** (PHP Data Objects), moderne et sécurisée.

MÉTHODE Connecter une page PHP à MySQL en 4 lignes types

Dans `php/config.php` :

```
<?php
// 1. Définir le DSN : type de SGBD, hôte, nom de la base
$dsn = 'mysql:host=localhost;dbname=notes_college;charset=utf8mb4';
// 2. Créer la connexion PDO avec identifiant et mot de passe
$pdo = new PDO($dsn, 'root', '');
// 3. Demander à PDO de signaler les erreurs par des exceptions
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
// 4. Récupérer les résultats sous forme de tableaux associatifs
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
?>
```

Ensuite, chaque page commence par `require 'config.php'` ; et utilise la variable `$pdo`. Le couple `root / mot de passe vide` est la configuration par défaut de WampServer en local : acceptable pour le développement, mais à changer impérativement sur un serveur en production.

Une fois connecté, on interroge la base. Mais attention : une requête SQL construite en collant directement une donnée saisie par l'utilisateur est une porte ouverte à l'**injection SQL** (un attaquant tape du code SQL dans le formulaire et videra votre base). La parade systématique : les **requêtes préparées**, où la donnée est transmise séparément de la requête via des marqueurs `? OU :nom`.

ATTENTION Sécurité : contrôler toute donnée venant d'un formulaire

Toute donnée issue de `$_GET`, `$_POST` ou d'un cookie est hostile par principe : elle peut avoir été modifiée par un utilisateur malveillant. Avant de l'utiliser :

1. valider son type et sa plage (une note est un nombre entre 0 et 20);
2. jamais la concaténer dans une requête SQL : utiliser `prepare()` + `execute()` ;
3. à l'affichage, l'échapper avec `htmlspecialchars()` pour éviter les attaques XSS (injection de code JavaScript dans la page affichée).

ATTENTION Erreur fréquente : mots de passe en clair

Ne stockez jamais un mot de passe en clair dans la base : si la base fuit, tous les comptes sont compromis. Il faut les **hacher** : à l'inscription, enregistrer `password_hash($mdp, PASSWORD_DEFAULT)` ; à la connexion, comparer avec `password_verify($mdp_saisi, $hash_bdd)`. Le hachage est à sens unique : on ne peut pas « déchiffrer » un hash, on ne peut que hacher la saisie et comparer les deux empreintes.

3.4 Réalisation des fonctionnalités prioritaires du cahier des charges

Le cahier des charges (section 1) a classé les fonctionnalités par priorité. On développe dans l'ordre : **authentification** d'abord (tout le reste en dépend), puis **saisie des notes**, **consultation**, et enfin **édition des bulletins**.

1. Authentification. Le script `login.php` reçoit identifiant et mot de passe, récupère l'utilisateur en base, vérifie le hash, puis ouvre la session en mémorisant le profil :

```
<?php
session_start();
require 'config.php';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Requete preparee : l'identifiant saisi n'est jamais
    // interprete comme du SQL
    $stmt = $pdo->prepare(
        'SELECT id_user, mdp_hash, profil FROM utilisateurs
        WHERE identifiant = ?'
    );
    $stmt->execute([$_POST['identifiant']]);
    $user = $stmt->fetch();
    if ($user && password_verify($_POST['mdp'], $user['mdp_hash'])) {
        $_SESSION['id_user'] = $user['id_user'];
        $_SESSION['profil'] = $user['profil']; // admin, enseignant, parent
        header('Location : index.php');
        exit();
    }
    $erreur = 'Identifiant ou mot de passe incorrect.';
}
?>
<!-- Ici : le formulaire HTML de connexion, qui affiche
```

\$erreur (echappée avec [htmlspecialchars](#)) si elle existe -->

Remarque le message d'erreur volontairement vague : on ne précise jamais si c'est l'identifiant ou le mot de passe qui est faux, pour ne pas confirmer à un attaquant qu'un compte existe.

2. Saisie des notes. C'est la fonctionnalité cœur du métier. Détaillons-la entièrement.

EXEMPLE Exemple commenté : saisie sécurisée d'une note

Le formulaire (saisie_note.php, partie HTML) :

```
<form method="post" action="saisie_note.php">
  <label>Eleve :</label>
  <select name="id_eleve">
    <option value="12">ABENA Marie (Tle TI)</option>
  </select>
  <label>Matiere :</label>
  <select name="id_matiere">
    <option value="3">Informatique</option>
  </select>
  <label>Note / 20 :</label>
  <input type="number" name="note" min="0" max="20" step="0.5" required>
  <button type="submit">Enregistrer</button>
</form>
```

Le traitement PHP au sommet du même fichier :

```
<?php
session_start();
require 'config.php';
// Controle d'accès : connecte ET enseignant
if (!isset($_SESSION['profil']) || $_SESSION['profil'] !== 'enseignant') {
    die('Acces refuse.');
```

```
}
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // 1. Validation : la note doit etre un nombre entre 0 et 20
    $note = filter_input(INPUT_POST, 'note', FILTER_VALIDATE_FLOAT);
    $id_eleve = (int) $_POST['id_eleve'];
    $id_matiere = (int) $_POST['id_matiere'];
    if ($note === false || $note === null || $note < 0 || $note > 20) {
        $message = 'Note invalide : saisir un nombre entre 0 et 20.';
    } else {
        // 2. Insertion par requete preparee (anti-injection SQL)
        $stmt = $pdo->prepare(
            'INSERT INTO notes (id_eleve, id_matiere, valeur, date_saisie)
            VALUES (?, ?, ?, CURDATE())'
        );
        $stmt->execute([$id_eleve, $id_matiere, $note]);
        $message = 'Note enregistree avec succes.';
    }
}
?>
```

ATTENTION La validation HTML ne suffit jamais

Les attributs `min`, `max` et `required` du formulaire sont une aide à la saisie, pas une sécurité : un utilisateur peut les contourner en modifiant le code HTML de la page dans son navigateur ou en envoyant la requête HTTP à la main. C'est pourquoi la validation est systématiquement refaite côté serveur en PHP (`filter_input`, tests de plage).

3. Consultation et 4. édition des bulletins. La consultation est une requête `SELECT` préparée dont les résultats sont affichés dans un tableau HTML (chaque valeur passée par `htmlspecialchars()`). Le bulletin est une page `bulletin.php?id_eleve=...` qui calcule les moyennes pondérées avec une requête SQL du type `SELECT SUM(valeur*coef)/SUM(coef) AS moyenne ...`, puis met en page le résultat en CSS pour l'impression.

3.5 Gestion des droits d'accès selon les profils

Intuition. Un parent ne doit pas modifier les notes ; un enseignant ne doit pas créer de comptes. Le contrôle des droits se fait **côté serveur** (jamais uniquement en cachant des boutons en HTML, qu'un utilisateur habile contourne en tapant directement l'URL de la page).

Fonctionnalité	Administrateur	Enseignant	Parent
Gérer les comptes utilisateurs	Oui	Non	Non
Saisir / modifier les notes	Non	Oui (ses matières)	Non
Consulter les bulletins	Oui (tous)	Oui (ses classes)	Oui (son enfant)
Éditer / imprimer les bulletins	Oui	Oui	Oui (son enfant)

La mise en œuvre repose sur deux réflexes systématiques en tête de chaque page : vérifier la session (`isset($_SESSION['id_user'])`) puis vérifier le profil (`$_SESSION['profil'] === 'enseignant'`), comme illustré dans le script de saisie ci-dessus. Pour la consultation parent, on ajoute un contrôle métier : l'`id_eleve` demandé doit bien correspondre à un enfant rattaché au compte du parent connecté — sinon, un parent pourrait consulter le bulletin de n'importe quel élève en changeant simplement le numéro dans l'URL.

3.6 Travail en équipe et intégration progressive

Un projet de classe se développe à plusieurs. La démarche qui fonctionne :

1. **Partager les tâches** selon les tiers : un binôme « présentation » (HTML/CSS), un binôme « traitement » (PHP), un binôme « données » (SQL et jeu d'essai) ;
2. **Geler les interfaces d'abord** : la liste exacte des pages, des paramètres d'URL et des tables — chacun peut alors travailler sans attendre les autres ;
3. **Intégrer progressivement** : on branche les modules un par un (login, puis saisie, puis consultation), en testant après chaque ajout, plutôt que de tout assembler à la fin ;
4. **Sauvegarder régulièrement** le projet complet (code + export SQL de la base) sur un support externe ou un espace partagé.

LESAVAIS-TU ? Complément utile au Bac : le versionning

Les professionnels ne se contentent pas de dossiers projet_v1, projet_v2_final, projet_v2_final_VRAI : ils utilisent un **gestionnaire de versions** comme Git (souvent avec GitHub ou GitLab). Git enregistre l'historique complet des modifications, permet de revenir en arrière après une erreur et de fusionner le travail de plusieurs développeurs sans écraser les fichiers des autres. Ce n'est pas exigible au programme de Terminale TI, mais en parler dans ton rapport de projet est un excellent point lors de l'évaluation.

EXERCICE RÉSOLU 3 Afficher la moyenne d'un élève de façon sécurisée

Énoncé : écrire le fragment PHP de `bulletin.php` qui récupère `id_eleve` depuis l'URL et affiche sa moyenne générale, calculée sur toutes les matières avec leurs coefficients, sans risque d'injection SQL.

SOLUTION

Solution détaillée :

```
<?php
session_start();
require 'config.php';
// 1. Contrôle d'accès
if (!isset($_SESSION['id_user'])) {
    header('Location : login.php');
    exit();
}
// 2. Validation du paramètre d'URL : un entier obligatoirement
$id_eleve = filter_input(INPUT_GET, 'id_eleve', FILTER_VALIDATE_INT);
if ($id_eleve === false || $id_eleve === null) {
    die('Parametre invalide.');
```

Points clés : le paramètre d'URL est validé (entier), la requête est préparée (marqueur ?), le résultat affiché est échappé. Si ABENA Marie a 14/20 en Informatique (coef 5) et 16/20 en Maths (coef 4), la requête calcule $\frac{14 \times 5 + 16 \times 4}{5 + 4} = \frac{134}{9} \approx 14,89$.

EXPÉRIENCE TP guidé : faire fonctionner la chaîne complète en local

Objectif : exécuter l'application sur le poste de développement avec WampServer. Matériel : un PC sous Windows, WampServer installé, les fichiers du projet.

1. Copier le dossier `projet_notes/` dans `C:\wamp64\www\`.
2. Lancer WampServer ; attendre que l'icône passe au vert (signe qu'Apache et MySQL sont démarrés).
3. Ouvrir `http://localhost/phpmyadmin`, créer la base `notes_college` (interclassement `utf8mb4_general_ci`) et y importer `sql/creation_base.sql`.
4. Insérer un utilisateur de test avec un mot de passe haché (via un petit script PHP appelant `password_hash`, car un mot de passe tapé en clair dans phpMyAdmin ne serait pas reconnu par `password_verify`).
5. Ouvrir `http://localhost/projet_notes/login.php`, se connecter, saisir une note, puis consulter le bulletin.

Observations : chaque action dans le navigateur déclenche une requête HTTP vers Apache, qui exécute le PHP, lequel interroge MySQL. Interprétation : on vient de reproduire localement l'architecture 3 tiers qui sera publiée sur le réseau du lycée à la section suivante.

À RETENIR

Une application web dynamique repose sur l'architecture **3 tiers** : HTML/CSS pour la présentation, PHP pour le traitement, MySQL pour les données. On organise le code en dossiers par rôle, on mémorise l'utilisateur grâce aux **sessions**, on se connecte à la base avec **PDO**, on sécurise chaque échange par des **requêtes préparées**, on **hache** les mots de passe et on contrôle les droits côté serveur selon le profil, sans jamais oublier `exit()` après une redirection. En équipe, on partage les tâches par tier et on intègre les modules progressivement, avec des sauvegardes régulières.

4 RÉSEAU ET PUBLICATION : HÉBERGER L'APPLICATION SUR UN RÉSEAU LOCAL

Après avoir développé ton application web dynamique (PHP, base de données MySQL, interface HTML/CSS/JS) dans la section précédente, celle-ci ne vit encore que sur ta machine de développement (`localhost`). Pour qu'elle devienne un véritable système d'information utilisable par l'administration du lycée, la bibliothèque ou le cybercafé du quartier, il faut la **déployer** sur un serveur accessible depuis le réseau local (intranet), voire la publier sur Internet. Cette section te guide pas à pas : de la configuration du réseau câblé du laboratoire à la mise en produc-

tion sur une pile WAMP, jusqu'aux notions essentielles d'hébergement distant et de sécurité minimale.

4.1 Rappel : configuration d'un petit réseau local

Avant d'installer le serveur, le réseau physique et logique doit être opérationnel. Dans un lycée camerounais typique, le laboratoire d'informatique est câblé en topologie **étoile** : chaque poste (client ou serveur) est relié par un câble RJ45 (catégorie 5e ou 6) à un **switch** (commutateur, souvent un modèle 24 ou 48 ports non administrable, marque TP-Link, D-Link ou Cisco). Le switch relie lui-même le routeur/pare-feu de l'établissement, qui fait office de passerelle vers Internet via la fibre CAMTEL ou MTN Business.

DÉFINITION Hébergement local (intranet) vs hébergement distant (Internet)

Hébergement local (intranet) : l'application tourne sur une machine (serveur) connectée au même réseau local (LAN) que les postes clients. L'accès se fait via l'adresse IP privée du serveur (ex. `http://192.168.1.10/monapp`). Aucune connexion Internet n'est requise. Coût nul hors matériel existant.

Hébergement distant (Internet) : l'application est installée sur un serveur loué chez un hébergeur (OVH, Ionos, ou camerounais comme Axis, Netcom, Kamsoft) dans un centre de données (datacenter). L'accès se fait via un nom de domaine (ex. `https://monlycee.cm`) résolu par le DNS mondial. Nécessite un abonnement, une bande passante suffisante et une configuration DNS.

Pour que les postes clients trouvent le serveur, chaque machine doit avoir une configuration IP cohérente. On utilise obligatoirement des **adresses IP privées** (RFC 1918) : 10.0.0.0/8, 172.16.0.0/12 ou 192.168.0.0/16. Le choix le plus courant dans les PME et établissements scolaires est 192.168.1.0/24.

PROPRIÉTÉ Plan d'adressage IP privé en /24

Pour un réseau 192.168.1.0/24 (masque 255.255.255.0) :

- Adresse réseau : 192.168.1.0 (inutilisable pour une machine).
- Adresse de broadcast : 192.168.1.255 (inutilisable pour une machine).
- Plage utilisable : 192.168.1.1 à 192.168.1.254, soit $2^8 - 2 = 254$ hôtes maximum.
- Passerelle par défaut (gateway) : souvent 192.168.1.1 ou 192.168.1.254 (adresse LAN du routeur/pare-feu).
- Serveur DNS : souvent la passerelle elle-même, qui relaie vers les DNS du fournisseur d'accès.

EXEMPLE Plan d'adressage du laboratoire de Terminale TI — 4 postes

Le tableau ci-dessous définit l'adressage statique pour notre projet : 1 serveur web + 3 postes clients. Le routeur/pare-feu de l'établissement (box fibre CAMTEL) conserve l'IP 192.168.1.254.

Rôle	Nom machine	Adresse IP	Commentaire
Routeur / Passerelle	ROUTEUR-LYCEE	192.168.1.254	Box fibre CAMTEL / pare-feu
Serveur Web (WAMP)	SRV-PROJET-TI	192.168.1.10	IP fixe obligatoire
Poste Client 1 (Prof)	PC-PROF-01	192.168.1.20	Statique ou bail DHCP réservé
Poste Client 2 (Élève)	PC-ELEVE-01	192.168.1.21	Statique ou bail DHCP réservé
Poste Client 3 (Élève)	PC-ELEVE-02	192.168.1.22	Statique ou bail DHCP réservé

Masque de sous-réseau : 255.255.255.0 (/24) pour tous. Passerelle : 192.168.1.254. DNS : 192.168.1.254 (ou 8.8.8.8 / 1.1.1.1 si la sortie Internet est autorisée).

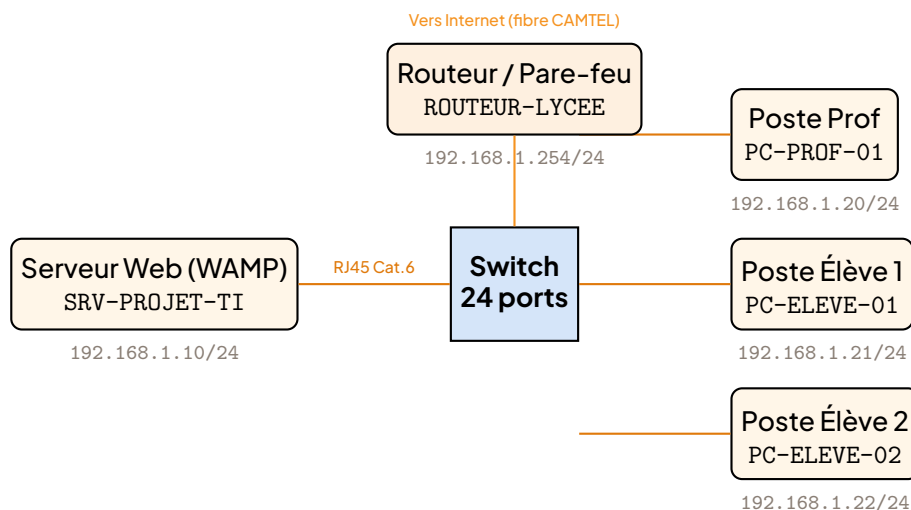


Schéma du réseau local du laboratoire : topologie étoile autour du switch. Le serveur (IP fixe .10) et les 3 clients communiquent via le switch. La passerelle (.254) relie le LAN à Internet (fibre CAMTEL).

4.2 Choix de la machine serveur et installation d'une pile WAMP/XAMPP

Le « serveur » n'a pas besoin d'être une machine rackable coûteuse (Dell PowerEdge, HP ProLiant). Un poste de travail récent (processeur 4 cœurs ou plus, 8 Go de RAM minimum, 16 Go

recommandés, SSD de 256 Go ou plus) sous Windows 10/11 Pro ou Linux Ubuntu Server fait l'affaire. **Critères de choix :**

1. **Disponibilité** : machine allumée en permanence pendant les heures de cours (mise en veille désactivée).
2. **Ressources** : suffisantes pour faire tourner Apache + MySQL + PHP simultanément (une trentaine de connexions élèves).
3. **Stabilité du système** : Windows Pro (gestion des stratégies et Bureau à distance) ou Linux Server (sans interface graphique, plus léger).

Pour l'approche pédagogique « clé en main » en Terminale TI, on utilise **WampServer** (Windows Apache MySQL PHP) version 3.3+ (PHP 8.2+, MySQL 8.0+, Apache 2.4+). L'alternative multi-plateforme est **XAMPP**.

MÉTHODE**Installer et configurer WampServer sur le poste serveur (5 étapes)**

1. **Prérequis VC++** : télécharge et installe les redistribuables Visual C++ (VC++ 2015–2022) depuis le site officiel Microsoft. Sans eux, les services Apache/MySQL ne démarrent pas (erreur VCRUNTIME140.dll manquant).
2. **Installation de WAMP** : exécute l'installateur `wampserver3.3.x_x64.exe` en tant qu'administrateur. Choisis le dossier par défaut `C:\wamp64`.
3. **Vérification des services** : lance WampServer. L'icône dans la barre des tâches doit devenir verte (les trois services Apache, MySQL et MariaDB démarrés). Si elle reste orange, un service n'a pas démarré : vérifie les prérequis VC++ et qu'aucun autre programme (Skype, IIS) n'occupe le port 80.
4. **Test local** : ouvre `http://127.0.0.1` ou `http://localhost` sur le serveur. La page d'accueil WampServer doit s'afficher. Clique sur `phpMyAdmin` : connexion avec l'utilisateur `root` (mot de passe vide par défaut).
5. **Configuration de l'IP fixe (étape critique)** : configure l'adresse IP statique `192.168.1.10` sur la carte Ethernet du serveur avant de continuer (voir l'encadré Attention et le TP guidé ci-dessous).

ATTENTION**Erreur fréquente : laisser le serveur en DHCP (adresse IP dynamique)**

Si le serveur obtient son IP par DHCP (via la box routeur), son adresse changera tôt ou tard (expiration du bail, redémarrage de la box, conflit d'adresses). Conséquence : tous les raccourcis, favoris et liens pointant vers `http://192.168.1.10/...` deviennent cassés. L'application devient inaccessible sans intervention de l'administrateur.

Parade : configurer une adresse IP statique sur la carte réseau du serveur (Panneau de configuration > Réseau > Propriétés IPv4), et/ou créer une réservation DHCP (bail per-

manent) sur le routeur en associant l'adresse MAC du serveur à 192.168.1.10. La méthode statique sur le serveur est la plus robuste en environnement scolaire, où le routeur est souvent verrouillé par l'administrateur réseau.

EXPÉRIENCE TP guidé : configurer l'IP fixe et valider la connectivité LAN

Objectif : rendre le serveur joignable de façon permanente à l'adresse 192.168.1.10.

Matériel : poste serveur Windows 10/11 Pro, câble RJ45 connecté au switch du laboratoire, droits administrateur.

Manipulation :

1. Win+R → tape `ncpa.cpl` (Connexions réseau).
2. Clic droit sur Ethernet → Propriétés.
3. Sélectionne Protocole Internet version 4 (TCP/IPv4) → Propriétés.
4. Coche Utiliser l'adresse IP suivante et saisis :
 - Adresse IP : 192.168.1.10
 - Masque de sous-réseau : 255.255.255.0 (se remplit automatiquement)
 - Passerelle par défaut : 192.168.1.254
 - Serveur DNS préféré : 192.168.1.254
 - Serveur DNS auxiliaire : 1.1.1.1
5. Valide par OK puis ferme les fenêtres.
6. Test sur le serveur : Win+R → `cmd` → `ipconfig`. Vérifie que la ligne Adresse IPv4 affiche 192.168.1.10.
7. Test depuis un poste client : `ping 192.168.1.10`. Réponse attendue : Réponse de 192.168.1.10 : octets=32 temps<1ms TTL=128.

Observation : si le ping échoue, vérifie le pare-feu Windows (autoriser le protocole ICMPv4 entrant sur le profil Privé) ou le câble et le switch.

Interprétation : le serveur possède maintenant une identité réseau stable. C'est la fondation de l'hébergement local.

4.3 Déploiement de l'application : fichiers, base de données, configuration

Une fois WampServer vert et l'IP fixe validée, on déploie le code produit dans la section précédente.

MÉTHODE Déployer l'application PHP/MySQL sur WampServer (4 étapes)

1. Copier les fichiers sources : crée un dossier projet dans C:\wamp64\www\ (ex. gestion_biblio). Copie tous tes fichiers (index.php, config.php, dossiers controllers/, views/, assets/) dans ce dossier.
2. Créer et importer la base de données :
 - Ouvre `http://localhost/phpmyadmin` sur le serveur.
 - Onglet Bases de données → crée la base `db_biblio` (interclassement : `utf8mb4_general_ci`).
 - Onglet Importer → choisis ton fichier `schema_biblio.sql` (export de ta base de développement) → Exécuter.
 - Vérifie que les tables (utilisateurs, ouvrages, emprunts) sont présentes, puis crée un utilisateur MySQL dédié (jamais `root` pour l'application) avec la requête SQL suivante :

```
-- Creation d'un utilisateur applicatif aux droits limites
CREATE USER 'biblio_user'@'localhost'
  IDENTIFIED BY 'MotDePasseFort123!';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON db_biblio.* TO 'biblio_user'@'localhost';
FLUSH PRIVILEGES;
```

3. Paramétrer la connexion : adapte le fichier `config.php` aux identifiants du serveur de production locale (voir l'exemple de code ci-dessous).
4. Droits des dossiers et pare-feu Windows :
 - Dossier `www\gestion_biblio` : accorder le droit d'écriture uniquement aux sous-dossiers qui en ont besoin (`uploads/`, `logs/`).
 - Pare-feu Windows > Règles de trafic entrant > Nouvelle règle > Port > TCP > port local 80 (HTTP) > Autoriser la connexion > Profil : Privé > Nom : WAMP Apache HTTP.

```
<?php
/**
 * Configuration de l'application "Gestion Bibliotheque"
 * Environnement : PRODUCTION LOCALE (intranet du lycee)
 * Fichier : C:\wamp64\www\gestion_biblio\config\config.php
 * Ce fichier contient des secrets : ne jamais le publier (Git public, etc.)
 */

// 1. Constantes d'environnement
define('APP_ENV', 'production'); // 'development' sur le poste de dev
define('APP_DEBUG', false); // pas d'affichage d'erreurs detaillees
define('APP_URL', 'http://192.168.1.10/gestion_biblio'); // URL publique LAN

// 2. Base de donnees MySQL (utilisateur dedie, pas root !)
define('DB_HOST', '127.0.0.1'); // MySQL tourne sur le meme serveur qu'Apache
```

```

define('DB_PORT', 3306);
define('DB_NAME', 'db_biblio');
define('DB_USER', 'biblio_user'); // utilisateur cree a l'etape 2
define('DB_PASS', 'MotDePasseFort123!');
define('DB_CHARSET', 'utf8mb4');

// 3. Connexion PDO
try {
    $dsn = "mysql:host=" . DB_HOST . ";port=" . DB_PORT
        . ";dbname=" . DB_NAME . ";charset=" . DB_CHARSET;
    $options = [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false, // vraies requetes preparees
    ];
    $pdo = new PDO($dsn, DB_USER, DB_PASS, $options);
} catch (PDOException $e) {
    // En production : on journalise l'erreur, on n'affiche rien de technique
    error_log("Connexion DB echouee : " . $e->getMessage());
    if (APP_DEBUG) {
        die("Erreur DB : " . $e->getMessage());
    } else {
        die("Service temporairement indisponible. Contactez l'administrateur.");
    }
}

// 4. Sécurisation des sessions
ini_set('session.cookie_httponly', 1); // cookie inaccessible au JS (anti-XSS)
ini_set('session.use_strict_mode', 1); // empeche la fixation de session
session_start();

// 5. Fuseau horaire du Cameroun (WAT, UTC+1, pas d'heure d'ete)
date_default_timezone_set('Africa/Douala');

```

4.4 Accès depuis les postes clients du réseau

L'application est maintenant accessible depuis n'importe quel navigateur du LAN à l'adresse http://192.168.1.10/gestion_biblio/.

EXERCICE RÉSOLU 4 Test de recette d'accès multi-postes

Énoncé : vérifier que l'application fonctionne correctement depuis les 3 postes clients du plan d'adressage (PC-PROF-01, PC-ELEVE-01, PC-ELEVE-02). Tester : page d'accueil, connexion utilisateur, requête SQL (enregistrement d'un emprunt), déconnexion.

SOLUTION

Solution détaillée :

1. Sur PC-PROF-01 (192.168.1.20) : ouvrir le navigateur → http://192.168.1.10/gestion_biblio/. Résultat attendu : page de connexion affichée. Connexion avec le compte administrateur → tableau de bord OK. Créer un emprunt → insertion en base vérifiée dans phpMyAdmin sur le serveur.

2. Sur PC-ELEVE-01 (192.168.1.21) : même URL. Résultat attendu : page de connexion, catalogue visible après authentification, bouton « Emprunter » fonctionnel.
3. Sur PC-ELEVE-02 (192.168.1.22) : test simultané avec PC-ELEVE-01. Résultat attendu : aucun conflit de session (chaque navigateur possède son propre identifiant de session PHP, stocké côté serveur) ; les requêtes SQL concurrentes sont gérées par le moteur InnoDB (verrouillage au niveau ligne).
4. Test de robustesse : redémarrer le service Apache (icône Wamp > Apache > Redémarrer le service). Rafraîchir les clients → coupure temporaire, puis retour à la normale. Redémarrer complètement le serveur → l'IP fixe est conservée et les services WampServer repartent automatiquement (configurés en démarrage automatique via `services.msc`) → l'application redevient accessible sans intervention.

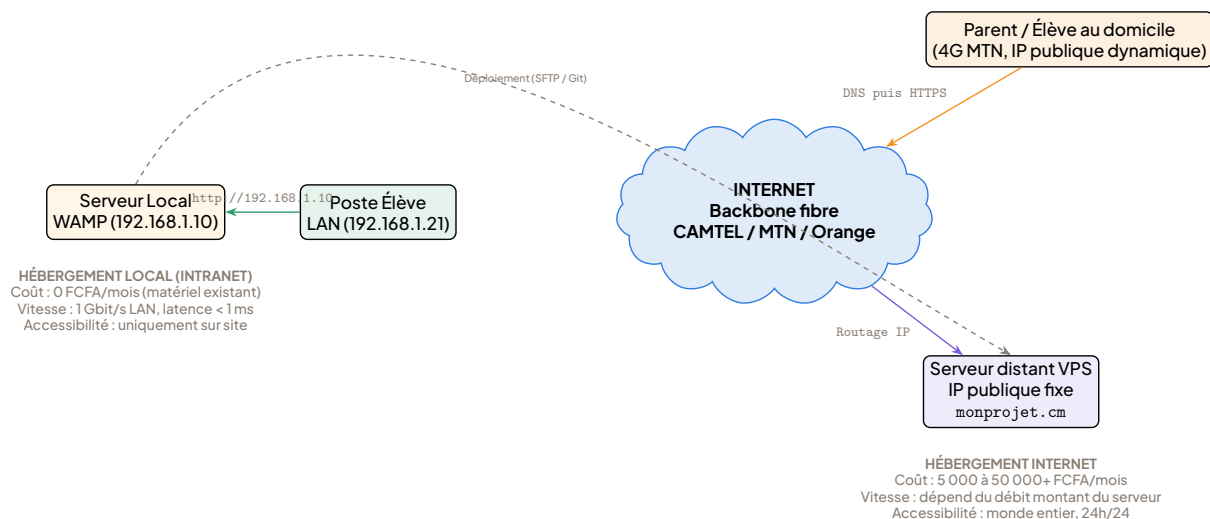
Conclusion : l'hébergement local est validé. L'application est devenue un véritable service réseau.

4.5 Publication sur Internet : principes et différences avec le local

Passer du LAN à l'Internet public (par exemple pour que les parents consultent les notes depuis la maison, ou pour une startup camerounaise comme GiftedMom ou Kiro'o Games) change la donne technique et contractuelle.

DÉFINITION Hébergement distant (mutualisé / VPS / Cloud) — composants clés

- Hébergeur : fournisseur d'infrastructure (OVHcloud, Scaleway, AWS, ou locaux : Axis Cameroun, Netcom, CAMTEL Business). Il loue les ressources (CPU, RAM, disque, bande passante).
- Nom de domaine : adresse lisible (ex. `monlycee.cm`), achetée chez un registrar (souvent l'hébergeur lui-même) et gérée via une zone DNS (enregistrements A, AAAA, CNAME, MX).
- Transfert de fichiers SFTP/SSH : protocole pour envoyer les fichiers locaux vers le serveur distant. SFTP (port 22) est obligatoire : le FTP classique (port 21) transmet les mots de passe en clair. Clients : FileZilla, WinSCP, ou les commandes `scp` / `rsync`.
- Serveur web distant : Apache ou Nginx, configuré par l'hébergeur (mutualisé) ou par toi (VPS/Cloud).
- Certificat SSL/TLS (HTTPS) : obligatoire aujourd'hui (Let's Encrypt, gratuit et renouvelé automatiquement). Les navigateurs signalent comme « non sécurisées » les pages HTTP contenant des formulaires de connexion ou de paiement.



Comparatif des architectures : hébergement local (intranet) vs hébergement Internet. En local, le client atteint le serveur directement via son IP privée sur le switch. Sur Internet, le trafic traverse le backbone national (fibre CAMTEL/MTN), résout le nom de domaine via le DNS, puis atteint le serveur distant. La publication exige un nom de domaine, un certificat SSL/TLS et une bande passante suffisante.

LESAVAIS-TU ? Le serveur WAMP du lycée : une « mini-CAMTEL » interne ?

Savais-tu qu'un simple PC sous WampServer bien configuré (IP fixe, pare-feu ouvert sur le port 80, services en démarrage automatique) transforme ton laboratoire en fournisseur d'applications pour tout l'établissement, sans aucun abonnement Internet ? Le principal peut consulter les statistiques de la bibliothèque depuis son bureau (s'il est câblé sur le même réseau), le surveillant général peut pointer les présences depuis la salle des professeurs, le tout à 0 FCFA/mois de frais d'hébergement. C'est la puissance de l'intranet : la bande passante locale (1 Gbit/s sur Ethernet) est gratuite, illimitée et à latence quasi nulle. C'est pour cela que les banques (Afriland, BICEC, UBA) et l'État (MINFI, ANTIC) hébergent leurs applications critiques on-premise (sur leurs propres serveurs) et n'utilisent le Cloud public que pour des services non sensibles ou de la sauvegarde externe.

4.6 Sécurité minimale : pare-feu, droits d'accès, sauvegarde

Même en local, un serveur exposé sur le réseau est une cible potentielle. Le programme MINE-SEC exige une « administration des réseaux informatiques » responsable.

PROPRIÉTÉ Checklist de sécurité minimale pour un serveur WAMP local

1. Pare-feu Windows : règle entrante uniquement sur le port 80 (HTTP, et 443 si HTTPS) pour le profil `Privé`. Bloquer le port 3306 (MySQL) depuis le réseau : la base ne doit être accessible que via `localhost` (127.0.0.1) pour les scripts PHP.
2. Droits sur les dossiers : lecture et exécution pour le groupe `Utilisateurs` sur le dossier du projet ; droit d'écriture uniquement sur les sous-dossiers `uploads/` et `logs/`. Jamais de droit d'écriture sur les fichiers `.php` exécutables.
3. Mots de passe MySQL : définir un mot de passe fort pour `root` (via `phpMyAdmin` > Comptes utilisateurs). L'utilisateur applicatif (`biblio_user`) ne reçoit que les droits `SELECT`, `INSERT`, `UPDATE`, `DELETE` sur `db_biblio.*` (jamais `DROP`, `GRANT` ni `SHUTDOWN`).
4. Masquer les erreurs en production : dans `php.ini` (icône Wamp > PHP > `php.ini`) : `display_errors = Off`, `log_errors = On`. Cela empêche la fuite de chemins de fichiers et de structure de la base dans le navigateur.
5. Sauvegarde automatisée : script `backup_daily.bat` lancé par le Planificateur de tâches Windows chaque nuit (voir le code ci-dessous).

```
@echo off
REM -----
REM Sauvegarde quotidienne de la base db_biblio
REM Auteur : Prof. TI - Terminale TI
REM Planificateur de taches : tous les jours a 02h00
REM -----

set DATESTAMP=%DATE:~6,4%%DATE:~3,2%%DATE:~0,2%_%TIME:~0,2%%TIME:~3,2%
set DATESTAMP=%DATESTAMP: =0%
set BACKUP_DIR=D:\Backups\db_biblio
set MYSQL_DUMP="C:\wamp64\bin\mysql\mysql8.0.36\bin\mysqldump.exe"
set DB_USER=biblio_user
set DB_PASS=MotDePasseFort123!
set DB_NAME=db_biblio

mkdir "%BACKUP_DIR%" 2>nul

echo [%DATE% %TIME%] Debut de la sauvegarde de %DB_NAME%...
%MYSQL_DUMP% -u %DB_USER% -p%DB_PASS% --single-transaction ^
--routines --triggers %DB_NAME% ^
> "%BACKUP_DIR%\%DB_NAME%_%DATESTAMP%.sql"

if %ERRORLEVEL%==0 (
    echo [%DATE% %TIME%] Succes : %DB_NAME%_%DATESTAMP%.sql
    REM Nettoyage : on ne garde que les 30 derniers jours
    forfiles /p "%BACKUP_DIR%" /m %DB_NAME%_*.sql /d -30 /c "cmd /c del @path"
) else (
    echo [%DATE% %TIME%] ERREUR mysqldump ! Verifier les logs.
)
```

ATTENTION Pièges de sécurité classiques en TP et au projet du BAC

- Utilisateur `root` sans mot de passe : laisser `root` avec un mot de passe vide en production est une faille critique : n'importe qui sur le LAN peut ouvrir phpMyAdmin et détruire la base. Correction : définir un mot de passe fort via phpMyAdmin > Comptes utilisateurs.
- Dossiers `.git` ou fichier `.env` accessibles via HTTP : si tu déploies avec `git clone` dans `www/`, Apache peut servir le dossier `.git` (historique complet) et le fichier `.env` (mots de passe) à quiconque les demande ! Correction : placer ces fichiers hors de la racine web, ou les bloquer par une règle `.htaccess`.
- Port 3306 ouvert sur le LAN : un élève curieux équipé de HeidiSQL ou DBeaver pourrait tenter de se connecter directement à la base depuis son poste. Correction : laisser `bind-address = 127.0.0.1` dans `my.ini` (configuration par défaut de WampServer) et bloquer le port 3306 entrant dans le pare-feu.
- HTTP au lieu de HTTPS pour l'authentification : sur le LAN, les mots de passe circulent en clair et peuvent être interceptés (sniffing avec Wireshark). Idéal : générer un certificat auto-signé (OpenSSL ou `mkcert`) pour `192.168.1.10` et configurer Apache en SSL (port 443). Au minimum, sensibiliser les utilisateurs au risque.

À RETENIR Synthèse : de l'application locale au service réseau

1. Réseau : topologie étoile autour d'un switch, plan d'adressage `192.168.1.0/24`, IP fixe pour le serveur (`.10`), passerelle (`.254`).
2. Serveur : poste dédié, WampServer (Apache/MySQL/PHP), services en démarrage automatique, IP statique validée par ping.
3. Déploiement : fichiers copiés dans `www/`, base importée via phpMyAdmin, utilisateur MySQL dédié aux droits limités, `config.php` adapté (PDO, charset `utf8mb4`, fuseau Africa/Douala), pare-feu ouvert sur le port 80 uniquement.
4. Accès clients : navigateur → `http://192.168.1.10/projet/`, test multi-postes simultanés validé.
5. Internet vs local : local = gratuit, rapide, privé (intranet). Internet = payant, nom de domaine + DNS, SSL/TLS obligatoire, hébergeur tiers.
6. Sécurité minimale : pare-feu restrictif (port 3306 fermé au LAN), utilisateur de base de données au moindre privilège, `display_errors = Off`, sauvegarde quotidienne automatisée (`mysqldump` + rotation), protection des fichiers sensibles (`.env`, `.git`).

4.7 Exercices d'application

EXERCICE 2 Configuration réseau et déploiement

Contexte : tu dois héberger l'application « Gestion Notes » (PHP/MySQL) sur le réseau du lycée. Le routeur a l'adresse 10.0.0.254/24. Tu disposes d'un serveur et de 20 clients.

1. Propose un plan d'adressage complet (IP du serveur, plage clients, masque, passerelle, DNS) en utilisant le réseau 10.0.0.0/24.
2. Explique pourquoi il est impératif de fixer l'IP du serveur (statique ou bail DHCP permanent) et non de la laisser en DHCP standard.
3. Liste les 3 modifications exactes à faire dans `config.php` pour passer de l'environnement de développement (localhost, utilisateur root, mot de passe vide, APP_DEBUG à true) à l'environnement de production locale (10.0.0.10, utilisateur notes_user, mot de passe fort, APP_DEBUG à false).
4. Donne la commande `mysqldump` complète (avec les options `--single-transaction` `--routines` `--triggers`) pour sauvegarder la base `db_notes` avec l'utilisateur `notes_user`.

EXERCICE 3 Sécurité et publication

1. Cite 3 risques majeurs si le port MySQL (3306) est ouvert sur le pare-feu Windows du serveur en profil « Privé ».
2. Différencie hébergement mutualisé et VPS pour une startup camerounaise qui lance une application de e-commerce (paiement MTN Mobile Money / Orange Money). Donne 2 avantages et 2 inconvénients de chaque solution.
3. Pourquoi le protocole SFTP (port 22) est-il obligatoire pour transférer les fichiers vers un serveur distant, contrairement au FTP standard (port 21) ?
4. Quel est le rôle de l'enregistrement DNS de type A et de type CNAME pour publier `monappli.monlycee.cm` vers l'IP du VPS ?

CORRIGÉ Exercice 1

1. Plan d'adressage 10.0.0.0/24 : masque 255.255.255.0. Adresse réseau 10.0.0.0, broadcast 10.0.0.255. Passerelle et DNS : 10.0.0.254 (routeur). Serveur : 10.0.0.10 (IP fixe). Clients : 10.0.0.20 à 10.0.0.39 (20 postes, statiques ou baux DHCP réservés). Le reste de la plage reste disponible pour de futurs équipements (bornes Wi-Fi, imprimantes réseau).

2. IP fixe impérative : l'URL d'accès de l'application (`http://10.0.0.10/notes`) est mémorisée dans les raccourcis du Bureau des professeurs, les favoris des navigateurs et potentiellement dans la configuration de l'application. Si l'IP change (par exemple 10.0.0.15 après un redémarrage), tous ces liens cassent instantanément : l'application devient « perdue » sur le réseau. La réservation DHCP (par adresse MAC) ou l'IP statique sur la carte réseau garantit la persistance de l'adresse du service.

3. Modifications de `config.php` :

- (a) `define('APP_URL', 'http://10.0.0.10/notes');` (au lieu de `http://localhost/notes`)
- (b) `define('DB_USER', 'notes_user');` et `define('DB_PASS', '...mot de passe fort...');` (au lieu de `root` et mot de passe vide)
- (c) `define('APP_DEBUG', false);` (au lieu de `true`), ce qui désactive l'affichage des erreurs techniques.

4. Commande `mysqldump` (sous Linux) :

```
mysqldump -u notes_user -p --single-transaction \  
--routines --triggers db_notes > backup_db_notes.sql
```

(Sous Windows, dans le script batch : `%MYSQL_DUMP% -u %DB_USER% -p%DB_PASS% --single-transaction --routines --triggers %DB_NAME% > "%BACKUP_DIR%\%DB_NAME%_%DATESTAMP%.sql".`)

CORRIGÉ Exercice 2

1. Risques liés à l'ouverture du port 3306 sur le LAN : 1) les échanges MySQL non chiffrés peuvent être interceptés (sniffing des identifiants et des requêtes avec Wireshark); 2) attaque par force brute sur les comptes `root` ou `notes_user` depuis n'importe quel poste élève (outils comme Hydra); 3) exploitation d'éventuelles failles de sécurité de MySQL pour exécuter du code à distance ou provoquer un déni de service sur le serveur.
2. Mutualisé vs VPS :

	Mutualisé (shared)	VPS (serveur privé virtuel)
Avantages	Peu cher (environ 2 000 à 5 000 FCFA/mois); administré par l'hébergeur (mises à jour, sauvegardes); panneau cPanel/Plesk simple.	Ressources garanties et isolées; accès root (installer Redis, configurer Nginx, etc.); adresse IP dédiée; évolutif.
Inconvénients	Ressources partagées (ralentissements possibles); pas d'accès root; configuration PHP/MySQL figée; IP partagée (risque de blacklist).	Plus cher (environ 10 000 à 30 000 FCFA/mois); compétences d'administration système requises (sécurité, mises à jour, pare-feu, supervision); sauvegardes à ta charge.

Choix pour la startup e-commerce : le VPS est recommandé (paiement Mobile Money = données sensibles, besoin d'un SSL maîtrisé, de tâches de fond et d'une IP propre pour les API MoMo/OM).

3. SFTP vs FTP : le FTP transmet les identifiants et les données en clair (port 21). Sur un réseau public, n'importe quel équipement intermédiaire compromis peut capturer le login, le mot de passe et les fichiers. SFTP (SSH File Transfer Protocol) chiffre tout (authentification et données) via SSH sur le port 22. C'est le standard industriel obligatoire.
4. Enregistrement A : il fait pointer `monappli.monlycee.cm` vers l'adresse IPv4 du VPS (ex. 196.200.50.10). Enregistrement CNAME : il crée un alias vers un autre nom de domaine, par exemple `www.monlycee.cm` → `monlycee.cm`. Pour un sous-domaine applicatif, on utilise un enregistrement A vers l'IP du serveur, ou un CNAME vers un nom existant si c'est le même serveur.

5

TESTS, VALIDATION ET RECETTE DE L'APPLICATION

Après avoir déployé votre application sur le réseau local (cf. section précédente), vous disposez d'une version fonctionnelle accessible aux utilisateurs tests. Mais « fonctionnelle » ne signifie pas « validée ». Un site qui s'affiche dans le navigateur peut encore cacher des failles de sécurité, des erreurs de calcul sur les bulletins, ou une incompatibilité avec le navigateur du visiteur. Cette section vous apprend à **prouver** que votre logiciel respecte le cahier des charges, à travers une démarche de test rigoureuse, tracée et professionnelle.

5.1 Pourquoi tester : la confiance par la preuve

Dans tout projet informatique, le test n'est pas une étape optionnelle réalisée « si on a le temps » : c'est l'activité qui transforme un code source en **produit livrable**. Le cahier des charges (CDC) définit ce que l'application doit faire ; les tests vérifient qu'elle le fait vraiment, dans toutes les

situations prévues — et même dans certaines situations imprévues.

DÉFINITION Test logiciel

Un **test logiciel** est l'exécution d'un programme ou d'une partie de programme dans le but de trouver des **anomalies** (écarts entre le comportement observé et le comportement attendu dans le CDC). Un test réussi est un test qui révèle une anomalie ; un test qui ne trouve rien n'a pas « passé », il n'a simplement pas détecté de défaut cette fois-ci.

PROPRIÉTÉ Objectifs des tests dans un projet scolaire

Dans le cadre du Baccalauréat TI, les tests servent à :

1. Valider la conformité : chaque exigence fonctionnelle (ex. « calculer la moyenne pondérée ») est couverte par au moins un cas de test.
2. Assurer la robustesse : l'application ne plante pas face à des saisies aberrantes (lettres dans un champ note, date future, injection SQL).
3. Garantir la sécurité : mots de passe hachés, protection contre les injections SQL et les attaques XSS, droits d'accès respectés (un élève ne voit pas les notes des autres).
4. Produire la traçabilité : le jury (ou le client) peut lire le plan de tests et vérifier que tout a été testé.

À RETENIR Le paradoxe du test

« Les tests montrent la présence de défauts, jamais leur absence. » (Edsger Dijkstra). Même 1000 tests passés ne prouvent pas que le logiciel est parfait ; un seul test échoué prouve qu'il est défectueux. D'où l'importance de bien choisir les cas de test.

5.2 Les trois niveaux de test : unitaire, intégration, recette

On ne teste pas tout au même moment ni avec les mêmes outils. Les référentiels professionnels (comme l'ISTQB, référence internationale des testeurs) distinguent trois niveaux que vous devez tous pratiquer dans votre projet.

DÉFINITION Test unitaire (Unit Testing)

Test d'une seule unité de code (une fonction, une méthode, une procédure) isolée du reste. Exemple : tester la fonction `calculerMoyenne($notes, $coeffs)` avec divers tableaux, sans base de données ni interface web.

- Quand ? Pendant le développement (idéalement avant d'écrire le code : c'est la démarche TDD, Test Driven Development).
- Outils ? PHPUnit (PHP), Jest (JS), ou simple script `test_moyenne.php` appelant la fonction et affichant OK / ECHEC.
- But ? Vérifier l'algorithme pur (logique, calculs, cas limites).

DÉFINITION Test d'intégration (Integration Testing)

Test de l'interaction entre plusieurs modules : contrôleur + modèle + base de données ; formulaire HTML + script PHP + session ; formulaire JavaScript + backend.

- Exemple : Soumettre le formulaire « Saisie des notes » → vérifier que l'INSERT SQL est correct, que la redirection vers la liste fonctionne, que le message de confirmation s'affiche.
- But ? Détecter les erreurs d'interface entre modules (mauvais nom de paramètre, type incompatible, requête SQL mal formée).

DÉFINITION Test de validation / Recette (Acceptance Testing)

Test de l'application complète dans un environnement proche de la production, réalisé avec le client (ou le jury/professeur) pour valider que le produit répond au besoin métier.

- Critère d'arrêt : Signature du Procès-Verbal de Recette (PVR) par le client.
- But ? Valider l'adéquation besoin/réponse, l'ergonomie, les performances acceptables.

MÉTHODE Organiser la campagne de tests en 4 phases

1. Préparation : Rédiger le **Plan de Tests** (tableau de cas de test) à partir du CDC. Préparer le **Jeu d'essai** (données de test en BDD : élèves, matières, notes types).
2. Exécution unitaire : Lancer les tests unitaires (PHPUnit / scripts) à chaque modification de code. Corriger immédiatement.
3. Exécution intégration : Parcourir les scénarios utilisateur complets (parcours « Professeur saisit notes → valide → génère bulletin »). Remplir le tableau de résultats (Obtenu / Verdict).
4. Recette finale : Démonstration devant le client. Signature du PVR. Remise du Guide d'utilisation.

5.3 Conception d'un plan de tests : le tableau de cas de test

Le **Plan de Tests** est un document vivant (souvent un fichier Excel/Calc ou un tableau dans le rapport) qui liste **tous** les cas de test à exécuter. Chaque ligne = un cas de test atomique.

PROPRIÉTÉ Structure standard d'un cas de test

Un cas de test comporte au minimum : un identifiant (ID), les données d'entrée / actions à effectuer, le résultat attendu (d'après le CDC). À l'exécution, on complète : le résultat obtenu et le verdict (PASS / FAIL / BLOQUÉ).

ID	Données d'entrée / Actions	Résultat attendu (CDC)	Résultat obtenu	Verdict
UT-01	Appel calculerMoyenne([12,15], [1,1])	Retourne 13,5	13,5	PASS
UT-02	Appel calculerMoyenne([], [])	Lance une exception « Notes vides »	Exception levée	PASS
IT-05	POST /notes/save avec note=25	Message erreur « Note invalide (0-20) », pas d'INSERT	Message affiché, BDD inchangée	PASS
VAL-12	Professeur connecté, génère bulletin classe 6A	PDF généré, notes correctes, signature directrice	PDF OK, données exactes	PASS

Convention de nommage des ID : UT = test Unitaire, IT = test d'Intégration, VAL = test de Validation (recette).

EXEMPLE Plan de tests extrait — Saisie des notes (5 cas)

Le tableau ci-dessous est un extrait du plan de tests pour la fonctionnalité « Saisie des notes par un professeur ». Il montre la couverture : cas normal, cas limite, cas erroné, sécurité, intégration complète.

ID	Données d'entrée / Action	Résultat attendu	Résultat obtenu	Verdict
UT-01	Appel <code>moyenne([12, 14], [2, 1])</code>	Retourne 12.67 (arrondi 2 déc.)	12.67	PASS
UT-02	Appel <code>moyenne([0, 20], [1, 1])</code>	Retourne 10.00	10.00	PASS
UT-03	Appel <code>moyenne([-5, 12], [1, 1])</code>	Lance <code>InvalidArgumentException</code>	Exception levée	PASS
IT-04	POST <code>note=15; DROP TABLE notes;--</code>	Requête préparée : saisie traitée comme chaîne, erreur métier « Note invalide », table intacte	Erreur métier affichée, table OK	PASS
VAL-05	Professeur saisit 30 notes valides, clique « Valider »	Redirection liste, message « 30 notes enregistrées », BDD : 30 lignes insérées	Redirection OK, message OK, 30 lignes	PASS

Vérification du calcul UT-01 : $(12 \times 2 + 14 \times 1) / (2 + 1) = 38 / 3 \approx 12,67$. Toujours vérifier à la main le résultat attendu avant d'exécuter le test.

5.4 Le jeu d'essai : données normales, limites, erronées

Le choix des données de test (**jeu d'essai**) détermine la qualité de la détection. Les deux techniques les plus enseignées et les plus efficaces sont la **partition d'équivalence** (on regroupe les entrées en classes qui doivent se comporter pareil, et on teste un représentant de chaque classe) et l'**analyse des valeurs limites** (on teste exactement aux frontières entre les classes, car c'est là que se cachent les erreurs de comparaison).

EXEMPLE Jeu d'essai chiffré pour la fonction `calculerMoyenne($notes, $coeffs)`

La fonction attend deux tableaux de même taille, notes dans `[0, 20]`, coefficients strictement positifs. Elle retourne la moyenne pondérée arrondie à 2 décimales.

Type	Données \$notes / \$coeffs	Résultat attendu	Objectif
Normal	[12, 15, 10] / [2, 1, 3]	11.50	Cas nominal standard
Normal	[10] / [1]	10.00	Un seul élément (taille min)
Limite basse	[0, 0] / [1, 1]	0.00	Notes minimales autorisées
Limite haute	[20, 20] / [1, 1]	20.00	Notes maximales autorisées
Erronée	[-1, 12] / [1, 1]	Exception InvalidArgumentException	Note < 0
Erronée	[12, 21] / [1, 1]	Exception InvalidArgumentException	Note > 20
Erronée	[12, "abc"] / [1, 1]	Exception (type incorrect)	Chaîne au lieu de nombre
Erronée	[12, 15] / [1]	Exception (tailles différentes)	Tableaux de longueurs inégales
Erronée	[] / []	Exception (tableaux vides)	Évite la division par zéro

Remarque : Tester -1 et 21 (juste hors bornes) est plus puissant que tester -100 et 1000. Les erreurs de comparaison (< au lieu de ≤) se cachent aux frontières.

ATTENTION Piège classique : « Ça marche chez moi » avec des données propres

- Erreur : Ne tester qu'avec des notes correctes (10, 12, 15) saisies au clavier par le développeur.
- Conséquence : L'application plante le jour de la démonstration quand un utilisateur tape « dix » au lieu de « 10 », ou quand un professeur copie-colle depuis Excel avec des virgules (12,5) au lieu de points (12.5).
- Règle d'or : environ la moitié de vos cas de test doivent utiliser des données invalides ou limites. C'est là que se cachent les bugs critiques (division par zéro, injection SQL, erreur d'arrondi).
- Automatisez : Écrivez un script PHP `tests/run_all_tests.php` qui exécute tous les cas unitaires en une commande, et relancez-le après chaque modification du code.

5.5 Traçabilité : lier chaque test au cahier des charges

La **traçabilité** est la preuve que vous n'avez rien oublié. Elle se matérialise par une **Matrice de Traçabilité des Exigences** : un tableau qui associe chaque exigence du CDC aux cas de test qui la valident.

PROPRIÉTÉ Règle de traçabilité

- Couverture complète : Chaque exigence fonctionnelle (EF-xx) du CDC doit être référencée par au moins un cas de test (UT, IT ou VAL).
- Bidirectionnelle : Depuis le CDC, on trouve les tests ; depuis un test, on sait quelle exigence il valide.
- Preuve pour le jury : Dans le rapport de réalisation, insérez le tableau récapitulatif de couverture.

EXEMPLE Matrice de traçabilité simplifiée (extrait)

ID Exigence (CDC)	Libellé	Cas de tests associés	Statut
EF-03	Saisie notes par prof	UT-01, UT-02, UT-03, IT-04, VAL-05	Couvert
EF-07	Calcul moyenne pondérée	UT-01, UT-02, UT-03	Couvert
EF-12	Sécurité injection SQL	IT-04, IT-09 (login), IT-11 (recherche)	Couvert
EF-15	Génération bulletin PDF	VAL-05, VAL-08, VAL-12	Partiel

Si une ligne reste « Non couvert », le projet n'est pas livrable.

5.6 Correction des anomalies et tests de régression

Quand un test échoue (Verdict = FAIL), on ouvre une **fiche d'anomalie** (ticket) : ID, description, gravité (Bloquante / Majeure / Mineure / Cosmétique), étapes de reproduction, correction apportée.

DÉFINITION Test de régression (Regression Testing)

Après chaque correction, on re-joue les tests concernés (au minimum celui qui a échoué, idéalement toute la suite liée au module modifié) pour s'assurer que :

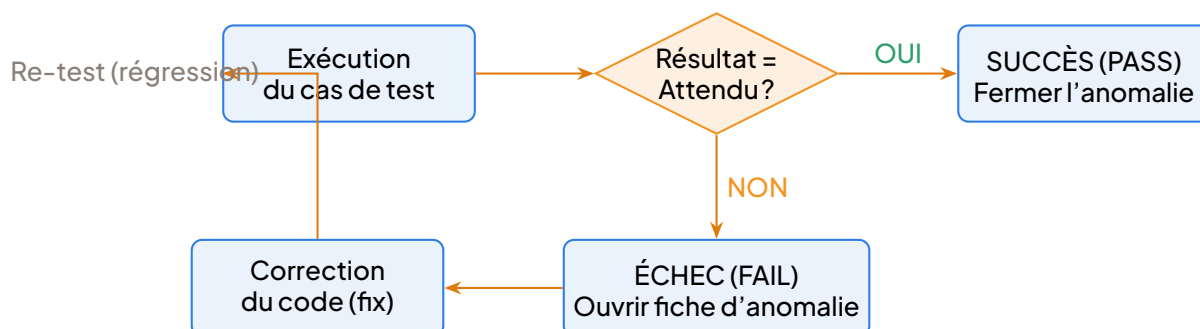
1. L'anomalie est corrigée (le test passe maintenant).

2. Aucune régression n'a été introduite : les tests qui passaient avant passent encore.

MÉTHODE Cycle de vie d'une anomalie (Bug Lifecycle)

1. NOUVEAU : Anomalie détectée lors d'un test (FAIL). Documentée dans l'outil de suivi (tableur, GitHub Issues, Trello).
2. ASSIGNÉ : Développeur responsable identifié.
3. EN COURS : Correction du code (fix).
4. RÉSOLU : Le développeur estime que c'est bon. Test de régression lancé.
5. VÉRIFIÉ : Le testeur rejoue le cas de test → PASS. Fermeture du ticket.
6. FERMÉ : Validé en recette finale.

Règle d'équipe : on ne ferme jamais un ticket sans un test qui passe.



Cycle Test → Anomalie → Correction → Re-test (régression). La boucle se répète jusqu'au PASS.

5.7 La recette finale : signature et livraison

La **recette** (ou User Acceptance Testing — UAT) est l'acte contractuel de fin de projet. En contexte scolaire, c'est la soutenance devant le jury ; en entreprise, c'est la signature du client.

DÉFINITION Procès-Verbal de Recette (PVR)

Document officiel, signé par le maître d'ouvrage (client/professeur) et le maître d'œuvre (équipe de développement), qui atteste :

1. L'application a été présentée et démontrée.
2. Les tests de validation (scénarios métier) ont été exécutés avec succès (ou avec ano-

malies mineures acceptées et documentées).

3. La documentation est remise : Rapport de réalisation + Guide d'utilisation + Code source commenté + Script BDD.
4. Les identifiants d'accès (admin, professeur, élève) sont communiqués.
5. Réserves éventuelles : liste des anomalies mineures non corrigées mais acceptées (ex. « alignement CSS imparfait sur Safari »), avec engagement de correction ultérieure.

EXEMPLE Modèle de PVR simplifié pour le Bac TI

PROCES-VERBAL DE RECETTE – PROJET "GESTION NOTES LYCEE"

Date : 15/05/2025 | Lieu : Salle Info 2 – Lycee de Nkolbisson

PRESENTS :

- M. LE PROVISEUR (Maitre d'Ouvrage) : _____
- ELEVE NGOUMOU Jean (Chef de projet) : _____
- ELEVE MEKONGO Grace (Dev Backend) : _____
- ELEVE TCHINDA Kevin (Dev Frontend/BDD) : _____

OBJET : Recette de l'application web "GestionNotes v1.0"

1. DEMONSTRATION FONCTIONNELLE (Scenarios VAL-01 a VAL-15) :

- [X] Connexion Professeur / Saisie notes / Validation -> OK
- [X] Connexion Admin / Gestion utilisateurs -> OK
- [X] Generation Bulletins PDF (6eme A) -> OK
- [X] Export Excel des notes -> OK
- [X] Securite : tentative d'injection SQL bloquee -> OK
- [X] Affichage mobile (responsive) -> OK

2. DOCUMENTATION REMISE (Version papier + Cle USB) :

- [X] Rapport de realisation (30 pages) -> OK
- [X] Guide d'utilisation Professeur (5 pages) -> OK
- [X] Guide d'utilisation Admin (3 pages) -> OK
- [X] Code source complet (dossier /src) -> OK
- [X] Script BDD (schema.sql + data_test.sql) -> OK

3. RESERVES ACCEPTEES (Non bloquantes) :

- R-01 : Pied de page PDF legerement decale a l'impression (mineur).
- R-02 : Pas de notification email (non prevu au CDC initial).

4. DECISION :

- [X] APPLICATION RECUE SANS RESERVE MAJEURE

Signature Maitre d'Ouvrage : _____

Signature Maitre d'Ouvre : _____

LESAVAIS-TU ? Le « Bug de l'an 2000 » et le coût de la correction tardive

À la fin des années 1990, le monde a dépensé plusieurs centaines de milliards de dollars pour corriger le « bug de l'an 2000 » (années stockées sur 2 chiffres : « 99 » puis « 00 »). Les études industrielles montrent qu'un bug corrigé en phase de conception coûte 1 ; en phase de test : environ 15 ; en production : jusqu'à 100. C'est pourquoi votre plan de tests, rédigé avant de coder, est l'investissement le plus rentable de votre projet. Au Cameroun, l'ANTIC (Agence Nationale des Technologies de l'Information et de la Communication) promeut les audits de sécurité avant la mise en production des applications administratives en ligne (impots.cm, portails de services publics, etc.).

5.8 Exercice résolu : Écrire un cas de test complet pour « Connexion »**EXERCICE RÉSOLU 5** Cas de test VAL-03 : Connexion Professeur avec identifiants valides

Énoncé : Rédiger le cas de test complet (identification, entrées, résultat attendu, traçabilité) pour le scénario : « Un professeur saisit son email et son mot de passe corrects, et accède à son tableau de bord ». Préciser le jeu d'essai nécessaire en BDD.

SOLUTION

Solution détaillée :

1. Précondition (jeu d'essai en BDD) :

```
-- Table users : le mot de passe est stocké haché (password_hash)
INSERT INTO users (email, password_hash, role, active) VALUES
('prof.martin@lycee.cm', '$2y$10$...hash_de_martin123...', 'professeur', 1);

-- Table professeurs (liée à users)
INSERT INTO professeurs (user_id, nom, prenom, matiere_id) VALUES
(LAST_INSERT_ID(), 'MARTIN', 'Paul', 3); -- 3 = Mathématiques
```

2. Fiche de cas de test :

Champ	Valeur
ID Cas de test	VAL-03
Exigence CDC	EF-01 : Authentification par rôle
Niveau	Validation (Recette)
Description	Connexion réussie d'un professeur avec identifiants valides
Données d'entrée / Actions	1. Ouvrir <code>http://localhost/gestionnotes/login</code> 2. Saisir Email : <code>prof.martin@lycee.cm</code> 3. Saisir Mot de passe : <code>martin123</code> 4. Cliquer sur le bouton « Se connecter »
Résultat attendu	- Redirection (HTTP 302) vers <code>/professeur/dashboard</code> - Session PHP active : <code>\$_SESSION['user_role']</code> vaut 'professeur' - Affichage « Bienvenue M. MARTIN » dans la barre de navigation - Menu latéral : « Mes classes », « Saisir notes », « Mes bulletins »
Résultat obtenu	(À remplir lors de l'exécution)
Verdict	(PASS / FAIL / BLOQUÉ)
Anomalie liée	(ID de la fiche si FAIL, ex. ANOM-12)

3. Vérifications techniques (côté développeur, pour le test d'intégration IT-02) :

- Vérifier en BDD que la date de connexion est mise à jour : `UPDATE users SET last_login = NOW() WHERE email = 'prof.martin@lycee.cm'` a bien été exécuté.
- Vérifier les logs Apache : la requête `POST /login` répond 302 (redirection), pas 200 avec un message d'erreur.
- Tenter d'accéder à `/admin/users` avec ce compte professeur → l'application doit rediriger vers `/login` ou répondre 403 Forbidden (contrôle d'accès par rôle).

5.9 Exercice d'application : À vous de jouer

EXERCICE 4 Rédaction d'un plan de tests pour « Génération Bulletin PDF »

Vous devez tester la fonctionnalité « Génération du bulletin trimestriel PDF pour une classe donnée » (Exigence EF-15).

1. Identifiez 3 partitions d'équivalence pour les entrées (Classe, Trimestre, Données élèves).

2. Pour chaque partition, proposez 1 cas normal, 1 cas limite, 1 cas erroné.
3. Rédigez le tableau des 9 cas de test (ID, Partition, Entrée, Attendu) avec traçabilité vers EF-15.
4. Quelle anomalie critique pourriez-vous découvrir si le PDF est généré mais que la moyenne de la classe est fausse ? Quel test unitaire aurait dû la détecter ?

CORRIGÉ Exercice : Plan de tests Bulletin PDF

1. Partitions d'équivalence :

- P1 — Classe : existante avec notes / existante sans notes / inexistante.
- P2 — Trimestre : valeur valide (1, 2, 3) / valeur hors bornes (0, 4) / valeur non numérique.
- P3 — Données élèves : notes complètes / notes manquantes / coefficient nul (erreur de saisie admin).

2. et 3. Tableau des 9 cas de test :

ID	Partition	Entrée	Attendu
VAL-15a	P1 normal	Classe 6A (30 élèves, notes saisies), Trimestre 2	PDF de 31 pages (1 couverture + 30 élèves), moyennes justes
VAL-15b	P1 limite	Classe 6B (1 seul élève, notes saisies), Trimestre 1	PDF de 2 pages, calcul correct
VAL-15c	P1 erroné	Classe 6Z (inexistante), Trimestre 2	Message « Classe introuvable », pas de PDF, erreur journalisée
VAL-15d	P2 normal	Classe 6A, Trimestre 3	PDF généré, mention « 3 ^e Trimestre »
VAL-15e	P2 limite	Classe 6A, Trimestre 1 (premier)	PDF OK, rang calculé sur 1 seul trimestre
VAL-15f	P2 erroné	Classe 6A, Trimestre 0	Erreur de validation « Trimestre invalide (1-3) »
VAL-15g	P3 normal	Élèves avec notes et coefficients variés	Moyennes pondérées exactes (vérification manuelle sur 3 élèves)
VAL-15h	P3 limite	Élève sans aucune note (absent tout le trimestre)	Moyenne « N/A », pas d'erreur de division par zéro
VAL-15i	P3 erroné	Coefficient = 0 pour une matière	Exception « Coefficient invalide » bloquant la génération

Les 9 cas sont tous tracés vers l'exigence EF-15.

4. Anomalie critique : la moyenne générale de la classe est calculée comme une simple moyenne arithmétique des moyennes d'élèves au lieu d'une moyenne pondérée. Test unitaire manquant : `UT-MoyenneClasse` — appeler `calculerMoyenneClasse($eleves)` avec un jeu où la moyenne arithmétique diffère de la moyenne pondérée. Exemple : 2 élèves, moyennes 10 (coefficient 1) et 20 (coefficient 3) :

$$\text{Moyenne pondérée} = \frac{10 \times 1 + 20 \times 3}{1 + 3} = \frac{70}{4} = 17,5 \quad (\text{et non } 15).$$

Ce test unitaire aurait détecté l'erreur d'algorithme avant l'intégration dans le module PDF.

À RETENIR Check-list finale avant la soutenance (Recette)

- Plan de tests complet (100 % des exigences couvertes).
- Tous les cas de test exécutés : Verdict = PASS (ou réserves mineures signées).
- Aucune anomalie Bloquante ou Majeure ouverte.
- Tests de régression passés après la dernière correction.
- Rapport de réalisation : Contexte, CDC, diagrammes UML (cas d'utilisation, classes, séquence), conception de la BDD (MCD/MLD), choix techniques, difficultés et solutions, plan de tests et résultats, conclusion.
- Guide d'utilisation : captures d'écran annotées, pas-à-pas pour chaque rôle (Admin, Professeur, Élève), FAQ, contacts support.
- Code source propre : indentation, commentaires, pas de `var_dump` oublié, fichier de configuration sensible (`.env`) non versionné.
- BDD fournie : `schema.sql` (structure) + `data_demo.sql` (jeu de démonstration réaliste).
- Démonstration fluide préparée : scénario nominal + 1 scénario d'erreur correctement gérée.

Bonne recette ! Votre rigueur dans les tests fait la différence entre un « devoir » et un « projet professionnel ».

6 DOCUMENTATION : RAPPORT DE RÉALISATION ET GUIDE D'UTILISATION

Les tests sont passés, la recette est signée : ton application de gestion des notes fonctionne. Beaucoup d'élèves s'arrêtent là... et commettent une grave erreur. Une application sans documentation est comme un médicament sans notice : personne ne sait comment s'en servir, per-

sonne ne peut la réparer, et le jury du Baccalauréat ne peut pas l'évaluer. Cette dernière section te montre comment produire les deux documents qui « vendent » ton projet : le **rapport de réalisation** et le **guide d'utilisation**.

6.1 Pourquoi documenter un projet informatique ?

Imagine que tu quittes le lycée et que, six mois plus tard, le proviseur demande à un autre élève d'ajouter une fonctionnalité à ton application. Sans documentation, cet élève devra tout deviner : où est le code, comment est structurée la base de données, pourquoi tel choix a été fait. La documentation résout quatre problèmes concrets :

- **Pérennité** : le projet survit au départ de son auteur. Le code seul ne suffit pas ; il faut expliquer les choix.
- **Maintenance** : corriger un bug ou ajouter un module est dix fois plus rapide quand on dispose du MCD, du dictionnaire de données et des commentaires.
- **Transmission** : le guide d'utilisation permet à n'importe quel utilisateur (le censeur, un enseignant) de se servir de l'application sans formation.
- **Évaluation** : au Baccalauréat TI, le jury juge ta capacité à communiquer sur ton travail, pas seulement à coder.

DÉFINITION Rapport de réalisation

Le **rapport de réalisation** est un document technique et réflexif qui retrace l'ensemble de la démarche du projet : contexte, problématique, cahier des charges, conception (UML, MCD/MLD), réalisation, tests, difficultés rencontrées et solutions apportées. Il s'adresse à un public technique : jury d'examen, enseignants, futurs développeurs.

DÉFINITION Guide d'utilisation

Le **guide d'utilisation** (ou manuel utilisateur) est un document pratique qui explique, pas à pas et avec des captures d'écran, comment installer et utiliser l'application. Il s'adresse à des utilisateurs non informaticiens : il ne contient ni code, ni jargon technique.

ATTENTION Deux documents, deux publics

La confusion la plus fréquente consiste à écrire un seul document « fourre-tout ». Retiens bien : le rapport convainc un technicien que ton travail est rigoureux ; le guide permet à ta grand-mère de saisir une note sans te téléphoner. Vocabulaire, niveau de détail et ton sont différents.

6.2 Structure du rapport de réalisation

Un rapport de projet de Terminale TI fait typiquement entre 20 et 40 pages. Voici la structure attendue par les jurys, en dix rubriques.

MÉTHODE Structurer un rapport de projet en 10 rubriques

1. Page de garde : nom du lycée, année scolaire, titre du projet, noms des membres du groupe, encadreur, logo éventuel.
2. Remerciements : une demi-page maximum, sobre et sincère.
3. Résumé : 10 à 15 lignes présentant le problème, la solution et les résultats. Rédigé en dernier !
4. Table des matières : générée automatiquement (Word : Références → Table des matières), avec numéros de pages.
5. Introduction : contexte (le lycée, la tâche manuelle actuelle), problématique (une question claire, ex. « Comment automatiser le calcul des moyennes et l'édition des bulletins ? ») et annonce du plan.
6. Analyse et cahier des charges : besoins fonctionnels et non fonctionnels, acteurs, contraintes, planning prévisionnel (diagramme de Gantt).
7. Conception : diagrammes UML (cas d'utilisation, classes, séquence), MCD puis MLD de la base de données, dictionnaire de données, maquettes des écrans.
8. Réalisation : technologies choisies (PHP, MySQL, WampServer) et justification, architecture client-serveur, captures d'écran commentées, extraits de code courts et commentés (le code complet va en annexe).
9. Tests et validation : cahier de tests (tableau : test, donnée d'entrée, résultat attendu, résultat obtenu, statut), recette.
10. Difficultés et solutions, conclusion et perspectives : honnêteté intellectuelle sur les bugs rencontrés, bilan, améliorations futures (application mobile, SMS aux parents...). Suivent la bibliographie et les annexes.

EXEMPLE Sommaire chiffré du rapport « Gestion des notes »

Répartition indicative pour un rapport de 32 pages :

N°	Rubrique	Pages
1–4	Garde, remerciements, résumé, table des matières	4
5	Introduction (contexte, problématique)	1
6–9	Analyse et cahier des charges	4
10–16	Conception (UML, MCD/MLD, maquettes)	7
17–24	Réalisation (captures, code commenté)	8
25–27	Tests et validation	3
28–29	Difficultés, conclusion, perspectives	2
30	Bibliographie	1
31–32	Annexes (code complet, guide d'installation)	2

Règle d'or : la conception et la réalisation représentent environ la moitié du rapport.

6.3 Règles de rédaction et honnêteté intellectuelle

Un bon rapport respecte des règles de forme strictes :

- **Clarté** : phrases courtes, orthographe soignée, une idée par paragraphe. Fais relire par un camarade.
- **Pagination** : numérote chaque page (sauf la page de garde) et vérifie que la table des matières correspond.
- **Figures légendées** : chaque capture d'écran ou diagramme porte un numéro et une légende (« Figure 5 : MCD de la base de données »).
- **Citation des sources** : tout texte, image ou morceau de code emprunté (documentation PHP, tutoriel, forum) doit être cité en bibliographie.

ATTENTION Le plagiat est sanctionné

Copier un rapport trouvé sur Internet ou celui d'un ancien élève sans le citer est du **plagiat**. À l'examen, cela peut entraîner l'annulation de l'épreuve. Le jury repère facilement les copier-coller (changement de style, incohérences avec ta démonstration). Citer une source n'est jamais honteux ; la voler, si.

6.4 Structure du guide d'utilisation

Le guide d'utilisation est court (8 à 15 pages) et très visuel. Sa structure type :

1. **Présentation** : à quoi sert l'application, en trois phrases simples.

2. **Prérequis et installation** : matériel nécessaire, étapes d'installation numérotées (ex. « 1. Double-cliquez sur `wampserver.exe`. 2. Cliquez sur Suivant... »), avec une capture par étape délicate.
3. **Prise en main** : un scénario complet pas à pas, de la connexion à la première tâche réussie (ex. « Enregistrer les notes de la classe de Tle TI 2 »).
4. **Description de chaque fonctionnalité** : une sous-section par écran, avec capture annotée (flèches, numéros sur les boutons).
5. **Messages d'erreur et solutions** : tableau « Si vous voyez... alors faites... ».
6. **Contacts** : qui appeler en cas de problème.

ATTENTION Pas de jargon dans le guide

Erreur fréquente : écrire « Exécutez la requête d'insertion après avoir démarré Apache » dans un guide destiné au secrétaire du lycée. Le guide doit être compréhensible par un non-informaticien : on écrit « Cliquez sur le bouton vert Enregistrer », jamais « le script POST persiste l'entité ». Teste ton guide : donne-le à un élève de Première littéraire et observe s'il y arrive seul.

Rapport de réalisation (jury, techniciens)

1. Garde, remerciements, résumé
2. Introduction : contexte, problématique
3. Analyse et cahier des charges
4. Conception : UML, MCD/MLD
5. Réalisation : captures, code commenté
6. Tests, difficultés, conclusion
7. Bibliographie et annexes

Guide d'utilisation (utilisateurs)

1. Présentation simple de l'application
2. Installation pas à pas
3. Prise en main avec captures
4. Chaque fonctionnalité expliquée
5. Erreurs fréquentes et solutions
6. Contacts et assistance

Comparaison des structures : le rapport prouve la démarche technique, le guide accompagne l'utilisateur.

6.5 Préparer la soutenance

Le rapport est écrit ; reste à le défendre oralement. Une soutenance de 10 à 15 minutes suit un plan éprouvé :

1. **Accroche** (1 min) : la situation réelle qui a motivé le projet (« Au lycée de Bafoussam, le calcul des moyennes prenait trois jours... »).
2. **Problématique et objectifs** (2 min) : une diapositive, une question, les besoins principaux.

3. **Conception** (3 min) : montrer le diagramme de cas d'utilisation et le MCD, expliquer deux choix.
4. **Démonstration** (5 min) : faire tourner l'application en direct sur un scénario préparé (saisir des notes, générer un bulletin). Prévois un plan B : une vidéo ou des captures si le réseau tombe en panne.
5. **Bilan** (2 min) : difficultés surmontées, ce que tu as appris, perspectives.

Répète au moins trois fois, chronomètre-toi, et prépare les réponses aux questions classiques du jury : « Pourquoi PHP plutôt que Python ? », « Comment sécurisez-vous les mots de passe ? », « Que se passe-t-il si deux enseignants saisissent en même temps ? »

6.6 Intégration finale : la cohérence globale

Dernière vérification avant de tout imprimer : les quatre livrables doivent raconter la même histoire. Si le cahier des charges promet « l'envoi de SMS aux parents », que la conception le modélise, mais que l'application ne le fait pas, le jury le verra. Utilise une matrice de cohérence :

Besoin (cahier des charges)	Conception	Application	Documentation
Saisir les notes	Cas d'utilisation « Saisir note »	Écran <code>notes.php</code> testé	Guide §4.2, rapport §8
Calculer les moyennes	Méthode <code>moyenne()</code>	Formule SQL vérifiée	Rapport §8.3
Imprimer les bulletins	Diagramme de séquence	Bulletin PDF testé	Guide §4.4

EXERCICE RÉSOLU 6 Vérifier la cohérence d'un projet

Énoncé. Le rapport d'un groupe affirme : « L'application permet à trois rôles (administrateur, enseignant, élève) de se connecter. » Mais le diagramme de cas d'utilisation ne montre que deux acteurs, et le guide d'utilisation ne décrit la connexion que pour l'enseignant. Relève les incohérences et propose une correction.

SOLUTION

Solution. Trois incohérences :

1. Le diagramme de cas d'utilisation doit comporter trois acteurs (Administrateur, Enseignant, Élève) reliés au cas « S'authentifier » ; on corrige le diagramme.
2. Le guide doit contenir une section « Se connecter » valable pour chaque rôle, avec une capture de l'écran de connexion et un tableau des identifiants de démonstration par profil.
3. Le cahier de tests doit inclure un test de connexion par rôle (3 lignes), et le rapport doit les présenter dans la rubrique Tests.

Principe général : toute affirmation du rapport doit être traçable — prouvée par un diagramme, un test ou une capture.

Les 6 phases du projet et leurs livrables



Frise récapitulative : chaque phase produit un livrable qui alimente la suivante; la documentation clôt le cycle.

LESAVAIS-TU ? Le rapport compte autant que le code

Au Baccalauréat TI, la qualité du rapport de réalisation et du guide d'utilisation pèse aussi lourd que le fonctionnement de l'application. Un jury pardonne un petit bug expliqué honnêtement dans la rubrique « difficultés » ; il ne pardonne pas une application parfaite sans aucune documentation. Dans la vie professionnelle aussi : les développeurs de startups camerounaises qui livrent à leurs clients (banques, opérateurs comme MTN ou Orange) un logiciel sans manuel ni dossier technique perdent le contrat de maintenance. Documenter, c'est une compétence qui se paie comptant.

À RETENIR L'essentiel de la section

- Le **rapport de réalisation** (public technique) retrace la démarche en 10 rubriques, de la page de garde aux annexes.
- Le **guide d'utilisation** (public non informaticien) explique l'installation et chaque fonctionnalité pas à pas, avec captures et sans jargon.
- Rédaction : clarté, pagination, figures légendées, sources citées — le plagiat est sanctionné.
- La soutenance suit un plan : accroche, problématique, conception, démonstration, bilan.
- L'exigence finale est la cohérence : cahier des charges ↔ conception ↔ application ↔ documentation.

EXERCICE 5 À toi de jouer

1. Rédige la page de garde et le résumé (15 lignes maximum) du rapport de ton projet « Gestion des notes ».
2. Écris la section « Se connecter » du guide d'utilisation, destinée au censeur (non informaticien), en 5 étapes numérotées.
3. Construis la matrice de cohérence de ton projet avec au moins 5 besoins.

CORRIGÉ Exercice

1. La page de garde contient : MINESEC / nom du lycée / année scolaire / « Projet intégrateur de Terminale TI : Application web de gestion des notes » / noms des membres du groupe / nom de l'encadreur. Le résumé suit le schéma : problème (calcul manuel lent et source d'erreurs) → solution (application web PHP/MySQL en réseau local) → résultats (bulletin généré en moins d'une minute, tests validés). 2. Exemple de rédaction : « 1. Allumez l'ordinateur. 2. Double-cliquez sur l'icône GestionNotes du bureau. 3. Dans la case Nom d'utilisateur, tapez le nom fourni par l'administrateur. 4. Dans la case Mot de passe, tapez votre mot de passe (il s'affiche en points, c'est normal). 5. Cliquez sur le bouton bleu Connexion. Si le message Identifiants incorrects apparaît, vérifiez les majuscules et recommencez. » 3. Matrice à 5 lignes : authentification, saisie des notes, calcul des moyennes, édition des bulletins, sauvegarde de la base — chaque ligne croisée avec un diagramme UML, un test réussi et une section du guide.

7 ACTIVITÉ D'INTÉGRATION

7.1 Objectifs de l'activité

Cette activité d'intégration vise à vous faire vivre, en équipe et sur une version réduite mais complète, l'ensemble du cycle de vie d'un projet informatique professionnel. À l'issue de cette activité, vous serez capable de :

- Analyser un besoin réel et le formaliser dans un **cahier des charges** structuré (fonctionnel et technique) ainsi qu'une **planification** (diagramme de Gantt).
- Concevoir le système d'information à l'aide du langage **UML** (diagramme de cas d'utilisation, diagramme de classes) et modéliser la base de données relationnelle (**MLD**, script SQL).
- Développer une **application web dynamique** en **PHP/MySQL** respectant l'architecture MVC simplifiée (séparation traitement/présentation), sécurisée contre les injections SQL (PDO) et les failles XSS.
- Déployer l'application sur un **réseau local** (configuration IP statique, serveur web Wamp-Server/XAMPP, pare-feu Windows, test d'accès distant via navigateur).
- Élaborer une **stratégie de tests** (plan de tests, cas de tests, recette) et produire la documentation finale : **rapport de réalisation** et **guide d'utilisation**.
- Présenter et défendre votre solution lors d'une **soutenance orale** face au client (le professeur).

7.2 Situation

Le lycée bilingue de **Nkolbisson** (Yaoundé) organise chaque année une « Semaine de l'Orientation » pour les élèves de Terminale. Actuellement, l'information sur les filières universitaires camerounaises (Université de Yaoundé I, II, Ngaoundéré, Dschang, Buea, Douala, Maroua, ENS, ENSET, Mines de Ngoa-Ekelle, IRIC, ESSTIC, etc.) est dispersée : brochures papier obsolètes, sites web institutionnels parfois lents ou incomplets, rumeurs dans les couloirs. L'administration souhaite doter le lycée d'un outil numérique interne, simple, rapide et fiable.

Commande du client (le Proviseur, représenté par le professeur) :

« Nous voulons une application web locale, accessible depuis n'importe quel poste de la salle informatique (réseau LAN câblé, 20 postes clients + 1 poste serveur), sans connexion Internet obligatoire. L'application, nommée **InfoConseil**, doit permettre à un élève de consulter les filières par établissement, par domaine (Sciences, Lettres, Droit, Économie, Technique, Santé) et par niveau de recrutement (Bac, Bac+1, Concours). Un administrateur (le professeur documentaliste) doit pouvoir ajouter, modifier ou supprimer des fiches filières. L'interface doit être sobre, responsive (mobile first pour consultation sur smartphone via Wi-Fi local), aux couleurs du lycée (bleu marine #003366 et or #FFD700). Le tout doit être livré avec un rapport technique et un guide utilisateur. Durée impartie : 4 séances de 2h30 (10h au total). »

Contraintes techniques imposées :

- Serveur : Poste « Serveur » (Windows 10/11 Pro) avec WampServer 3.3.x (Apache 2.4, PHP 8.2, MySQL 8.0).
- Réseau : Adressage statique classe C 192.168.10.0/24. Serveur : 192.168.10.10. Masque : 255.255.255.0. Passerelle : 192.168.10.1 (box/routeur de la salle). DNS : 8.8.8.8.
- Pare-feu Windows : Règle entrante autoriser TCP port 80 (HTTP) pour le profil « Domaine/-Privé ».
- Base de données : Nom `infoconseil_db`, encodage `utf8mb4_general_ci`.
- Sécurité : Connexion PDO avec requêtes préparées, protection XSS via `htmlspecialchars()`, mots de passe hachés (`password_hash()`) pour l'admin.
- Équipe : 4 élèves (Rôles : Chef de projet/Analyste, Concepteur UML/BD, Développeur Backend/PHP, Intégrateur Frontend/Réseau/Testeur). Polyvalence exigée.

7.3 Consignes

Travail en équipe de 4. Chaque élève tient un journal de bord individuel (dates, tâches, difficultés, solutions). La note finale est collective (production) + individuelle (soutenance + journal).

1. Étape 1 — Cadrage et Planification (Séance 1, 2h30) :

- Rédiger le **mini-cahier des charges** (max 5 fonctionnalités principales, acteurs, contraintes techniques, planning).
- Établir le **diagramme de Gantt** prévisionnel sur les 4 séances (jalons : MCD/MLD validé, Base créée, Backend core, Frontend + CSS, Déploiement, Tests, Doc).
- Valider le périmètre avec le « client » (professeur) avant de coder.

2. Étape 2 — Conception UML et Base de Données (Séance 1 fin + Séance 2 début) :

- Dessiner le **diagramme de cas d'utilisation** (acteurs : Élève, Administrateur ; cas d'utilisation : Consulter filières, Rechercher par critères, Voir détail, Gérer filières (CRUD), S'authentifier Admin).
- Dessiner le **diagramme de classes** (classes : `Etablissement`, `Filiere`, `Domaine`, `Admin`, associations, multiplicités, attributs, méthodes principales).
- Produire le **MLD** (norme Merise) et le script SQL de création (CREATE TABLE avec PK, FK, index, ENGINE=InnoDB).
- Exécuter le script sur le serveur MySQL (via phpMyAdmin ou ligne de commande) et insérer un **jeu de test** (minimum 5 établissements, 3 domaines, 15 filières variées, 1 admin).

3. Étape 3 — Développement de l'Application Web (Séance 2 fin + Séance 3) :

- Architecture : `index.php` (routeur simple), `config/bdd.php` (PDO), `models/` (classes DAO), `views/` (templates HTML/PHP), `public/css/style.css`, `public/js/main.js`.
- Page **Accueil** (`index.php`) : Barre de navigation, héros « Bienvenue sur InfoConseil », 3 cartes « Rechercher par Établissement / Domaine / Niveau » lien vers `recherche.php`.

- (c) Page **Recherche** (`recherche.php`) : Formulaire GET (select établissement, select domaine, select niveau, bouton « Filtrer »). Affichage résultats en tableau responsive (cartes sur mobile) : Nom filière, Établissement, Domaine, Niveau, Bouton « Détail » (`detail.php?id=...`).
- (d) Page **Détail** (`detail.php`) : Infos complètes (description, conditions admission, débouchés, contact établissement, lien site web).
- (e) Espace **Admin** (`admin/login.php`, `admin/dashboard.php`, `admin/filiere_form.php`) : Authentification session, CRUD complet sur `filiere` (ajout, modif, suppression avec confirmation JS).
- (f) CSS : Variables CSS pour couleurs, Flexbox/Grid, Media queries (@media (max-width: 768px)).

4. Étape 4 — Déploiement Réseau Local et Tests (Séance 4, 1h30) :

- (a) Configurer l'IP statique du serveur (192.168.10.10). Vérifier le ping depuis un poste client.
- (b) Configurer WampServer : « Put Online » (ou `Require all granted` dans `httpd-vhosts.conf` pour le VirtualHost `infoconseil.local` ou dossier `www/infoconseil`).
- (c) Créer règle Pare-feu Windows (Entrant, Port 80, TCP, Autoriser).
- (d) Depuis un poste client : Ouvrir navigateur, taper `http://192.168.10.10/infoconseil/` (ou `infoconseil.local` si hosts modifié).
- (e) Exécuter le **Plan de tests** (5 cas minimum) : TC-01 Affichage accueil, TC-02 Recherche multi-critères résultats, TC-03 Recherche sans résultat (message), TC-04 Connexion Admin + Ajout filière visible publiquement, TC-05 Tentative injection SQL dans champ recherche (doit échouer/être neutralisée), TC-06 Responsive mobile (outils devtools).
- (f) Corriger les anomalies bloquantes.

5. Étape 5 — Documentation et Soutenance (Séance 4, 1h + hors classe) :

- (a) Rédiger le **Rapport de réalisation** (4 pages max) : 1. Contexte & Objectifs, 2. Analyse (CdC, UML, MLD), 3. Choix techniques & Architecture, 4. Difficultés & Solutions, 5. Bilan personnel/équipe.
- (b) Rédiger le **Guide d'utilisation** (1 page recto) : Captures d'écran annotées, procédure « Élève : Trouver une filière », procédure « Admin : Ajouter une filière », FAQ courte (IP serveur, mot de passe admin par défaut).
- (c) Soutenance orale (5 min + 5 min questions) : Démonstration live, justification choix techniques, retour sur organisation équipe.

7.4 Ressources à mobiliser

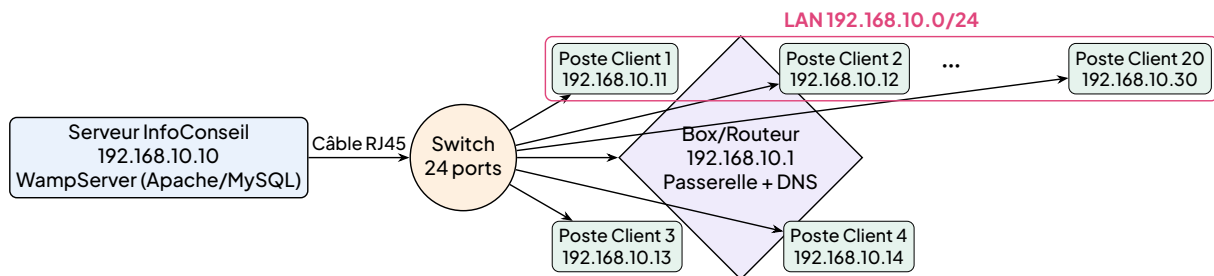
- **UML** : Notation cas d'utilisation (acteur, cas d'utilisation, include, extend, généralisation), Notation classe (attributs : visibilité type [multiplicité], méthodes, associations navigables, agrégation/composition, héritage).

- **Merise / MLD** : Passage Entité-Association → Tables (1:1 fusion ou FK, 1:N FK côté N, N:N table de liaison), Notation `Table(PK, FK, Att1, Att2...)`. Contraintes d'intégrité référentielle (ON DELETE RESTRICT/CASCADE).
- **SQL**: `CREATE DATABASE ... CHARACTER SET utf8mb4, CREATE TABLE ... ENGINE=InnoDB, INSERT INTO, SELECT ... JOIN ... WHERE ... ORDER BY ... LIMIT ..., PREPARE/EXECUTE (côté PHP PDO)`.
- **PHP 8 / PDO**: `new PDO("mysql:host=...;dbname=...;charset=utf8mb4", user, pass, [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]), $stmt = $pdo->prepare("SELECT * FROM filiere WHERE id_etab=? AND id_dom=?"); $stmt->execute([$idEtab, $idDom]); $filieres = $stmt->fetchAll();`.
- **Sécurité Web**: `htmlspecialchars($str, ENT_QUOTES, 'UTF-8')` (sortie HTML), `password_hash($pwd, PASSWORD_DEFAULT)` / `password_verify($pwd, $hash)` (auth), `session_start()`, `_SESSION`, protection CSRF (token dans formulaires admin).
- **Réseau** : Configuration IPv4 statique (Panneau de configuration → Centre réseau → Modifier paramètres carte → IPv4), Commandes `ping`, `ipconfig`, `netstat -an | findstr :80`, Pare-feu Windows (`wf.msc` → Règles entrantes → Nouvelle règle → Port 80 TCP).
- **Outils** : VS Code / Notepad++ (éditeur), Draw.io / StarUML / PlantUML (diagrammes), phpMyAdmin / MySQL Workbench (BD), Navigateur (DevTools F12 : Console, Network, Responsive Design Mode).

LESAVAIS-TU ?**Le numérique au service de l'orientation au Cameroun**

Le projet InfoConseil s'inscrit dans la dynamique du « Cameroun Numérique 2020 » et de la stratégie « École Numérique » portée par le MINPOSTEL et le MINESEC. Des initiatives comme `orientation.cm` (plateforme officielle) ou les applications mobiles développées par des startups camerounaises (ex : Kiro'o Games pour le serious gaming, GiftedMom pour la santé) montrent la voie. Votre projet, bien que local (LAN), reproduit l'architecture classique d'un portail d'orientation : annuaire de formations, moteur de recherche multicritères, back-office d'administration. C'est une compétence directement transférable vers un stage en SSII (ex : Netcom, ActivSpaces incubateur) ou une poursuite d'études en BTS/-DUT Informatique (IRIC, ENSET, IUT Douala).

7.5 Architecture réseau cible (Salle informatique)



Topologie étoile du réseau de la salle informatique : 20 clients + 1 serveur derrière un switch, connectés à la box/routeur (passerelle).

7.6 Diagramme de cas d'utilisation (UML)

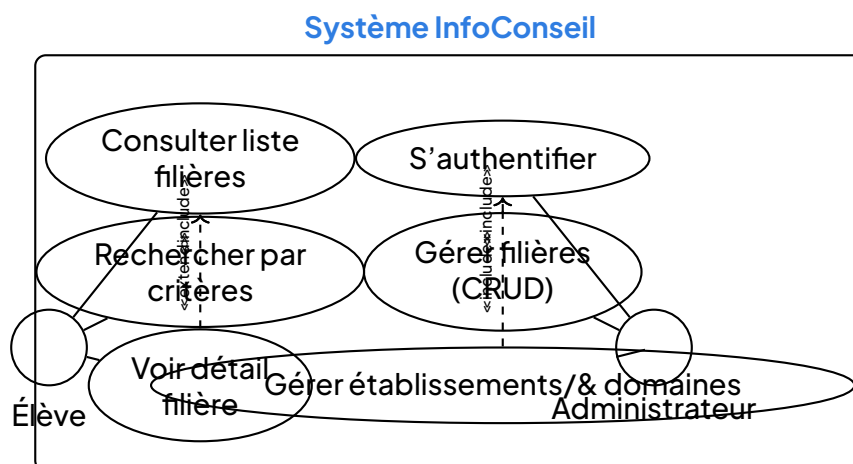


Diagramme de cas d'utilisation : L'élève consulte et recherche; l'administrateur s'authentifie puis gère les données (inclusion de l'authentification).

7.7 Diagramme de classes (UML)

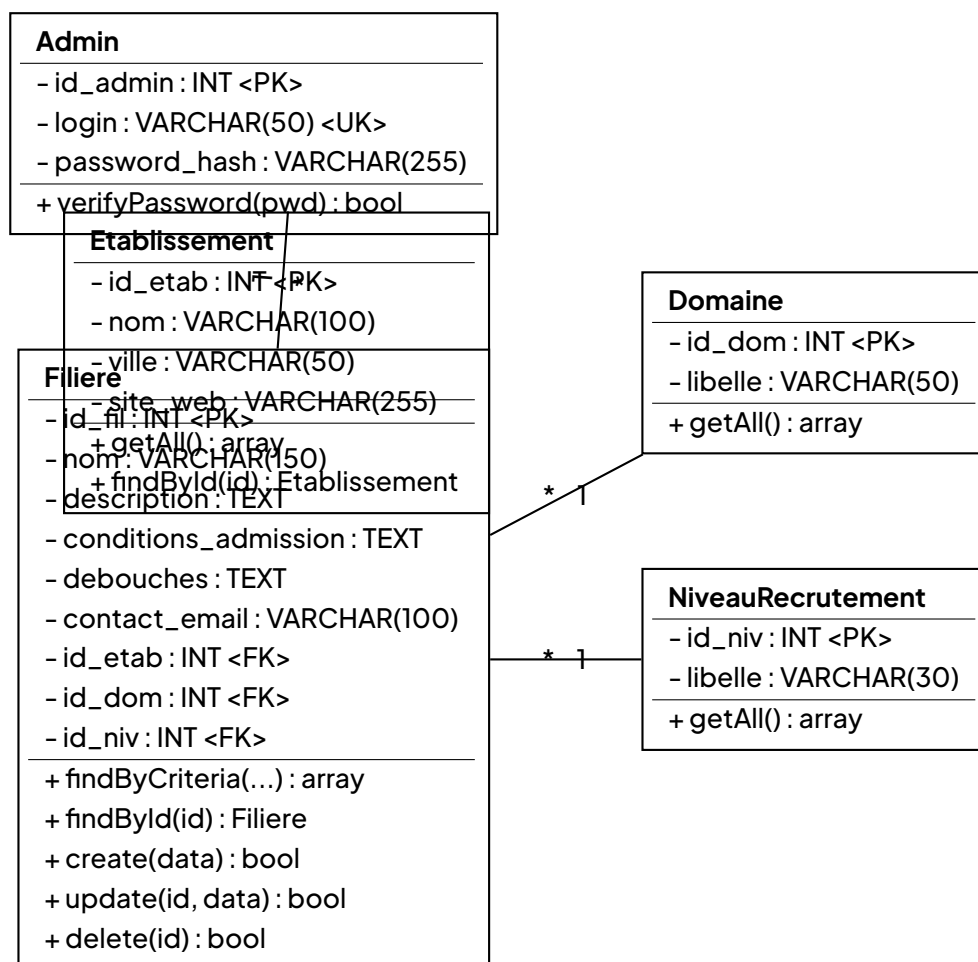


Diagramme de classes du domaine métier. La classe Filiere est centrale, liée aux tables de référence (Etablissement, Domaine, NiveauRecrutement).

7.8 Modèle Logique de Données (MLD) — Norme Merise

PROPRIÉTÉ Règles de passage MCD → MLD (Rappel)

- Entité → Table (PK, Attributs...).
- Relation 1 : N → Clé étrangère (FK) du côté N (ex : filiere porte id_etab).
- Relation N : M → Table de liaison (PK composée des deux FK).
- Relation 1 : 1 → Fusion possible ou FK unique (contrainte UNIQUE).
- Intégrité référentielle : ON DELETE RESTRICT (empêche suppression parent si enfants existent) ou CASCADE.

Table	Structure (PK, FK, Attributs)
etablissement	(<u>id_etab</u> , nom, ville, site_web)
domaine	(<u>id_dom</u> , libelle)
niveau_recrutement	(<u>id_niv</u> , libelle)
filiere	(<u>id_fil</u> , nom, description, conditions_admission, debouches, contact_email, #id_etab, #id_dom, #id_niv)
admin	(<u>id_admin</u> , login, password_hash)

EXEMPLE Script SQL complet — Création BDD, Tables, Index, FK et Jeu de test

```
-- 1. Base de données
CREATE DATABASE IF NOT EXISTS 'infoconseil_db'
  DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE 'infoconseil_db';

-- 2. Tables de référence (pas de FK entrantes)
CREATE TABLE 'etablissement' (
  'id_etab' INT UNSIGNED NOT NULL AUTO_INCREMENT,
  'nom' VARCHAR(100) NOT NULL,
  'ville' VARCHAR(50) NOT NULL,
  'site_web' VARCHAR(255) DEFAULT NULL,
  PRIMARY KEY ('id_etab')
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE 'domaine' (
  'id_dom' TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  'libelle' VARCHAR(50) NOT NULL,
  PRIMARY KEY ('id_dom')
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE 'niveau_recrutement' (
  'id_niv' TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  'libelle' VARCHAR(30) NOT NULL,
  PRIMARY KEY ('id_niv')
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. Table centrale (FK vers les 3 tables de référence)
CREATE TABLE 'filiere' (
  'id_fil' INT UNSIGNED NOT NULL AUTO_INCREMENT,
  'nom' VARCHAR(150) NOT NULL,
  'description' TEXT,
  'conditions_admission' TEXT,
  'debouches' TEXT,
  'contact_email' VARCHAR(100) DEFAULT NULL,
  'id_etab' INT UNSIGNED NOT NULL,
  'id_dom' TINYINT UNSIGNED NOT NULL,
  'id_niv' TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY ('id_fil'),
  KEY 'idx_etab' ('id_etab'),
  KEY 'idx_dom' ('id_dom'),
  KEY 'idx_niv' ('id_niv'),
  CONSTRAINT 'fk_fil_etab' FOREIGN KEY ('id_etab') REFERENCES 'etablissement' ('id_etab') ON DELETE
    RESTRICT ON UPDATE CASCADE,
```

```

CONSTRAINT 'fk_fil_dom' FOREIGN KEY ('id_dom') REFERENCES 'domaine'('id_dom') ON DELETE
    RESTRICT ON UPDATE CASCADE,
CONSTRAINT 'fk_fil_niv' FOREIGN KEY ('id_niv') REFERENCES 'niveau_recrutement'('id_niv') ON DELETE
    RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 4. Table Admin (authentification)
CREATE TABLE 'admin' (
    'id_admin' TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
    'login' VARCHAR(50) NOT NULL UNIQUE,
    'password_hash' VARCHAR(255) NOT NULL,
    PRIMARY KEY ('id_admin')
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 5. Jeu de test minimal (Données de référence)
INSERT INTO 'etablissement' ('nom', 'ville', 'site_web') VALUES
    ('Université de Yaoundé I', 'Yaoundé', 'http://www.univ-yaounde1.com'),
    ('Université de Douala', 'Douala', 'http://www.univ-douala.com'),
    ('ENSET Douala', 'Douala', 'http://www.enset.univ-douala.com'),
    ('IRIC', 'Yaoundé', 'http://www.irc.cm'),
    ('ESSTIC', 'Yaoundé', 'http://www.esstic.cm');

INSERT INTO 'domaine' ('libelle') VALUES
    ('Sciences'), ('Lettres'), ('Droit'), ('Économie/Gestion'), ('Technique'), ('Santé');

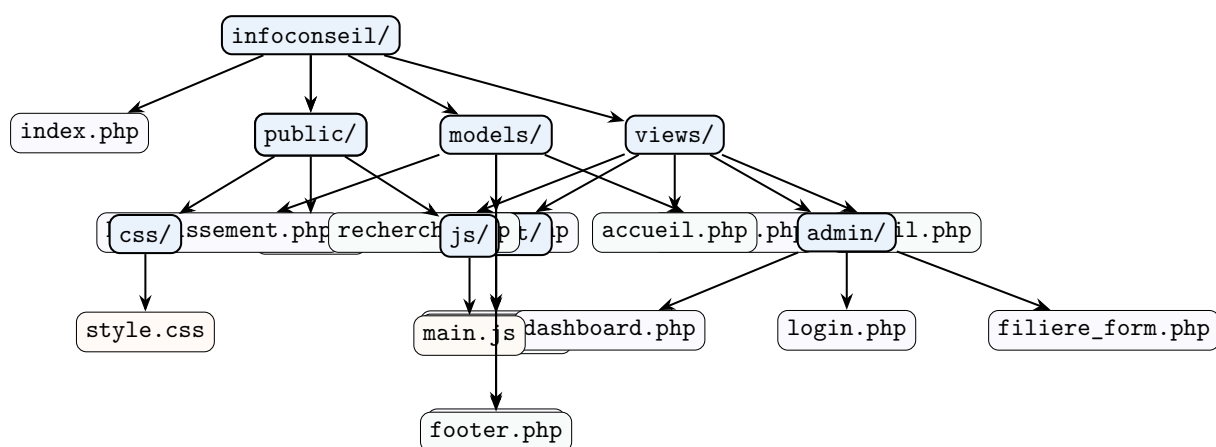
INSERT INTO 'niveau_recrutement' ('libelle') VALUES
    ('Bac'), ('Bac+1'), ('Concours');

-- 6. Filières exemples (15+ lignes recommandées, 4 ici pour l'exemple)
INSERT INTO 'filiere' ('nom', 'description', 'conditions_admission', 'debouches', 'contact_email', 'id_etab', 'id_dom', 'id_niv') VALUES
    ('Licence Sciences Mathématiques', 'Formation générale en mathématiques pures et appliquées.', 'Bac C, D, E, F, G ou équivalent.', 'Enseignement, Banque, Assurance, Data Science.', 'math@univ-yaounde1.com', 1, 1, 1),
    ('Licence Droit Privé', 'Étude du droit civil, pénal, des affaires.', 'Bac A, B, D ou équivalent.', 'Avocat, Magistrat, Juriste d\'entreprise.', 'droit@univ-yaounde1.com', 1, 3, 1),
    ('Cycle Ingénieur Génie Civil', 'Formation d\'ingénieurs en BTP.', 'Concours après Bac+2 (DUT, BTS, Licence) ou Bac S.', 'Ingénieur BTP, Chef de chantier.', 'geniecivil@enset.univ-douala.com', 3, 5, 3),
    ('Licence Journalisme', 'Métiers de l\'information et communication.', 'Concours entrée ESSTIC (Bac+2).', 'Journaliste, Chargé de comm.', 'admission@esstic.cm', 5, 2, 3),
    ('Licence Physique Fondamentale', 'Physique théorique et expérimentale.', 'Bac C, D, E, F.', 'Recherche, Enseignement, Industrie.', 'physique@univ-yaounde1.com', 1, 1, 1),
    ('Licence Économie Gestion', 'Gestion d\'entreprise, comptabilité.', 'Bac A, B, C, D, G.', 'Comptable, Gestionnaire, Banquier.', 'ecogest@univ-douala.com', 2, 4, 1),
    ('Doctorat Médecine', 'Formation médicale générale.', 'Concours très sélectif (Bac+1 validé).', 'Médecin, Spécialiste.', 'medecine@univ-yaounde1.com', 1, 6, 3);

-- 7. Admin par défaut (Mot de passe : 'InfoConseil2025!' hashé via password_hash)
-- Hash généré : $2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi (exemple pour 'password')
-- REMPLACER PAR LE VRAI HASH DE 'InfoConseil2025!' GÉNÉRÉ EN PHP AVANT INSERTION.
INSERT INTO 'admin' ('login', 'password_hash') VALUES
    ('admin', '$2y$10$VOTRE_VRAI_HASH_ICI_GENERE_PAR_PHP');

```

7.9 Architecture logicielle MVC simplifiée (Arborescence)



Arborescence du projet InfoConseil. Point d'entrée unique `index.php` (Front Controller). Séparation stricte : `models/` (DAO), `views/` (Templates), `public/` (Assets statiques).

7.10 Étape 3 — Développement : Points clés du code

MÉTHODE Connexion PDO sécurisée (`config/bdd.php`)

1. Définir le DSN (Data Source Name) avec `charset=utf8mb4`.
2. Configurer les options : `ERRMODE_EXCEPTION` (lever exceptions), `FETCH_ASSOC` (tableaux associatifs), `EMULATE_PREPARES => false` (vraies requêtes préparées côté serveur).
3. Instancier PDO dans un bloc `try/catch`.
4. En cas d'erreur : logger (fichier) et afficher page d'erreur générique (ne pas divulguer les identifiants).

EXEMPLE `config/bdd.php`

```

<?php
// config/bdd.php
// Fichier inclus par le routeur (index.php) pour fournir $pdo aux modèles

try {
    $dsn = 'mysql:host=127.0.0.1;dbname=infoconseil_db;charset=utf8mb4';
    $user = 'root';           // Utilisateur par défaut WampServer
    $pass = '';               // Mot de passe vide par défaut WampServer
    $options = [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ];
}

```

```

        PDO::ATTR_EMULATE_PREPARES => false, // Sécurité : vraies requêtes préparées
    ];
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (PDOException $e) {
    // En production : error_log($e->getMessage());
    // Afficher une page 500 générique (include 'views/errors/500.php'; exit;)
    die('Erreur de connexion à la base de données. Veuillez contacter l\'administrateur.');
```

MÉTHODE **Modèle DAO** — **Recherche multi-critères avec jointures**
(models/Filiere.php)

1. Construire la requête SQL dynamiquement : base `SELECT ... JOIN ... WHERE 1=1`.
2. Ajouter des clauses `AND colonne = ?` uniquement si le filtre est fourni.
3. Collecter les valeurs dans un tableau `$params` dans l'ordre des ?.
4. Préparer (`$pdo->prepare`), Exécuter (`$stmt->execute($params)`), Récupérer (`$stmt->fetchAll()`).
5. Retourner un tableau associatif prêt pour la vue.

EXEMPLE **models/Filiere.php — Méthode findByCriteria**

```

<?php
// models/Filiere.php
require_once __DIR__ . '/../config/bdd.php';

class Filiere {
    private PDO $pdo;

    public function __construct(PDO $pdo) {
        $this->pdo = $pdo;
    }

    /**
     * Recherche filières avec filtres optionnels + jointures pour affichage direct
     * @param int|null $idEtab
     * @param int|null $idDom
     * @param int|null $idNiv
     * @return array Liste des filières (tableaux associatifs avec noms joints)
     */
    public function findByCriteria(?int $idEtab = null, ?int $idDom = null, ?int $idNiv = null) : array {
        $sql = "SELECT f.*, e.nom AS etab_nom, d.libelle AS dom_libelle, n.libelle AS niv_libelle
                FROM filiere f
                JOIN etablissement e ON f.id_etab = e.id_etab
```

```

        JOIN domaine d ON f.id_dom = d.id_dom
        JOIN niveau_recrutement n ON f.id_niv = n.id_niv
        WHERE 1=1";
$params = [];

if ($idEtab !== null) { $sql .= " AND f.id_etab = ?"; $params[] = $idEtab; }
if ($idDom !== null) { $sql .= " AND f.id_dom = ?"; $params[] = $idDom; }
if ($idNiv !== null) { $sql .= " AND f.id_niv = ?"; $params[] = $idNiv; }

$sql .= " ORDER BY e.nom, f.nom LIMIT 50"; // Pagination simple

$stmt = $this->pdo->prepare($sql);
$stmt->execute($params);
return $stmt->fetchAll();
}

public function findById(int $id) : ?array {
    $sql = "SELECT f.*, e.nom AS etab_nom, e.ville AS etab_ville, e.site_web AS etab_site,
        d.libelle AS dom_libelle, n.libelle AS niv_libelle
        FROM filiere f
        JOIN etablissement e ON f.id_etab = e.id_etab
        JOIN domaine d ON f.id_dom = d.id_dom
        JOIN niveau_recrutement n ON f.id_niv = n.id_niv
        WHERE f.id_fil = ?";
    $stmt = $this->pdo->prepare($sql);
    $stmt->execute([$id]);
    return $stmt->fetch() ?: null;
}

// CRUD Admin (create, update, delete) analogues avec requêtes préparées...
public function create(array $data) : bool {
    $sql = "INSERT INTO filiere (nom, description, conditions_admission, debouches, contact_email,
        id_etab, id_dom, id_niv)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
    $stmt = $this->pdo->prepare($sql);
    return $stmt->execute([
        $data['nom'], $data['description'], $data['conditions_admission'],
        $data['debouches'], $data['contact_email'],
        $data['id_etab'], $data['id_dom'], $data['id_niv']
    ]);
}
// update(), delete() à implémenter de même...
}
?>

```

MÉTHODE **Routeur Front Controller** (index.php)

1. `session_start()` tout en haut.
2. Inclure `config/bdd.php` et instancier tous les Modèles (DAO) en leur passant `$pdo`.
3. Récupérer les listes de référence (Établissements, Domaines, Niveaux) nécessaires

aux formulaires de toutes les pages.

4. Router sur `_GET['page']` (défaut 'accueil').
5. Inclure `views/layout/header.php`.
6. Structure `match` (PHP 8) ou `switch` pour inclure la vue correspondante.
7. Inclure `views/layout/footer.php`.

EXEMPLE index.php — Routeur principal

```
<?php
// index.php — Point d'entrée unique (Front Controller)
session_start();
require_once __DIR__ . '/config/bdd.php';
require_once __DIR__ . '/models/Filiere.php';
require_once __DIR__ . '/models/Etablissement.php';
require_once __DIR__ . '/models/Domaine.php';
require_once __DIR__ . '/models/Niveau.php';
require_once __DIR__ . '/models/Admin.php';

// 1. Instanciation des Modèles (DAO)
$filiModel = new Filiere($pdo);
$etabModel = new Etablissement($pdo);
$domModel = new Domaine($pdo);
$nivModel = new Niveau($pdo);
$adminModel = new Admin($pdo);

// 2. Données communes aux vues (listes déroulantes)
$etabissements = $etabModel->getAll();
$domaines = $domModel->getAll();
$niveaux = $nivModel->getAll();

// 3. Routing
$page = $_GET['page'] ?? 'accueil';
$id = isset($_GET['id']) ? (int)$_GET['id'] : 0;

// 4. Traitement POST (Actions Admin) – Simplifié ici
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action'])) {
    // Exemple : if ($_POST['action'] === 'add_filiere') { ... $filiModel->create($_POST); header('Location : ...'); exit; }
    // Vérification token CSRF obligatoire ici !
}

// 5. Données spécifiques à la page courante
$resultats = null;
$filiereDetail = null;

if ($page === 'recherche' && isset($_GET['id_etab'], $_GET['id_dom'], $_GET['id_niv'])) {
    $idEtab = $_GET['id_etab'] !== '' ? (int)$_GET['id_etab'] : null;
    $idDom = $_GET['id_dom'] !== '' ? (int)$_GET['id_dom'] : null;
    $idNiv = $_GET['id_niv'] !== '' ? (int)$_GET['id_niv'] : null;
```

```

    $resultats = $filModel->findByCriteria($idEtab, $idDom, $idNiv);
}
elseif ($page === 'detail' && $id > 0) {
    $filiereDetail = $filModel->findById($id);
    if (!$filiereDetail) { $page = '404'; }
}

// 6. Affichage
include __DIR__ . '/views/layout/header.php';

match($page) {
    'accueil'      => include __DIR__ . '/views/accueil.php',
    'recherche'    => include __DIR__ . '/views/recherche.php',
    'detail'       => include __DIR__ . '/views/detail.php',
    'admin_login'  => include __DIR__ . '/views/admin/login.php',
    'admin_dashboard' => include __DIR__ . '/views/admin/dashboard.php',
    'admin_form'   => include __DIR__ . '/views/admin/filiere_form.php',
    default        => include __DIR__ . '/views/404.php',
};

include __DIR__ . '/views/layout/footer.php';
?>

```

EXEMPLE views/recherche.php — Formulaire GET & Affichage résultats (Extrait)

```

<!-- views/recherche.php -->
<h1 class="page-title">Rechercher une filière</h1>

<form action="index.php" method="GET" class="search-form" role="search">
    <input type="hidden" name="page" value="recherche">
    <div class="form-row">
        <div class="form-group">
            <label for="id_etab">Établissement</label>
            <select name="id_etab" id="id_etab">
                <option value="">--- Tous ---</option>
                <?php foreach ($etablisements as $etab) : ?>
                    <option value="<?=$etab['id_etab'] ?>"
                        <?=(isset($_GET['id_etab']) && $_GET['id_etab'] == $etab['id_etab']) ? 'selected'
: '' ?>>
                        <?=$htmlspecialchars($etab['nom'], ENT_QUOTES, 'UTF-8') ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </div>
        <div class="form-group">
            <label for="id_dom">Domaine</label>
            <select name="id_dom" id="id_dom">
                <option value="">--- Tous ---</option>
                <?php foreach ($domaines as $dom) : ?>
                    <option value="<?=$dom['id_dom'] ?>"
                        <?=(isset($_GET['id_dom']) && $_GET['id_dom'] == $dom['id_dom']) ? 'selected'
: '' ?>>

```

```

        <?= htmlspecialchars($dom['libelle'], ENT_QUOTES, 'UTF-8') ?>
    </option>
    <?php endforeach; ?>
</select>
</div>
<div class="form-group">
    <label for="id_niv">Niveau recrutement</label>
    <select name="id_niv" id="id_niv">
        <option value="">-- Tous --</option>
        <?php foreach ($niveaux as $niv) : ?>
            <option value="<?= $niv['id_niv'] ?>"
                <?= (isset($_GET['id_niv']) && $_GET['id_niv'] == $niv['id_niv']) ? 'selected' : '' ?>>
                <?= htmlspecialchars($niv['libelle'], ENT_QUOTES, 'UTF-8') ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
</div>
<button type="submit" class="btn btn-primary">Filtrer</button>
<a href="index.php?page=recherche" class="btn btn-secondary">Réinitialiser</a>
</form>

<?php if (isset($_GET['id_etab'], $_GET['id_dom'], $_GET['id_niv'])) : ?>
    <hr class="mt-4">
    <h2>Résultats (<span id="count"><?= count($resultats ?? []) ?></span>)</h2>
    <div class="results-grid">
        <?php if (empty($resultats)) : ?>
            <p class="alert alert-info">Aucune filière ne correspond à vos critères.</p>
        <?php else : ?>
            <?php foreach ($resultats as $fil) : ?>
                <article class="card filiere-card">
                    <h3><?= htmlspecialchars($fil['nom'], ENT_QUOTES, 'UTF-8') ?></h3>
                    <p class="meta"><strong>Établissement :</strong> <?= htmlspecialchars($fil['
    etab_nom'], ENT_QUOTES, 'UTF-8') ?></p>
                    <p class="meta"><strong>Domaine :</strong> <?= htmlspecialchars($fil['dom_libelle
    '], ENT_QUOTES, 'UTF-8') ?> | <strong>Niveau :</strong> <?= htmlspecialchars($fil['niv_libelle'],
    ENT_QUOTES, 'UTF-8') ?></p>
                    <a href="index.php?page=detail&id=<?= $fil['id_fil'] ?>" class="btn btn-sm btn-
    outline">Voir détails</a>
                </article>
            <?php endforeach; ?>
        <?php endif; ?>
    </div>
<?php endif; ?>

```

MÉTHODE**Sécurité Admin — Middleware de protection (admin/guard.php)**

1. Démarrer la session (`session_start()`).
2. Vérifier l'existence de `$_SESSION['admin_id']`.
3. Si absent : rediriger vers `index.php?page=admin_login` et exit.

4. (Optionnel) Régénérer l'ID de session périodiquement (`session_regenerate_id(true)`) pour prévenir la fixation de session.

EXEMPLE admin/guard.php

```
<?php
// admin/guard.php — Inclure en TOUT DEBUT de chaque fichier dans admin/ (dashboard, form, actions)
session_start();

if (!isset($_SESSION['admin_id'])) {
    // Redirection vers login avec message "Accès réservé"
    header('Location : /infoconseil/index.php?page=admin_login&error=auth_required');
    exit;
}

// Sécurité additionnelle : vérifier l'IP ou User-Agent ? Pas nécessaire pour mini-projet.
// Régénération ID session (ex : à chaque requête sensible ou toutes les 15 min)
if (!isset($_SESSION['last_regeneration']) || time() - $_SESSION['last_regeneration'] > 900) {
    session_regenerate_id(true);
    $_SESSION['last_regeneration'] = time();
}
?>
```

7.11 Étape 4 — Déploiement Réseau Local : Procédure pas à pas

EXPÉRIENCE Configuration du poste Serveur (Windows 10/11 + WampServer)

Objectif : Rendre l'application accessible sur `http://192.168.10.10/infoconseil/` depuis n'importe quel poste client du LAN.

Matériel / Logiciels : Poste Serveur (Windows Pro), WampServer 3.3.x installé dans `C:\wamp64`, câble RJ45, Switch, 1 Poste Client pour test.

Manipulation :

1. IP Statique : Panneau de configuration → Réseau et Internet → Centre réseau et partage → Modifier les paramètres de la carte → Clic droit Ethernet → Propriétés → IPv4 → Propriétés.
 - Adresse IP : 192.168.10.10
 - Masque : 255.255.255.0
 - Passerelle : 192.168.10.1
 - DNS : 8.8.8.8 (et 8.8.4.4)
2. Profil Réseau (CRITIQUE) : Paramètres → Réseau et Internet → Ethernet → Profil ré-

seau : Privé (pas Public). Si Public, le pare-feu bloquera la règle entrante même si elle existe.

3. WampServer : Icône verte (services démarrés). Clic gauche → Put Online (passe l'icône en verte avec flèche blanche). OU (meilleur pour VirtualHost) : Modifier C:\wamp64\bin\apache\apache2.4.x\conf\extra\httpd-vhosts.conf :

```
<VirtualHost *:80>
    ServerName infoconseil.local
    DocumentRoot "c:/wamp64/www/infoconseil"
    <Directory "c:/wamp64/www/infoconseil/">
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted <!-- ESSENTIEL pour accès LAN -->
    </Directory>
</VirtualHost>
```

Puis redémarrer les services (Clic gauche → Restart All Services).

4. Pare-feu Windows (wf.msc) :
 - (a) Règles de trafic entrant → Nouvelle règle → Port → Suivant.
 - (b) TCP, Ports locaux spécifiques : 80 → Suivant.
 - (c) Autoriser la connexion → Suivant.
 - (d) Profils : Domaine, Privé (décocher Public) → Suivant.
 - (e) Nom : HTTP InfoConseil LAN → Terminer.
5. Test Local (Serveur) : Navigateur → `http://localhost/infoconseil/` ou `http://192.168.10.10/infoconseil/`. Page d'accueil OK.
6. Test Distant (Client) : Sur un poste client (IP 192.168.10.x) :
 - (a) `ping 192.168.10.10` → Réponses attendues (TTL=128).
 - (b) Navigateur → `http://192.168.10.10/infoconseil/` → Page d'accueil affichée, CSS/JS chargés.

Observations / Interprétation : Si ping OK mais page web inaccessible : Vérifier Pare-feu (règle active ? profil Privé ?), WampServer "Online" (icône verte), Apache `httpd.exe` en écoute sur `0.0.0.0:80` (`netstat -an | findstr :80`). Si ping KO : Vérifier câble, switch, IP client (même /24), pare-feu client (autoriser ICMPv4 entrant).

7.12 Plan de Tests & Recette (Extrait)

ID	Scénario	Données / Actions	Résultat Attendu	Critère
TC-01	Accueil accessible LAN	Client → <code>http://192.168.10.10</code>	Page d'accueil (Héros 3 cartes, nav) s'affiche, CSS/JS chargés, pas erreur 404/500	Fonctionnel
TC-02	Recherche multi-critères	Sélectionner « Univ. Yaoundé I » + « Sciences » + « Bac », Filtrer	Résultats : Licence Maths + Physique (min 2), liens détail fonctionnels	Fonctionnel
TC-03	Recherche sans résultat	Sélectionner « ENSET Douala » + « Santé » + « Bac », Filtrer	Message « Aucune filière ne correspond... », pas d'erreur PHP (tableau vide géré)	Robustesse
TC-04	Auth Admin + CRUD	<code>/admin_login</code> → login :admin / pwd :Info-Conseil2025! → Dashboard → Ajouter « Test Filière » → Déconnexion → Recherche publique	Connexion OK (session), Ajout OK (flash success), Déconnexion OK (session destroy), Nouvelle filière visible en public	Sécurité / CRUD
TC-05	Injection SQL (Sécurité)	URL <code>?id_etab=1' OR '1'='1</code> ou tentative dans formulaire	Erreur PDO gérée (page blanche ou message générique), PAS de dump BDD, PAS de données non autorisées. Requête préparée neutralise.	Sécurité (OWASP Top 10)
TC-06	Responsive Mobile	DevTools (F12) → Toggle Device Toolbar (iPhone 12 / Galaxy S20)	Nav en burger/-menu, formulaires empilés verticalement, cartes résultats plein écran, lisible sans zoom horizontal, boutons tactiles ≥ 44px	Ergonomie / UI
TC-07	Intégrité Référentielle	Tentative suppression Établissement ayant des Fi-	MySQL bloque : Error 1451: Cannot delete or update a	BDD / Intég

7.13 Documentation : Structure du Rapport de réalisation (4 pages max)

DÉFINITION Contenu attendu du Rapport Technique

1. Page 1 — Contexte & Analyse. Présentation du lycée, problème (info dispersée), solution (InfoConseil LAN), Acteurs identifiés, Cahier des charges résumé (5 User Stories), Diagramme de Gantt Réalisé vs Prévisionnel (écarts justifiés : ex : « CSS responsive a pris 1h de plus que prévu »).
2. Page 2 — Conception. Diagramme de Cas d'Utilisation (image vectorielle SVG/PDF), Diagramme de Classes (image), MLD (tableau Merise), Dictionnaire de données complet (Table, Champ, Type, Taille, Contrainte, Description). Justification choix techniques : PHP 8/PDO natif (pas de framework pour légèreté et maîtrise), MVC simplifié (Front Controller), CSS natif (Variables, Grid/Flexbox), utf8mb4.
3. Page 3 — Réalisation & Difficultés. Architecture dossiers (arborescence), Extraits de code commentés (PDO config, Requête préparée dynamique, Protection XSS `htmlspecialchars`, Hachage `password_hash`, Middleware Guard). Difficulté majeure 1 : Pare-feu Windows bloquait malgré règle créée → Cause : Profil réseau « Public » au lieu de « Privé » → Solution : Paramètres → Réseau → Ethernet → Profil Privé. Difficulté 2 : Caractères spéciaux (é, à, η, Ξ) corrompus → Cause : DSN PDO sans `charset=utf8mb4` ou Table en `utf8_general_ci` → Solution : DSN `charset=utf8mb4 + Tables utf8mb4_general_ci`.
4. Page 4 — Tests, Bilan & Perspectives. Tableau plan de tests (7 cas), Taux de réussite (ex : 7/7 = 100% après correction TC-06 CSS). Bilan équipe (répartition rôles, communication via Git/GitHub ou partage Windows, réunions quotidiennes 15min). Perspectives : Pagination AJAX (infinite scroll), Export PDF fiche filière, API REST JSON pour future appli mobile Flutter/React Native, Hébergement cloud (OVH, MTN Cloud Cameroon) avec HTTPS (Let's Encrypt).

7.14 Guide d'utilisation (Aperçu 1 page recto)

EXEMPLE INFO CONSEIL — GUIDE RAPIDE UTILISATEUR v1.0

ACCÈS APPLICATION

Ouvrir navigateur (Chrome, Firefox, Edge) → Taper : `http://192.168.10.10/infoconseil/`
(Ajouter aux favoris pour accès rapide)

ÉLÈVE — TROUVER UNE FILIÈRE

1. Page d'accueil : Cliquer sur la carte « Rechercher une filière » ou Menu Recherche.
2. Choisir un ou plusieurs filtres : Établissement, Domaine, Niveau.
3. Cliquer bouton Filtrer.
4. Résultats : Parcourir les cartes. Cliquer « Voir détails » sur la filière choisie.
5. Fiche détail : Lire Description, Conditions d'admission, Débouchés, Contact, Site web.

ADMINISTRATEUR — GÉRER LES FILIÈRES

1. Menu Admin (en haut à droite) → Page de connexion.
2. Identifiants par défaut : login: admin mdp: InfoConseil2025!
3. Dashboard : Liste complète des filières (boutons Modifier / Supprimer par ligne).
4. Ajouter : Bouton « + Nouvelle filière » → Remplir formulaire (listes déroulantes pré-remplies).
5. Valider → Message vert « Filière ajoutée avec succès ».
6. Déconnexion : Lien « Déconnexion » en haut à droite.

FAQ / DÉPANNAGE RAPIDE

- « Page inaccessible / Temps dépassé » ? Vérifier : Câble RJ45 branché, IP poste client en 192.168.10.x, ping 192.168.10.10 OK, Icône WampServer VERTE, Pare-feu Windows : Règle « HTTP InfoConseil » activée sur profil Privé.
- « Erreur Connexion BDD » ? Vérifier service MySQL : Icône WampServer → MySQL → Service administration → Démarrer (ou Redémarrer).
- « Mot de passe admin oublié » ? Ouvrir phpMyAdmin (`http://192.168.10.10/phpmyadmin/`), table admin, éditer ligne admin, coller nouveau hash généré par `password_hash('NouveauMdp2025!', PASSWORD_DEFAULT)`.
- « Caractères bizarres (≡) » ? Vérifier encodage : Fichiers PHP sauvés en UTF-8 (VS Code : bas droite), DB/Tables en `utf8mb4_general_ci`, DSN PDO `charset=utf8mb4`.

ATTENTION Pièges classiques & Erreurs fréquentes (Check-list de relecture)

- PHP `header()` après sortie HTML : `session_start()` et `header('Location: ...')` doivent être avant tout `echo`, espace blanc, ou balise HTML. Utiliser `ob_start()` en tout début de `index.php` si nécessaire.
- Variable `$resultats` indéfinie : Dans `index.php`, initialiser `$resultats = null;` avant le

match. La vue teste `isset($_GET[...])` pour savoir si afficher la grille.

- **CSRF Admin manquant** : Tout formulaire POST en admin (ajout, modif, suppression) doit contenir un champ caché `<input type="hidden" name="csrf_token" value="<?=$_SESSION['csrf_token'] ?>">` vérifié côté serveur (`hash_equals`).
- **Profil Réseau « Public »** : Windows classe les réseaux non identifiés en « Public ». Le pare-feu bloque tout entrant. Solution : Paramètres → Réseau → Ethernet → Profil Privé.
- **Encodage UTF-8** : `utf8` (3 octets) ne gère pas les émojis ni certains caractères africains étendus. Toujours `utf8mb4` (4 octets) : DB, Tables, DSN PDO, `htmlspecialchars(..., 'UTF-8'), <meta charset="utf-8">`.
- **Injection SQL via LIMIT / ORDER BY** : Les marqueurs `?` ne fonctionnent pas pour les identificateurs (noms de colonnes, tables) ni pour `LIMIT ?` (émulation désactivée). Valider `$orderBy` contre une liste blanche (`in_array`) avant concaténation.
- **Mot de passe en clair dans le code** : Jamais de `$pass = '...'` en dur dans le code source (Git). Utiliser variables d'environnement (`_ENV`) ou fichier `.env` (non versionné) en production. Pour le TP : `root/'` accepté car local uniquement.

EXERCICE RÉSOLU 7 **Projet InfoConseil — Synthèse des livrables attendus**

Énoncé : Concevoir, développer, déployer et documenter l'application web InfoConseil (orientation filières universitaires camerounaises) en équipe de 4 sur 10h (4 séances), en respectant le cahier des charges client (LAN, MVC, Sécurité, Responsive, Doc).

SOLUTION

Solution complète (Résumé des livrables validants) :

1. Cahier des charges & Planning (Séance 1)

- 5 User Stories (US-01 à US-05) validées par le client (Professeur).
- Diagramme de Gantt prévisionnel (4 séances) avec jalons : MLD validé (fin S1), BDD créée (début S2), Backend Core (fin S2), Frontend + CSS (S3), Déploiement + Tests (S3-S4), Doc + Soutenance (S4).

2. Conception UML & MLD (Séances 1-2)

- Diagramme Cas d'Utilisation : Acteurs Élève/Admin, UC (Consulter, Rechercher, Détail, Auth, CRUD), relations include/extend.
- Diagramme de Classes : 5 classes (Etablissement, Domaine, NiveauRecrutement, Filiere, Admin), attributs typés, multiplicités (1:N), méthodes DAO.
- MLD Merise : 5 tables, PK, FK, Index, ENGINE=InnoDB, utf8mb4, ON DELETE RESTRICT.
- Script SQL exécuté, jeu de test inséré (5 étab, 6 domaines, 3 niveaux, 15+ filières, 1 ad-

min).

3. Développement PHP/MySQL MVC (Séances 2–3)

- **Architecture** : `index.php` (Front Controller), `config/bdd.php` (PDO singleton), `models/` (DAO purs), `views/` (templates PHP/HTML), `public/` (CSS/JS).
- **Sécurité** : `PDO::ATTR_EMULATE_PREPARES=false`, `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')` **systématique**, `password_hash/verify`, `session_start()`, **Middleware** `admin/guard.php`, **Token CSRF** formulaires `admin`.
- **Fonctionnalités** : Accueil, Recherche multi-critères (GET, jointures SQL), Détail (ID URL), Admin CRUD complet (POST-Redirect-GET pattern).
- **Frontend** : **CSS Variables** (`--primary: #003366; --gold: #FFD700;`), **Grid/Flexbox**, `@media (max-width: 768px)` (**Mobile First**), **JS léger** (confirmation suppression, toggle menu burger).

4. Déploiement LAN & Tests (Séance 4)

- **IP Serveur fixe** `192.168.10.10`, **Profil réseau Privé**.
- **WampServer** `Require all granted` (**VirtualHost** ou **alias**), services **Apache/MySQL** démarrés.
- **Pare-feu Windows** : Règle Entrante TCP/80, Profils **Domaine/Privé**.
- **Tests croisés** : 7 Cas de Test (TC-01 à TC-07) tous **PASS** (Recette validée).

5. Documentation & Soutenance

- **Rapport** 4 pages (Contexte, Conception, Réalisation/Difficultés, Tests/Bilan/Perspectives) + **Annexes** (Code source complet, MLD, UML).
- **Guide** 1 page recto (**Visuel**, Procédures pas-à-pas, FAQ Dépannage).
- **Soutenance** 10 min : **Démo live**, **Justification technique** (Pourquoi PDO? Pourquoi **RESTRICT**? Pourquoi `utf8mb4`?), **Retour équipe** (Git, communication, gestion temps).

7.15 Bilan de l'activité

Cette activité d'intégration **InfoConseil** a permis de mettre en évidence la nécessité absolue de **rigueur méthodologique** avant d'écrire la première ligne de code : un cahier des charges flou ou un MLD incomplet entraîne des allers-retours coûteux en développement (refactoring de la BDD, modification des jointures SQL, casse de l'interface). Vous avez constaté que la **séparation des couches** (Modèle/Vue/Contrôleur), même simplifiée, est le seul garde-fou contre le « code spaghetti » ingérable en binôme ou quatuor.

Sur le plan **réseau**, la configuration de l'IP statique, du pare-feu Windows (profil Privé vs Public) et de la directive `Require all granted` sont des compétences « invisibles » mais bloquantes : une application parfaite en local (`localhost`) qui ne répond pas sur `192.168.10.10` est une application **non livrée**. Le plan de tests structuré (TC-01 à TC-07) a transformé le « ça marche sur ma machine » en une **preuve de recette** traçable, exigée par tout client profession-

nel.

Enfin, la **documentation** (Rapport + Guide) n'est pas une corvée de fin de projet : c'est la **garantie de maintenabilité**. Dans 6 mois, un autre élève (ou vous-même) devra corriger un bug ou ajouter une fonctionnalité (ex : « Favoris élève », « Notification nouvelle filière »). Sans MLD à jour, sans dictionnaire de données, sans guide d'installation (IP, BDD, Pare-feu), le projet devient « legacy » ingérable.

Vous avez ainsi validé, en situation quasi-réelle, les trois piliers de la Terminale TI :

1. **Programmation** : PHP 8 orienté objet, PDO, sécurité (SQLi, XSS, CSRF), architecture MVC simplifiée.
2. **Réseaux/Sécurité** : Adressage LAN statique, services (HTTP/DNS), pare-feu hôte, déploiement serveur web, tests d'intrusion basiques (injection SQL).
3. **Systèmes d'Information / BDD** : Merise (MCD/MLD), UML (Cas d'utilisation, Classes), SQL relationnel (Jointures, FK, Index), intégrité référentielle, données de référence.

Cette expérience constitue la **pièce maîtresse de votre portfolio** pour le Baccalauréat et vos futures études (BTS, DUT, Licence Pro, Écoles d'ingénieurs). Conservez précieusement le code (Git/GitHub), le rapport PDF et le guide. Ils prouvent votre capacité à **livrer un produit complet, testé, documenté et déployé**. Félicitations pour ce travail d'équipe exigeant !

8 MÉTHODES ET EXERCICES RÉSOLUS

9 LEÇON 15 : PROJET INTÉGRATEUR — CONCEPTION ET DÉVELOPPEMENT D'UNE APPLICATION INFORMATIQUE

9.1 Méthode 1 : Rédiger un cahier des charges et planifier un projet

MÉTHODE Rédiger le cahier des charges d'un projet informatique

Le **cahier des charges** est le document contractuel qui décrit ce que l'application doit faire, pour qui, et dans quelles contraintes. Pour le rédiger correctement :

1. Présenter le contexte et le besoin : qui est le client (ex. le proviseur du lycée), quel problème concret l'application résout-elle ?
2. Définir les objectifs : formuler des objectifs précis et mesurables (ex. « permettre la saisie des notes de 1200 élèves en moins de 2 jours »).
3. Lister les besoins fonctionnels : ce que le système doit faire (saisir, consulter, imprimer un bulletin, authentifier un utilisateur). Les numéroter (BF1, BF2, ...).
4. Lister les besoins non fonctionnels : contraintes de qualité (temps de réponse, sécurité des mots de passe, compatibilité navigateurs, accessibilité). Les numéroter (BNF1, BNF2, ...).
5. Préciser les contraintes : budget, matériel disponible (salle TI du lycée), délais, technologies imposées (PHP/MySQL, réseau local, WampServer).
6. Planifier : découper le projet en tâches, estimer leur durée et les ordonner dans un tableau ou un diagramme de Gantt (jalons, chemin critique).
7. Définir les critères d'acceptation : comment le client vérifiera-t-il que le travail est conforme (scénarios de recette, jeu d'essai) ?

Réflexe Bac : un cahier des charges sans besoins fonctionnels numérotés ni planning est incomplet. Toujours distinguer fonctionnel (ce que fait le système : verbes d'action sur les données) et non fonctionnel (comment il le fait : performance, sécurité, ergonomie).

EXERCICE RÉSOLU 8 Cahier des charges d'une application de gestion de bibliothèque

Le lycée bilingue de Nkongsamba veut informatiser sa bibliothèque (environ 5000 ouvrages, 800 élèves abonnés). Le proviseur te confie le projet pour une durée de 8 se-

maines.

1. Rédige le contexte et trois besoins fonctionnels de l'application.
2. Donne deux besoins non fonctionnels pertinents.
3. Propose un planning simplifié du projet sur 8 semaines.

SOLUTION

1. Contexte et besoins fonctionnels.

Contexte : la bibliothèque du lycée gère actuellement les prêts sur un registre papier, ce qui provoque des pertes d'ouvrages et l'impossibilité de connaître rapidement les livres disponibles. Le proviseur souhaite une application web accessible depuis les postes de la salle TI.

Besoins fonctionnels (numérotés, formulés par des verbes d'action) :

- BF1 : le bibliothécaire doit pouvoir enregistrer, modifier et supprimer un ouvrage (titre, auteur, cote, nombre d'exemplaires).
- BF2 : le bibliothécaire doit pouvoir enregistrer un prêt (abonné, ouvrage, date de retour prévue) et un retour effectif.
- BF3 : tout utilisateur (élève, enseignant) doit pouvoir rechercher un ouvrage par titre ou auteur et voir sa disponibilité en temps réel.

2. Besoins non fonctionnels.

- BNF1 (sécurité) : l'accès aux fonctions d'administration (BF1, BF2) est protégé par une authentification forte ; les mots de passe sont hachés (bcrypt/argon2) ; gestion des rôles (bibliothécaire / lecteur).
- BNF2 (performance) : une recherche d'ouvrage (BF3) doit répondre en moins de 2 secondes sur le réseau local du lycée (LAN 100 Mbit/s minimum).
- BNF3 (ergonomie) : l'interface doit être responsive (utilisable sur poste fixe et tablette) et respecter les contrastes d'accessibilité (WCAG AA).

3. Planning sur 8 semaines (diagramme de Gantt simplifié).

Semaines	Tâches principales et livrables
S1	Analyse du besoin, rédaction du cahier des charges, validation par le fournisseur (signature)
S2	Conception : diagrammes UML (cas d'utilisation, classes, séquence), MCD/MLD, script SQL de création
S3	Installation environnement (WampServer), création base MySQL, maquettes HTML/CSS (wireframes)
S4 – S5	Développement back-end PHP : authentification, CRUD ouvrages, gestion prêts/retours (PDO, requêtes préparées)
S6	Développement front-end : pages catalogue, tableau de bord bibliothécaire, AJAX pour recherche live
S7	Tests unitaires (PHPUnit ou scripts manuels), tests d'intégration, correction des anomalies, hardening sécurité
S8	Recette utilisateur avec le bibliothécaire (PV de recette), rédaction rapport de réalisation + guide d'utilisation, déploiement sur serveur local

Conclusion : le cahier des charges fixe un périmètre clair (3 besoins fonctionnels prioritaires), des contraintes mesurables (2 s, hachage, responsive) et un planning réaliste de 8 semaines avec des jalons validables : il servira de référence lors de la recette finale.

9.2 Méthode 2 : Concevoir le système d'information avec UML

MÉTHODE Construire un diagramme de cas d'utilisation et un diagramme de classes

A. Diagramme de cas d'utilisation (vue fonctionnelle, externe) :

1. Identifier les **acteurs** : toute entité externe qui interagit avec le système (humain ou système tiers). Un acteur n'est jamais une donnée interne. Stéréotyper si besoin (`<<actor>>`).
2. Identifier les **cas d'utilisation** : chaque grand service rendu par le système, formulé par un verbe à l'infinitif (« S'authentifier », « Emprunter un ouvrage », « Générer un rapport »).
3. Relier chaque acteur aux cas qu'il déclenche (association simple) ; vérifier que tout cas est relié à au moins un acteur.
4. Ajouter si besoin les relations `<<include>>` (cas obligatoirement inclus, ex. « S'authentifier » inclus dans « Emprunter ») et `<<extend>>` (cas optionnel, ex. « Prolonger un prêt » étend « Emprunter »).
5. Dessiner la frontière du système (rectangle) pour délimiter le périmètre.

B. Diagramme de classes (vue statique, structurelle) :

1. Repérer les noms importants du cahier des charges (entités métier) : ce sont les classes candidates.
2. Pour chaque classe, lister les **attributs** avec leurs types (nom : String, dateRetour : Date, nbExemplaires : Integer). Préciser la visibilité (– privé, + public, # protégé).
3. Ajouter les **méthodes** (opérations) significatives : emprunter(), estDisponible(), calculerRetard().
4. Tracer les associations avec les **multiplicités** : 1 (un et un seul), 0..1 (zéro ou un), 1..* (au moins un), 0..* (zéro ou plusieurs). Nommer l'association (ex. « effectuée »).
5. Identifier les classes-association si l'association porte des attributs (ex. Pret porte datePret, dateRetour).

Réflexe Bac : une multiplicité se lit dans les deux sens : « un Abonne effectue 0..* Pret ; un Pret concerne 1 Abonne ». La multiplicité se place à l'extrémité opposée de la classe qu'elle décrit.

EXERCICE RÉSOLU 9**Modélisation UML de la bibliothèque de Nkongsamba**

On reprend le projet de bibliothèque. On distingue deux profils : le bibliothécaire (qui gère les ouvrages et les prêts) et l'élève abonné (qui consulte le catalogue).

1. Dessine le diagramme de cas d'utilisation.
2. Propose un diagramme de classes avec les classes Abonne, Ouvrage et Pret, leurs attributs essentiels et les multiplicités.

SOLUTION**1. Diagramme de cas d'utilisation.**

Acteurs : Bibliothecaire et Eleve (abonné). **Cas d'utilisation** : S'authentifier, Gérer les ouvrages, Enregistrer un prêt, Enregistrer un retour, Consulter le catalogue. **Relation** <<include>> : Enregistrer un prêt <<include>> S'authentifier.

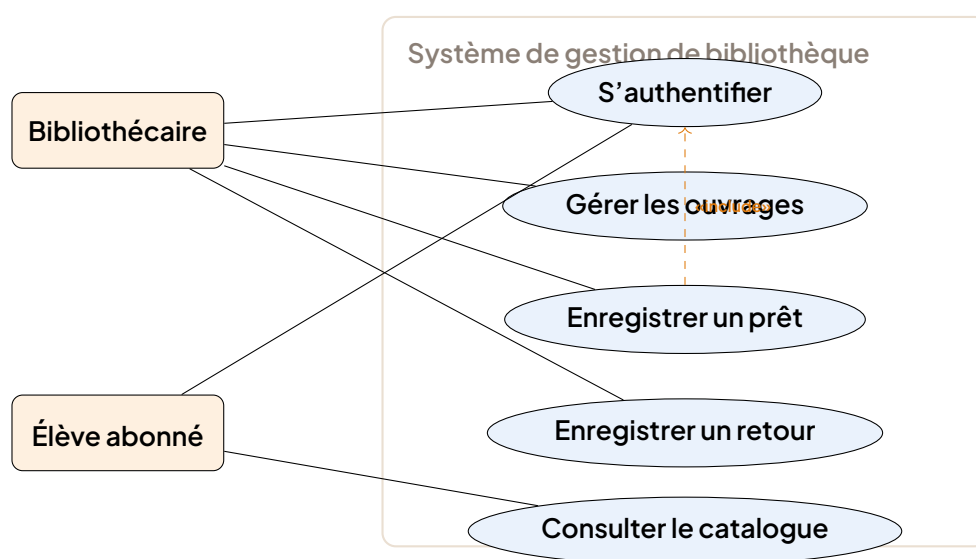


Diagramme de cas d'utilisation du système de gestion de bibliothèque

Justification : seul le bibliothécaire gère les ouvrages et les prêts (besoins BF1 et BF2) ; l'élève ne fait que consulter (BF3). L'authentification est commune aux deux acteurs (inclusion dans les cas sensibles).

2. Diagramme de classes.

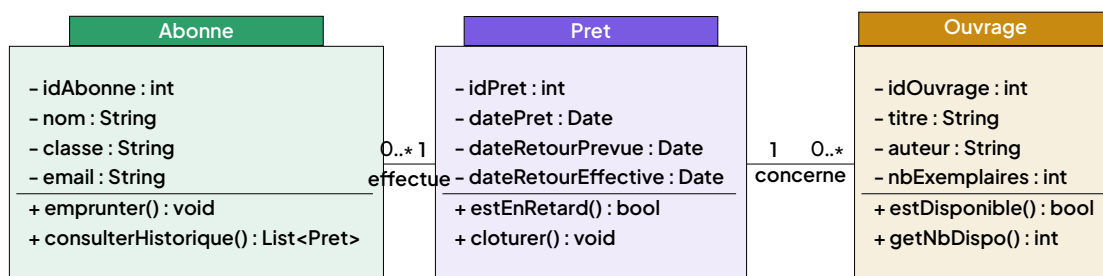


Diagramme de classes : un abonné effectue plusieurs prêts, chaque prêt concerne un ouvrage

Lecture des multiplicités : un Abonne peut avoir 0..* prêts ; un Pret concerne exactement 1 abonné et 1 ouvrage ; un Ouvrage peut apparaître dans 0..* prêts (au fil du temps). La classe Pret est une classe-association : elle porte les attributs propres à la relation (dates), ce qui prépare directement le schéma de la base de données (table de liaison enrichie).

9.3 Méthode 3 : Passer du diagramme de classes à la base de données

MÉTHODE Concevoir la base de données et écrire le script SQL de création

1. Traduire chaque classe en table : une classe devient une table, ses attributs deviennent des colonnes.
2. Choisir la clé primaire de chaque table : identifiant unique, souvent un entier `INT AUTO_INCREMENT` (ou `BIGINT`).
3. Traduire les associations : la multiplicité « plusieurs » (côté `*`) porte la **clé étrangère**. Une association `*..*` devient une table intermédiaire (ici `Pret`, déjà identifiée comme classe-association).
4. Choisir les types SQL adaptés : `INT`, `VARCHAR(n)`, `DATE`, `DATETIME`, `DECIMAL(10,2)`, `TEXT`.
5. Écrire le script `CREATE TABLE` avec les contraintes : `PRIMARY KEY`, `FOREIGN KEY ... REFERENCES`, `NOT NULL`, `UNIQUE`, `DEFAULT`, `ENGINE=InnoDB` (pour les FK).
6. Ordre de création : créer d'abord les tables référencées (sans clé étrangère sortante), puis les tables qui les référencent ; supprimer dans l'ordre inverse (`DROP TABLE`).

Réflexe Bac : une clé étrangère doit référencer une clé primaire existante et porter un type strictement identique (`INT` avec `INT`, même taille, même `UNSIGNED` le cas échéant). Préférer `utf8mb4` pour le jeu de caractères (supporte emojis, accents complets).

EXERCICE RÉSOLU 10 Création de la base `biblio_nkongsamba`

À partir du diagramme de classes précédent, écris le script SQL complet de création de la base et de ses trois tables, avec toutes les contraintes d'intégrité. Insère ensuite un abonné, un ouvrage et un prêt.

SOLUTION

Étape 1 : traduction. Les classes `Abonne` et `Ouvrage` deviennent des tables indépendantes. L'association `*..*` « un abonné emprunte plusieurs ouvrages » devient la table `Pret`, qui porte deux clés étrangères et ses propres attributs (dates).

Étape 2 : script SQL (ordre : `Abonne` et `Ouvrage` d'abord, puis `Pret`) :

```
-- Création de la base de données
CREATE DATABASE IF NOT EXISTS biblio_nkongsamba
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE biblio_nkongsamba;

-- Table des abonnés (créée en premier : elle est référencée)
CREATE TABLE Abonne (
  idAbonne INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(80) NOT NULL,
```

```

    classe VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    motDePasseHash VARCHAR(255) NOT NULL, -- pour l'authentification
    role ENUM('eleve', 'bibliothecaire') NOT NULL DEFAULT 'eleve',
    dateInscription DATE NOT NULL DEFAULT (CURRENT_DATE)
) ENGINE=InnoDB;

-- Table des ouvrages (créée en premier : elle est référencée)
CREATE TABLE Ouvrage (
    idOuvrage INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    titre VARCHAR(120) NOT NULL,
    auteur VARCHAR(80) NOT NULL,
    isbn VARCHAR(13) UNIQUE,
    nbExemplaires INT UNSIGNED NOT NULL DEFAULT 1,
    cote VARCHAR(30) -- ex. Dewey
) ENGINE=InnoDB;

-- Table des prêts : porte les deux clés étrangères et les dates
CREATE TABLE Pret (
    idPret INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    datePret DATE NOT NULL DEFAULT (CURRENT_DATE),
    dateRetourPrevue DATE NOT NULL,
    dateRetourEffective DATE NULL, -- NULL = pas encore rendu
    idAbonne INT UNSIGNED NOT NULL,
    idOuvrage INT UNSIGNED NOT NULL,
    FOREIGN KEY (idAbonne) REFERENCES Abonne(idAbonne) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (idOuvrage) REFERENCES Ouvrage(idOuvrage) ON DELETE RESTRICT ON UPDATE CASCADE,
    -- Index pour accélérer les recherches fréquentes
    INDEX idx_pret_abonne (idAbonne),
    INDEX idx_pret_ouvrage (idOuvrage),
    INDEX idx_pret_en_cours (dateRetourEffective)
) ENGINE=InnoDB;

```

Étape 3 : insertion des données de test.

```

-- Abonné bibliothécaire (mot de passe haché en PHP via password_hash)
INSERT INTO Abonne (nom, classe, email, motDePasseHash, role)
VALUES ('M. Kamga', 'Personnel', 'kamga@lycee.cm', '$2y$10$...hash...', 'bibliothecaire');

-- Abonné élève
INSERT INTO Abonne (nom, classe, email, motDePasseHash)
VALUES ('Ngo Bell Sandrine', 'Tle T1', 'sandrine@eleve.cm', '$2y$10$...hash...');

-- Ouvrages
INSERT INTO Ouvrage (titre, auteur, nbExemplaires, cote)
VALUES ('Une vie de boy', 'Ferdinand Oyono', 3, '848.9 OYO'),
      ('Le Vieux Nègre et la Médaille', 'Ferdinand Oyono', 2, '848.9 OYO');

-- Prêt : Sandrine (idAbonne=2) emprunte 'Une vie de boy' (idOuvrage=1)
INSERT INTO Pret (datePret, dateRetourPrevue, idAbonne, idOuvrage)
VALUES ('2025-03-10', '2025-03-24', 2, 1);

```

Vérification mentale : idAbonne = 2 et idOuvrage = 1 existent bien; dateRetourEffective est NULL tant que le livre n'est pas rendu (pas de NOT NULL); les FK sont en UNSIGNED comme les PK référencées.

Conclusion : la base respecte l'intégrité référentielle (ON DELETE RESTRICT empêche de supprimer un abonné ou un ouvrage s'il a des prêts en cours) ; les index préparent les performances pour les requêtes de consultation de prêts en cours.

9.4 Méthode 4 : Développer l'application web dynamique en PHP

MÉTHODE Réaliser une page PHP sécurisée connectée à MySQL (PDO)

1. Se connecter à la base avec PDO (pilote natif, orienté objet, multi-SGBD) : `new PDO('mysql:host=localhost;dbname=...;charset=utf8mb4', $user, $pass)`.
2. Activer le mode exception : `$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION)` pour déboguer proprement (stack trace).
3. Ne jamais concaténer une saisie utilisateur (`$_GET`, `$_POST`, `$_COOKIE`) dans une requête SQL : utiliser des **requêtes préparées** (`prepare()` + `execute(...)`) pour bloquer les injections SQL.
4. Protéger l'affichage HTML : passer toute donnée dynamique par `htmlspecialchars($var, ENT_QUOTES, 'UTF-8')` contre les attaques XSS (Cross-Site Scripting).
5. Hacher les mots de passe avec `password_hash($pwd, PASSWORD_DEFAULT)` (bcrypt/argon2) à l'inscription ; vérifier avec `password_verify($pwd, $hash)` à la connexion. Jamais de mot de passe en clair ni de MD5/SHA1.
6. Gérer les sessions : `session_start()` en tout début de script protégé ; stocker `$_SESSION['user_id']` et `$_SESSION['role']` ; régénérer l'ID à la connexion (`session_regenerate_id(true)`).
7. Structurer le code : un fichier `connexion.php` réutilisable (retourne `$pdo`), un fichier `functions.php` pour les requêtes métier, une page par fonctionnalité (MVC simplifié).

EXERCICE RÉSOLU 11 Recherche d'un ouvrage dans le catalogue (page publique)

Écris le fichier `catalogue.php` qui affiche un formulaire de recherche par titre, interroge la table `Ouvrage` de façon sécurisée et affiche les résultats dans un tableau HTML accessible. Explique pourquoi la requête préparée et l'échappement HTML sont indispensables.

SOLUTION

Étape 1 : le fichier de connexion `includes/connexion.php` (inclus par `require_once`) :

```

<?php
// includes/connexion.php : retourne une instance PDO configurée
$host = '127.0.0.1';
$dbname = 'biblio_nkongsamba';
$user = 'biblio_user'; // utilisateur dédié, pas root !
$pass = 'MotDePasseFort123'; // stocké hors webroot en production (variables d'env)

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
        $user, $pass,
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false, // vraies requêtes préparées côté serveur
        ]
    );
} catch (PDOException $e) {
    // En production : logger l'erreur, afficher page générique
    error_log('PDO Connexion : ' . $e->getMessage());
    die('Service temporairement indisponible. Merci de réessayer plus tard.');
```

Étape 2 : la page publique catalogue.php :

```

<?php
require_once __DIR__ . '/includes/connexion.php';

$terme = '';
$resultats = [];

if (isset($_GET['titre']) && strlen(trim($_GET['titre'])) > 0) {
    $terme = trim($_GET['titre']);

    // Requête préparée : le paramètre :recherche est une valeur, jamais du code SQL
    $sql = "SELECT idOuvrage, titre, auteur, nbExemplaires
            FROM Ouvrage
            WHERE titre LIKE :recherche OR auteur LIKE :recherche
            ORDER BY titre
            LIMIT 50";

    $stmt = $pdo->prepare($sql);
    // Le joker % est concaténé EN PHP, pas dans le SQL
    $param = "%$terme%";
    $stmt->execute(['recherche' => $param]);
    $resultats = $stmt->fetchAll();
}

?>
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Catalogue – Bibliothèque de Nkongsamba</title>
    <style>
        table { border-collapse : collapse; width : 100%; max-width : 800px; margin : 1rem auto; }
        th, td { border : 1px solid #ddd; padding : 0.5rem; text-align : left; }
```

```

th { background-color : #f4f4f4; }
form { max-width : 600px; margin : 1rem auto; display : flex; gap : 0.5rem; }
input[type="text"] { flex : 1; padding : 0.5rem; }
button { padding : 0.5rem 1rem; }
</style>
</head>
<body>
  <header><h1>Catalogue de la bibliothèque</h1></header>
  <main>
    <form method="get" action="catalogue.php" role="search">
      <label for="titre" class="visually-hidden">Rechercher par titre ou auteur</label>
      <input type="text" id="titre" name="titre" placeholder="Titre ou auteur..." value="<?=
htmlspecialchars($terme, ENT_QUOTES, 'UTF-8') ?>" required>
      <button type="submit">Rechercher</button>
    </form>

    <?php if ($terme !== '') : ?>
      <h2><?= count($resultats) ?> ouvrage(s) trouvé(s) pour « <?= htmlspecialchars($terme,
ENT_QUOTES, 'UTF-8') »</h2>
      <?php if (empty($resultats)) : ?>
        <p>Aucun résultat.</p>
      <?php else : ?>
        <table>
          <thead>
            <tr><th>Titre</th><th>Auteur</th><th>Exemplaires totaux</th></tr>
          </thead>
          <tbody>
            <?php foreach ($resultats as $livre) : ?>
              <tr>
                <td><?= htmlspecialchars($livre['titre'], ENT_QUOTES, 'UTF-8') ?></td>
                <td><?= htmlspecialchars($livre['auteur'], ENT_QUOTES, 'UTF-8') ?></td>
                <td><?= (int)$livre['nbExemplaires'] ?></td>
              </tr>
            <?php endforeach; ?>
          </tbody>
        </table>
      <?php endif; ?>
    </main>
  </body>
</html>

```

Étape 3 : pourquoi la requête préparée et htmlspecialchars sont indispensables.

- **Injection SQL** : si l'on écrivait "SELECT ... WHERE titre LIKE '%" . \$_GET['titre'] . "%'", un attaquant saisissant ' OR '1'='1 transformerait la requête en WHERE titre LIKE '' OR '1'='1%', retournant tout le catalogue, voire permettant une suppression (; DROP TABLE Ouvrage; --). Avec prepare()/execute(), la saisie est transmise au serveur MySQL comme une valeur littérale, jamais interprétée comme du SQL : l'injection devient impossible.
- **Faible XSS (Cross-Site Scripting)** : si un ouvrage a pour titre <script>alert('vol de session')</script> (injecté via une faille d'admin ou un import), l'affichage direct <?= \$livre['titre'] ?> exécuterait le script dans le navigateur de la victime (vol de cookie de session, keylogger). htmlspecialchars(..., ENT_QUOTES, 'UTF-8') convertit

<en <;, >en >;, " en ";, neutralisant le script.

Conclusion : la page est fonctionnelle (recherche insensible à la casse et accents grâce à `utf8mb4_unicode_ci`, pagination implicite `LIMIT 50`, responsive) et sécurisée selon les deux règles d'or : requêtes préparées en entrée, échappement contextuel en sortie.

9.5 Méthode 5 : Configurer le réseau local et publier l'application

MÉTHODE Héberger l'application sur le réseau local du lycée (LAN)

1. Installer la plateforme serveur : WampServer (Windows) ou XAMPP/LAMP sur le poste désigné « serveur ». Vérifier que les services Apache (port 80/443) et MySQL (port 3306) démarrent (icône verte WampServer).
2. Déployer les fichiers : copier le dossier du projet (ex. `biblio`) dans le `DocumentRoot` d'Apache (`C:\wamp64\www\biblio` ou `/var/www/html/biblio`). Importer le script SQL via phpMyAdmin (`http://localhost/phpmyadmin`) ou ligne de commande `mysql -u user -p < schema.sql`.
3. Attribuer une adresse IP statique au serveur (ex. `192.168.1.10/24`) via la configuration IPv4 de la carte réseau (Panneau de configuration → Centre réseau et partage → Modifier les paramètres de la carte). Cela évite que le DHCP ne change l'IP et casse les raccourcis des clients.
4. Valider le plan d'adressage : vérifier que tous les postes (clients + serveur) sont dans le même sous-réseau (même adresse réseau après ET logique $IP \wedge \text{Masque}$). Pas de routeur nécessaire pour un LAN unique.
5. Autoriser le trafic entrant :
 - Apache : fichier `httpd-vhosts.conf` ou `httpd.conf`, directive `Require all granted` (Apache 2.4) pour le `<Directory>` du projet.
 - Pare-feu Windows : créer une règle « Entrante » pour le port TCP 80 (et 443 si HTTPS) autorisant le profil « Domaine/Privé ».
 - Antivirus : vérifier qu'il ne bloque pas le port 80 (ex. Avast « Bouclier Web »).
6. Tester la connectivité : depuis un poste client, `ping 192.168.1.10` (ICMP), puis ouvrir le navigateur sur `http://192.168.1.10/biblio`. Vérifier l'affichage, la connexion BDD, les fonctionnalités.

Formules à retenir :

- Adresse réseau = Adresse IP ET Masque (bit à bit).
- Nombre d'hôtes adressables = $2^n - 2$ où n = nombre de bits à 0 dans le masque (partie hôte). On retire l'adresse réseau (tout 0) et l'adresse de diffusion (tout 1).

- Notation CIDR : $192.168.1.0/24 \equiv$ Masque $255.255.255.0$.

EXERCICE RÉSOLU 12 Plan d'adressage de la salle TI

La salle TI du lycée comprend 20 postes clients et 1 serveur. L'administrateur choisit le réseau $192.168.1.0$ avec le masque $255.255.255.0$ (/24).

1. Calcule l'adresse réseau du poste $192.168.1.35$ et vérifie qu'il peut joindre le serveur $192.168.1.10$.
2. Combien de machines ce sous-réseau peut-il adresser au maximum? Est-ce suffisant pour 21 machines + marge?
3. Donne l'adresse de diffusion (broadcast) du réseau.

SOLUTION

1. Adresse réseau du poste. On effectue le ET logique bit à bit entre l'IP et le masque. Comme les 3 premiers octets du masque sont 255 (11111111), ils recopient l'IP. Seul le dernier octet change :

IP poste : $192.168.1.35$ ($35_{10} = 00100011_2$)
 Masque : $255.255.255.0$ ($0_{10} = 00000000_2$)
 ET logique : $192.168.1.0$ (adresse réseau)

Même calcul pour le serveur $192.168.1.10$ ($10 = 00001010_2$) $\rightarrow 192.168.1.0$. Les deux machines sont dans le même sous-réseau : elles communiquent directement via le switch (couche 2, trames Ethernet), sans passer par une passerelle (routeur). Le client pourra donc ouvrir <http://192.168.1.10/biblio>.

2. Nombre maximal de machines. Masque /24 $\Rightarrow$ partie hôte $n = 32 - 24 = 8$ bits.

Hôtes utilisables = $2^8 - 2 = 256 - 2 = 254$ adresses.

Besoin : 20 clients + 1 serveur = 21 adresses. Comme $21 \ll 254$, le plan est largement suffisant (marge de 233 adresses pour imprimantes, bornes WiFi, futurs postes).

3. Adresse de diffusion (broadcast). Tous les bits de la partie hôte à 1 : $192.168.1.255$ ($255_{10} = 11111111_2$). Utilisée par ARP/DHCP pour parler à tous les hôtes du LAN.

Conclusion : le réseau $192.168.1.0/24$ convient parfaitement. Affectation suggérée : Serveur .10 (statique), Clients .11 à .30 (DHCP ou statiques), Passerelle Internet .1 (box/-routeur), Broadcast .255. L'application est publiée à l'URL <http://192.168.1.10/biblio>.

9.6 Méthode 6 : Tester, valider et documenter l'application

MÉTHODE Conduire les tests et rédiger le rapport de réalisation + guide d'utilisation**A. Campagne de tests structurée (niveaux V) :**

1. Tests unitaires : chaque fonction/classe isolée (ex. `Connexion::getPDO()`, `Password::hash()`, `Ouvrage::estDisponible()`). Automatisables (PHPUnit).
2. Tests d'intégration : interactions entre modules (Formulaire HTML → Contrôleur PHP → Modèle/PDO → MySQL → Vue HTML). Vérifier les contrats d'interface.
3. Tests fonctionnels (boîte noire) : chaque besoin du cahier des charges (BF1..BF3), avec un **jeu d'essai** couvrant :
 - Cas nominaux (chemin heureux).
 - Cas limites / erreurs prévisibles (champ vide, doublon, format invalide, stock épuisé, droits insuffisants).
 - Cas de sécurité (tentative injection SQL, XSS, contournement auth, IDOR).
4. Tests de non-régression : relancer les tests passés après chaque correction.
5. Recette utilisateur (UAT) : le client (bibliothécaire) exécute les scénarios réels sur l'environnement de pré-production (ou production) et signe le Procès-Verbal de Recette (PV) : « Conforme » / « Réserves » / « Non conforme ».

Consigner chaque test dans un tableau : N°, Fonctionnalité (réf. BF), Données d'entrée, Résultat attendu, Résultat obtenu, Verdict (OK/KO), Commentaire/Anomalie.

B. Rapport de réalisation (structure type pour le Bac / soutenance) :

1. Page de garde (Titre, Nom/Prénom, Série TI, Établissement, Année, Tuteur).
2. Remerciements, Résumé (1/2 page), Sommaire.
3. Introduction : Contexte réel (lycée de Nkongsamba), Problématique, Objectifs, Périmètre.
4. Chapitre 1 — Analyse : Cahier des charges complet (BF, BNF, Contraintes, Planning Gantt, Acteurs).
5. Chapitre 2 — Conception : Diagrammes UML (Cas d'utilisation, Classes, Séquence pour « Emprunter »), MCD/MLD (Merise), Dictionnaire de données, Choix techniques (PHP 8.2, MySQL 8, Bootstrap 5, PDO, Architecture MVC simplifié).
6. Chapitre 3 — Réalisation : Arborescence du projet, Extraits de code commentés (connexion, authentification, CRUD, requêtes préparées, hachage), Captures d'écran annotées (pages principales, phpMyAdmin, structure BDD).
7. Chapitre 4 — Tests et Validation : Tableaux de tests (unitaires, intégration, fonctionnels), Anomalies trouvées → Corrections apportées, PV de recette signé.
8. Conclusion : Bilan compétences (Savoirs, Savoir-faire, Savoir-être), Difficultés rencontrées → Solutions, Perspectives (API REST, PWA, codes-barres, déploiement cloud).

9. Bibliographie / Webographie (docs PHP, MySQL, UML, RFC, OWASP Top 10).

10. Annexes : Script SQL complet, Guide d'utilisation, Manuel d'installation serveur.

C. Guide d'utilisation (mode d'emploi utilisateur final, sans jargon dev) :

- Accès : URL locale, identifiants par défaut (à changer), navigateurs supportés.
- Description écran par écran (copies d'écran annotées) : Connexion, Tableau de bord, Catalogue (recherche), Gestion ouvrages (ajout/modif/suppr), Prêts/Retours.
- Scénarios pas à pas numérotés : « Comment enregistrer un prêt », « Comment prolonger un prêt », « Comment ajouter un ouvrage ».
- Messages d'erreur fréquents et actions correctives (ex. « Ouvrage indisponible » → vérifier stock / retours en attente).
- Contacts support (email, téléphone, procédure ticket).

EXERCICE RÉSOLU 13 Jeu d'essai fonctionnel et plan de rapport

Pour la fonctionnalité « Enregistrer un prêt » (BF2) de l'application de bibliothèque :

1. Propose un tableau de 5 tests fonctionnels couvrant un cas normal et quatre cas d'erreur/limite.
2. Rédige le plan détaillé du rapport de réalisation que tu remettras au jury.

SOLUTION

1. Tableau des tests fonctionnels (extrait).

N°	Données d'entrée (Scénario)	Résultat attendu	Résultat obtenu	Verdict
T1	Abonné valide (id=2), Ouvrage dispo (id=1, 3 ex.), dates valides	Prêt inséré en BDD, message « Prêt enregistré », stock dispo décrémenté à l'affichage	Conforme	OK
T2	Abonné inexistant (id=999)	Rejet, message « Abonné inconnu », aucun INSERT	Message affiché, pas d'erreur SQL	OK
T3	Date de retour prévue vide	Rejet côté client (HTML5 required) + côté serveur (validation PHP), message « Date obligatoire »	Message affiché	OK
T4	Ouvrage avec 0 exemplaire dispo (tous prêts non rendus)	Rejet, message « Ouvrage indisponible (stock épuisé) », aucun INSERT	Prêt enregistré malgré tout	KO
T5	Tentative injection SQL dans champ recherche (ex. ' OR '1'='1)	Traitée comme chaîne littérale, 0 résultat ou erreur syntaxe gérée, pas d'exécution code	Requête préparée neutralise l'attaque	OK

Analyse T4 (KO) : le script PHP omet la vérification du stock disponible avant INSERT. Correction : ajouter dans `enregistrer_pret.php` une transaction :

```
$pdo->beginTransaction();
try {
    // 1. Verrouiller la ligne ouvrage pour éviter concurrence (SELECT ... FOR UPDATE)
    $stmt = $pdo->prepare("SELECT nbExemplaires FROM Ouvrage WHERE idOuvrage = ? FOR UPDATE");
    $stmt->execute([$idOuvrage]);
    $ouv = $stmt->fetch();
    // 2. Compter prêts en cours non rendus
    $stmt = $pdo->prepare("SELECT COUNT(*) FROM Pret WHERE idOuvrage = ? AND dateRetourEffective IS NULL");
    $stmt->execute([$idOuvrage]);
    $nbPretsEnCours = (int)$stmt->fetchColumn();
    // 3. Vérifier dispo
    if ($nbPretsEnCours >= $ouv['nbExemplaires']) {
        throw new Exception("Ouvrage indisponible (tous les exemplaires sont en prêt).");
    }
    // 4. Insérer le prêt
    $stmt = $pdo->prepare("INSERT INTO Pret (datePret, dateRetourPrevue, idAbonne, idOuvrage)
VALUES (CURDATE(), ?, ?, ?)");
    $stmt->execute([$dateRetourPrevue, $idAbonne, $idOuvrage]);
    $pdo->commit();
    $_SESSION['flash'] = 'Prêt enregistré avec succès.';
}
```

```
} catch (Exception $e) {  
    $pdo->rollBack();  
    $_SESSION['error'] = $e->getMessage();  
}  
header('Location : prets.php'); exit;
```

On relance T4 → OK. C'est le principe du **test de non-régression** : toute correction doit être re-testée.

2. Plan détaillé du rapport de réalisation (remis au jury).

1. Page de garde : Titre « Gestion de bibliothèque – Lycée de Nkongsamba », Auteur, Tle TI, Session 2025.
2. Remerciements & Résumé (150 mots : contexte, stack technique, résultat).
3. Introduction : Problématique (registre papier), Objectifs (BF1–BF3), Périmètre (intranet lycée), Acteurs.
4. Chapitre 1 — Analyse des besoins : Cahier des charges complet (tableaux BF/BNF), Planning Gantt 8 semaines, Analyse des risques (perte données, indisponibilité serveur).
5. Chapitre 2 — Conception :
 - Diagramme de cas d'utilisation (acteurs, UC, include/extend).
 - Diagramme de classes (attributs, méthodes, multiplicités, classes-association).
 - Diagramme de séquence « Enregistrer un prêt » (alt : stock épuisé).
 - MCD (Entités/Associations) → MLD (Tables, PK/FK, Notations flèches).
 - Dictionnaire de données (tables, colonnes, types, contraintes).
6. Chapitre 3 — Réalisation technique :
 - Environnement : WampServer 3.3 (Apache 2.4, PHP 8.2, MySQL 8.0), VS Code, Git.
 - Architecture : MVC simplifié (/public, /src/Controller, /src/Model, /templates, /assets).
 - Sécurité : PDO + requêtes préparées, password_hash/verify, htmlspecialchars, session_regenerate_id, CSRF tokens (formulaires POST), headers Content-Security-Policy.
 - Extraits commentés : Connexion.php, AuthController::login(), OuvrageModel::search(), PretController::create() (avec transaction).
 - Captures : Page d'accueil, Catalogue (résultats), Espace bibliothécaire (CRUD), phpMyAdmin (structure + données).
7. Chapitre 4 — Tests, Validation, Déploiement :
 - Tableaux de tests (Unitaires, Intégration, Fonctionnels) avec verdicts.

- Fiche d'anomalies (T4 KO → Correction → Retest OK).
 - Procès-Verbal de Recette signé par le bibliothécaire (M. Kamga) : « Application conforme aux besoins BF1-BF3, ergonomie satisfaisante, réserve : ajouter export CSV ».
 - Déploiement LAN : IP serveur 192.168.1.10, pare-feu, test ping + navigation poste client.
8. Conclusion : Compétences validées (Prog, BD, Réseau, Projet), Apports personnels (rigueur sécurité, UML), Perspectives (PWA, scan code-barres, API Mobile Money pour amendes).
 9. Bibliographie : PHP.net, MySQL.dev, UML 2.5 (OMG), OWASP Cheat Sheets, Programme MINESEC TI.
 10. Annexes : Script SQL complet (`schema.sql`), Guide d'utilisation (4 pages), Manuel install serveur.

Conclusion pédagogique : un test qui échoue (T4) n'est pas un échec mais une réussite du processus de qualité : il révèle un bug critique (survente de stock) avant la mise en production. Le rapport qui retrace honnêtement cette découverte, la correction et le re-test est très valorisé par le jury : il prouve la maîtrise de la démarche d'ingénierie.

ATTENTION Les 5 erreurs qui coûtent des points au Bac (et en projet réel)

1. Confondre besoin fonctionnel et non fonctionnel. « Le système doit être rapide » n'est pas une fonctionnalité : c'est une contrainte de qualité (BNF). Un besoin fonctionnel (BF) s'énonce par un verbe d'action métier sur les données : « Enregistrer un prêt », « Imprimer un bulletin », « Authentifier un utilisateur ». Astuce : si ça se teste par un scénario utilisateur → BF ; si ça se mesure par un indicateur technique → BNF.
2. Inverser les multiplicités UML. La multiplicité se place à l'extrémité opposée de celle qu'elle décrit : côté Pret, on écrit le nombre de prêts qu'un Abonne peut avoir (0..*). Côté Abonne, on écrit le nombre d'abonnés pour un prêt (1). Relis toujours l'association dans les deux sens : « Un Abonné effectue 0..* Prets ; Un Pret concerne 1 Abonné ».
3. Créer les tables dans le mauvais ordre (clé étrangère). Un `CREATE TABLE Pret` avant `Abonne` ou `Ouvrage` échoue (ERROR 1215 : Cannot add foreign key constraint). Règle : Tables référencées (parents) d'abord ; tables référençantes (enfants) après. Pour `DROP TABLE` : ordre inverse (enfants d'abord, ou `SET FOREIGN_KEY_CHECKS=0`).
4. Oublier le -2 dans le calcul d'hôtes (IPv4). Le nombre de machines adressables est $2^n - 2$, pas 2^n . On retire l'adresse réseau (identifiant du sous-réseau) et l'adresse de diffusion (broadcast). Pour un /24 : 254 hôtes utilisables, pas 256. Pour un /26 ($n = 6$) : 62 hôtes, pas 64.

5. Concaténer les saisies dans le SQL et stocker les mots de passe en clair. Toute requête construite par concaténation de `$_GET`, `$_POST`, `$_COOKIE` est une faille d'injection SQL critique (OWASP Top 10 A03). Exigence Bac : requêtes préparées (`prepare/execute`) systématiques. Mots de passe : jamais en clair, toujours `password_hash()` (`bcrypt/argon2`) + `password_verify()`. Au Bac, mentionner explicitement ces protections dans le rapport et le code est un critère d'excellence, pas une option.

10 EXERCICES

10.1 Je vérifie mes connaissances

EXERCICE 6 QCM : fondamentaux du projet informatique

Pour chaque question, une seule réponse est correcte. Recopiez le numéro et la lettre choisie.

1. Le document qui formalise les besoins du client, les contraintes techniques et les délais avant tout développement s'appelle :
 - A) Le guide d'utilisation.
 - B) Le cahier des charges.
 - C) Le rapport de recette.
 - D) Le diagramme de Gantt.
2. Dans un diagramme de cas d'utilisation UML, un acteur représente :
 - A) Un objet de la base de données.
 - B) Une fonctionnalité du système.
 - C) Un rôle joué par un utilisateur ou un système externe interagissant avec l'application.
 - D) Une table de la base de données.
3. Pour héberger un site web dynamique (PHP/MySQL) sur un serveur d'un réseau local d'entreprise, le serveur doit obligatoirement disposer de :
 - A) Un navigateur web uniquement.
 - B) Un serveur web (Apache/Nginx), un interpréteur PHP et un SGBD (MySQL/MariaDB).
 - C) Une adresse IP publique fixe uniquement.
 - D) Un pare-feu matériel Cisco uniquement.
4. La phase de recette (ou validation) consiste à :
 - A) Écrire le code source de l'application.
 - B) Vérifier que l'application répond aux exigences du cahier des charges en exécutant des scénarios de test.
 - C) Installer le serveur physique dans l'armoire de brassage.
 - D) Concevoir le MLD à partir du MCD.

EXERCICE 7 Vrai ou Faux ? Justifiez

Indiquez si chaque affirmation est vraie (V) ou fausse (F) et justifiez brièvement votre réponse.

1. Le diagramme de Gantt est l'outil unique pour représenter la structure de la base de données (tables, clés primaires, clés étrangères).
2. Le passage du MCD (Modèle Conceptuel de Données) au MLD (Modèle Logique de Données) transforme les entités en tables et les associations en clés étrangères.
3. Dans un réseau local en adressage IPv4 privé (classe C par défaut), le masque de sous-réseau 255.255.255.0 (/24) permet d'adresser jusqu'à 254 machines utilisables.
4. Le guide d'utilisation s'adresse aux développeurs pour la maintenance future du code source, tandis que le rapport de réalisation s'adresse aux utilisateurs finaux.

EXERCICE 8 Définitions et vocabulaire technique

Donnez une définition précise et professionnelle pour chacun des termes suivants, en lien avec la conduite de projet informatique :

1. Cahier des charges (CDC).
2. Diagramme de Gantt (chemin critique, jalons).
3. Recette (ou validation) utilisateur.
4. Guide d'utilisation (manuel utilisateur).
5. Mise en production (déploiement).

EXERCICE 9 Calcul d'adressage et planification

1. Un petit réseau local doit connecter 5 postes clients, 1 serveur web/base de données et 1 imprimante réseau (soit 7 équipements au total). On utilise l'adresse de réseau privé 192.168.10.0. Déterminez le masque de sous-réseau CIDR le plus adapté (économique) et donnez la première et la dernière adresse IP utilisables pour les hôtes.
2. Un projet est planifié sur 10 jours ouvrés. La tâche A (Conception BDD) dure 2 jours. La tâche B (Développement Backend) dure 4 jours et ne peut commencer qu'après A. La tâche C (Développement Frontend) dure 3 jours et peut commencer en parallèle de B après A. La tâche D (Tests) dure 2 jours et ne peut commencer qu'après B et C. Calculez la durée totale du chemin critique et la date de fin prévisionnelle si le projet démarre le lundi 3 mars (hors week-ends).

10.2 Je m'entraîne

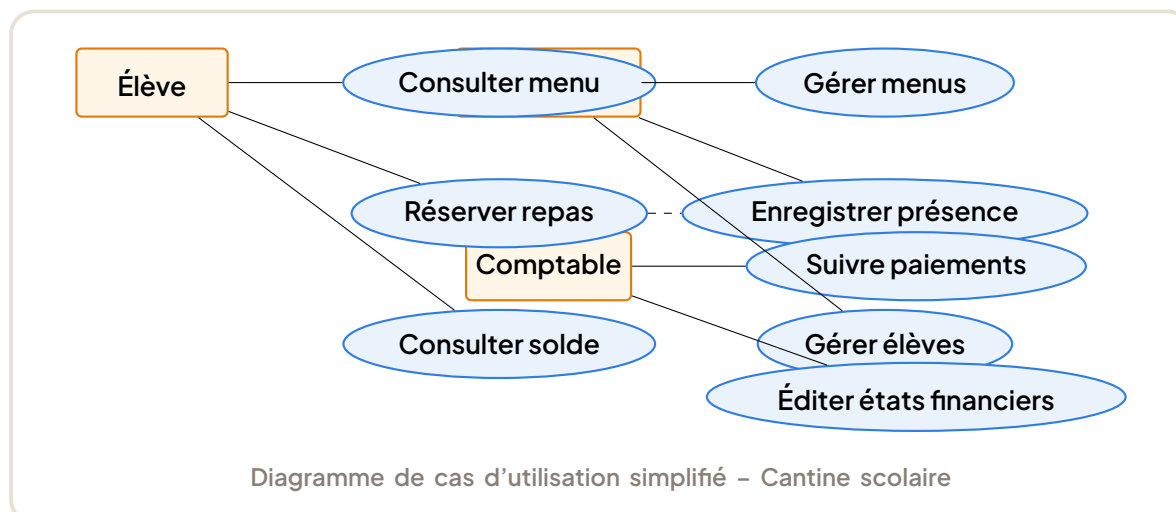
EXERCICE 10 Cahier des charges et planification : Bibliothèque scolaire

Le Lycée Bilingue de Douala-Bassa souhaite informatiser la gestion de sa bibliothèque (prêts, retours, catalogue, adhérents). Vous êtes chef de projet.

1. Rédigez la rubrique Objectifs du projet (3 à 4 points) et la rubrique Fonctionnalités principales (au moins 6 fonctionnalités numérotées, distinguant acteur « Bibliothécaire » et acteur « Élève »).
2. Proposez un diagramme de Gantt simplifié (texte ou tableau) pour 6 tâches principales : Étude de l'existant, Conception BDD (MCD/MLD), Développement Backend (PHP/PDO), Développement Frontend (HTML/CSS/JS), Tests unitaires et d'intégration, Recette utilisateur et déploiement. Indiquez pour chaque tâche : durée (en jours), dépendances, et période (ex : J1-J2). La durée totale ne doit pas dépasser 15 jours ouvrés.

EXERCICE 11 De l'UML au SQL : Application de cantine scolaire

On souhaite développer une application de gestion de cantine pour le Collège de la Salle à Yaoundé. Le diagramme de cas d'utilisation (fourni ci-après) montre les interactions.



1. Identifiez les acteurs et listez les cas d'utilisation associés à chacun.
2. Complétez le diagramme de classes UML suivant (donné incomplet) en ajoutant les attributs, types, multiplicités et les associations manquantes pour couvrir les besoins : classes Eleve, Menu, Reservation, Paiement. Précisez les clés primaires (PK) et étrangères (FK).
3. En déduire le MLD (Modèle Logique de Données) sous forme textuelle : NomTable (PK, FK, attributs...).

4. Écrire le script SQL (MySQL/MariaDB) de création des tables (CREATE TABLE) avec types adaptés (INT, VARCHAR, DATE, DECIMAL, ENUM), contraintes PRIMARY KEY, FOREIGN KEY, NOT NULL, DEFAULT. Inclure le moteur InnoDB.

EXERCICE 12 Réseau local et publication : Site PHP/MySQL sur serveur interne

L'association « Jeunes Codeurs Cameroun » a développé un site de e-learning (PHP 8.2, MySQL 8.0). Elle dispose d'une salle informatique avec 10 postes clients et 1 serveur sous Ubuntu Server 22.04 LTS, reliés par un switch 1 Gbit/s. Le réseau utilise l'adressage 192.168.20.0/24. Le serveur a l'IP 192.168.20.10.

1. Complétez le plan d'adressage suivant pour 5 postes clients (PC1 à PC5) : Adresse IP, Masque, Passerelle (Gateway), DNS (utiliser le serveur comme DNS local ou 8.8.8.8).

Poste	Adresse IP	Masque	Passerelle	DNS
Serveur	192.168.20.10	255.255.255.0	–	8.8.8.8
PC1				
PC2				
PC3				
PC4				
PC5				

2. Décrivez précisément les étapes de publication du site sur ce serveur (installation stack LAMP, configuration VirtualHost Apache, déploiement code via Git ou SCP, import BDD, droits fichiers www-data, test local curl http://localhost, test depuis un poste client).
3. Quelle commande `ufw` (pare-feu Ubuntu) autoriser l'accès au port HTTP (80) depuis le réseau local uniquement ?

EXERCICE 13 Plan de tests : Calcul de moyenne pondérée

On développe une fonction PHP `calculerMoyenne(array $notes, array $coeffs)` qui renvoie la moyenne pondérée arrondie à 2 décimales, ou `false` si erreur (tableaux vides, tailles différentes, coefficient nul ou négatif, note hors intervalle [0, 20]). Concevez un plan de tests sous forme de tableau contenant 6 cas de test couvrant : cas nominal, valeurs limites (notes 0 et 20), coefficients variés, données erronées (note > 20, coefficient = 0, tableaux tailles différentes), tableaux vides. Colonnes : #Cas, Description, Données entrées (\$notes, \$coeffs), Résultat attendu, Type de test (Nominal / Limite / Erreur / Robustesse).

10.3 Je me prépare au Bac

EXERCICE 14 Sujet type Bac – Rapport de réalisation et livrables

Contexte : L'équipe « DevTI » a réalisé le projet « GestionCyber » : application web de gestion de cybercafé (comptes clients, facturation temps, réservation postes, stats journalières). Le projet s'est déroulé sur 6 semaines.

1. Voici les rubriques d'un rapport de réalisation mélangées. Remettez-les dans l'ordre logique standard :
 - Conclusion et perspectives
 - Introduction (contexte, problématique, objectifs)
 - Cahier des charges
 - Conception (UML : cas d'utilisation, classes, séquence ; MCD/MLD)
 - Réalisation (arborescence, extraits de code clés, captures d'écran IHM)
 - Plan de tests et résultats de recette
 - Déploiement et guide d'utilisation
 - Planification (Gantt, répartition des rôles)
2. Rédigez l'Introduction du rapport pour le projet « GestionCyber » en intégrant obligatoirement : le contexte (cybercafé « Le Net Plus » à Bafoussam, gestion papier actuelle), la problématique (pertes financières, erreurs manuelles, attente clients), les objectifs (automatisation, traçabilité, rapport journalier automatique) et la solution retenue (application web PHP/MySQL hébergée en local, stack WampServer).

EXERCICE 15 Sujet type Bac – Projet intégrateur complet : Billetterie « Fête de la Musique Yaoundé »

L'agence « EventCM » commande une application de billetterie en ligne pour la Fête de la Musique (21 juin) à l'esplanade du Palais des Congrès. Besoins : inscription participants (nom, email, téléphone, type billet : VIP/Standard/Gratuit), paiement simulé (Mobile Money MTN/Orange), génération QR code billet, tableau de bord admin (stats ventes par type, liste participants).

1. Analyse : Identifiez les acteurs principaux. Dessinez (schéma textuel ou description précise) le diagramme de cas d'utilisation complet (au moins 6 cas d'utilisation).
2. Conception BDD : Proposez le MCD (entités : Participant, Billet, Paiement, TypeBillet; associations avec cardinalités). Donnez le MLD correspondant. Écrivez le script CREATE TABLE pour Participant et Billet (clé étrangère vers TypeBillet et Participant).

3. Développement Backend (PHP/PDO) : Écrivez le code complet du fichier `inscription.php` qui reçoit les données POST (validation basique, insertion BDD transactionnelle Participant puis Billet, génération référence unique TKT-XXXXXX, réponse JSON). Gérez les erreurs (PDOException, doublon email).
4. Réseau & Déploiement : Le serveur de prod est une VM Ubuntu (10.0.5.50/24) chez CAMTEL. Le pare-feu `ufw` est actif. Donnez les 3 commandes `ufw` pour autoriser HTTP/HTTPS depuis Internet et SSH depuis l'IP admin 192.168.1.100 seulement. Citez 2 directives Apache `VirtualHost` essentielles pour la sécurité (ex : `ServerName`, `DocumentRoot`, désactivation `.htaccess` via `AllowOverride None`).
5. Tests : Proposez 3 cas de test pour la fonction d'inscription (nominal, email invalide, type billet inexistant). Précisez données entrées et résultat attendu.

EXERCICE 16 Sujet type Bac – Synthèse : Articulation des compétences par phase

Un projet informatique suit les phases : 1. Lancement, 2. Conception, 3. Réalisation, 4. Tests/Recette, 5. Déploiement.

1. Pour chaque phase, citez au moins 2 livrables principaux (documents, code, configs, scripts).
2. Montrez comment les compétences des trois modules du programme s'articulent sur le livrable commun « Application web déployée et fonctionnelle » :
 - Module Programmation : quel apport concret ? (ex : architecture MVC, code PHP/JS, API REST).
 - Module Systèmes d'information / Bases de données : quel apport concret ? (ex : MCD/MLD, script SQL, optimisation indexes, transactions, vues, triggers).
 - Module Réseaux / Internet / Sécurité : quel apport concret ? (ex : config IP/VM, DNS, SSL/TLS, pare-feu, VPN, sauvegarde distante, hardening serveur).
3. Donnez un exemple concret de faille de sécurité (OWASP Top 10) qui pourrait survenir si le module Réseau/Sécurité est négligé lors de la réalisation, et la parade correspondante.

11 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : fondamentaux du projet informatique

1. B) Le cahier des charges. C'est le document contractuel rédigé en phase de lancement qui formalise les besoins du client, les contraintes (techniques, budgétaires, délais) et les critères d'acceptation. Le guide d'utilisation (A) est un livrable de fin de projet ; le rapport de recette (C) atteste de la validation ; le diagramme de Gantt (D) est un outil de planification, pas de spécification.
2. C) Un acteur est un rôle joué par une entité externe au système (utilisateur humain ou autre système, par exemple l'API MTN Mobile Money) qui interagit avec l'application. Une fonctionnalité (B) est un cas d'utilisation, pas un acteur.
3. B) Un site dynamique PHP/MySQL exige la pile serveur complète : serveur HTTP (Apache ou Nginx), interpréteur PHP et SGBD (MySQL/MariaDB) — c'est la stack LAMP/WAMP. Un navigateur (A) est côté client ; une IP publique (C) est inutile en réseau local ; un pare-feu matériel (D) n'est pas obligatoire.
4. B) La recette consiste à exécuter des scénarios de test (cahier de recette) pour vérifier la conformité de l'application aux exigences du cahier des charges, avant acceptation par le client.

CORRIGÉ Exercice 2 — Vrai ou Faux ? Justifiez

1. Faux. Le diagramme de Gantt est un outil de **planification temporelle** (tâches, durées, dépendances). La structure de la base de données se représente par le **MCD** (entités/associations) puis le **MLD** (tables, clés primaires et étrangères).
2. Vrai. Règle de dérivation MCD → MLD : toute entité devient une table dont l'identifiant devient la clé primaire ; une association binaire de type $(1,n)-(1,1)$ se traduit par une clé étrangère migrée côté $(1,n)$; une association (n,n) devient une table supplémentaire dont la clé primaire est composée des deux clés étrangères.
3. Vrai. Avec un masque /24, il reste $32 - 24 = 8$ bits pour la partie hôte, soit $2^8 = 256$ adresses. On retire l'adresse réseau et l'adresse de broadcast : $256 - 2 = 254$ hôtes adressables.
4. Faux. C'est l'inverse : le **guide d'utilisation** s'adresse aux utilisateurs finaux (procédures, captures d'écran, FAQ), tandis que le **rapport de réalisation** documente la démarche technique (conception, code, tests) pour le jury, le client et les développeurs chargés de la maintenance.

CORRIGÉ Exercice 3 — Définitions et vocabulaire technique

1. Cahier des charges (CDC) : document contractuel rédigé en phase de lancement, qui spécifie les besoins du client, les objectifs, les fonctionnalités attendues, les contraintes techniques et matérielles, le budget, les délais et les critères d'acceptation. Il sert de référence tout au long du projet et lors de la recette.
2. Diagramme de Gantt : outil de planification représentant les tâches du projet sous forme de barres horizontales sur un axe temporel, avec leurs durées et dépendances. Le **chemin critique** est la plus longue chaîne de tâches dépendantes : tout retard sur elle retarde le projet entier. Les **jalons** (milestones) sont des événements marquants de durée nulle (ex. « BDD validée », « recette signée ») qui jalonnent le suivi.
3. Recette (validation) utilisateur : phase où le client (ou ses représentants) exécute un cahier de recette — ensemble de scénarios de test — pour vérifier que l'application livrée est conforme au cahier des charges. Elle aboutit à un procès-verbal de recette (acceptation, acceptation avec réserves, ou refus).
4. Guide d'utilisation : document destiné aux utilisateurs finaux décrivant l'installation éventuelle, la prise en main, les procédures pas à pas (avec captures d'écran), les messages d'erreur courants et une FAQ. Il ne contient pas de détails techniques de code.
5. Mise en production (déploiement) : opération consistant à installer l'application validée sur l'environnement réel d'exploitation (serveur de production), à configurer les services (serveur web, SGBD, pare-feu, DNS), à migrer les données et à ouvrir l'accès aux utilisateurs, avec un plan de sauvegarde et éventuellement de retour arrière.

CORRIGÉ Exercice 4 — Calcul d'adressage et planification

1. Adressage. Il faut 7 adresses d'hôtes. On cherche le plus petit n (bits hôte) tel que $2^n - 2 \geq 7$:

$$n = 3 \Rightarrow 2^3 - 2 = 6 < 7 \quad (\text{insuffisant})$$

$$n = 4 \Rightarrow 2^4 - 2 = 14 \geq 7 \quad (\text{convient})$$

Le préfixe est donc $32 - 4 = 28$: masque /28, soit 255.255.255.240. Le bloc fait 16 adresses :

- Adresse réseau : 192.168.10.0
- Première adresse utilisable : 192.168.10.1
- Dernière adresse utilisable : 192.168.10.14
- Broadcast : 192.168.10.15

(On pouvait aussi accepter /24 par simplicité, mais /28 est la réponse économique demandée.)

2. Chemin critique. Graphe de dépendances : $A \rightarrow B$, $A \rightarrow C$, $(B \text{ et } C) \rightarrow D$.

- Chemin 1 : $A \rightarrow B \rightarrow D = 2 + 4 + 2 = 8$ jours.
- Chemin 2 : $A \rightarrow C \rightarrow D = 2 + 3 + 2 = 7$ jours.

Le chemin critique est A-B-D, durée totale 8 jours ouvrés (C dispose d'une marge de 1 jour).

Calendrier (hors week-ends), démarrage lundi 3 mars :

Tâche	Durée	Période
A Conception BDD	2 j	lun. 3 – mar. 4 mars
B Backend	4 j	mer. 5 – lun. 10 mars (week-end 8-9)
C Frontend	3 j	mer. 5 – ven. 7 mars
D Tests	2 j	mar. 11 – mer. 12 mars

Fin prévisionnelle : mercredi 12 mars (8 jours ouvrés : 3, 4, 5, 6, 7, 10, 11, 12 mars).

CORRIGÉ Exercice 5 — Cahier des charges et planification : Bibliothèque scolaire

1. Extraits du cahier des charges.

Objectifs du projet :

- Informatiser le catalogue de la bibliothèque (ouvrages, auteurs, rayons) pour supprimer les fiches papier.
- Automatiser la gestion des prêts et retours avec suivi des échéances et des retards.
- Gérer le fichier des adhérents (élèves et personnel) et l'historique de leurs emprunts.
- Réduire le temps de recherche d'un ouvrage à moins de 30 secondes et fiabiliser les statistiques de fréquentation.

Fonctionnalités principales :

1. (Bibliothécaire) Enregistrer, modifier et supprimer un ouvrage du catalogue.
 2. (Bibliothécaire) Inscrire et gérer les adhérents.
 3. (Bibliothécaire) Enregistrer un prêt (avec date de retour prévue) et un retour.
 4. (Bibliothécaire) Éditer la liste des retards et les statistiques mensuelles.
 5. (Élève) Rechercher un ouvrage par titre, auteur ou mot-clé et consulter sa disponibilité.
 6. (Élève) Consulter ses emprunts en cours et leurs dates limites.
2. Diagramme de Gantt simplifié (15 jours ouvrés) :

Tâche	Durée	Dépendances	Période
Étude de l'existant	2 j	–	J1–J2
Conception BDD (MCD/MLD)	2 j	Étude	J3–J4
Développement Backend (PHP/PDO)	5 j	Conception	J5–J9
Développement Frontend (HTML/CSS/JS)	4 j	Conception	J5–J8
Tests unitaires et d'intégration	3 j	Backend et Frontend	J10–J12
Recette utilisateur et déploiement	3 j	Tests	J13–J15

Chemin critique : Étude → Conception → Backend → Tests → Recette = 2 + 2 + 5 + 3 + 3 = 15 jours. Le Frontend (J5–J8) a 1 jour de marge.

Points de vigilance : distinguer clairement les fonctionnalités par acteur ; vérifier que les dépendances sont respectées (les tests ne peuvent précéder les développements) ; ne pas dépasser 15 jours.

CORRIGÉ**Exercice 6 — De l'UML au SQL : Application de cantine scolaire****1. Acteurs et cas d'utilisation :**

- Élève : Consulter menu, Réserver repas, Consulter solde.
- Gestionnaire : Gérer menus, Enregistrer présence, Gérer élèves.
- Comptable : Suivre paiements, Éditer états financiers.

2. Diagramme de classes complété :

- Eleve : idEleve : INT (PK), nom : VARCHAR, prenom : VARCHAR, classe : VARCHAR, solde : DECIMAL.
- Menu : idMenu : INT (PK), dateMenu : DATE, description : VARCHAR, prix : DECIMAL.
- Reservation : idReservation : INT (PK), dateReservation : DATETIME, statut : ENUM('confirmée', 'annulée', 'consommée'), idEleve (FK), idMenu (FK).
- Paiement : idPaiement : INT (PK), montant : DECIMAL, datePaiement : DATETIME, mode : ENUM('espèces', 'mtn_momo', 'orange_money'), idEleve (FK).

Associations et multiplicités : Eleve (1,1) → (0,n) Reservation ; Menu (1,1) → (0,n) Reservation ; Eleve (1,1) → (0,n) Paiement.

3. MLD textuel :

```
Eleve(idEleve, nom, prenom, classe, solde)
Menu(idMenu, dateMenu, description, prix)
Reservation(idReservation, dateReservation, statut, #idEleve, #idMenu)
Paiement(idPaiement, montant, datePaiement, mode, #idEleve)
```

4. Script SQL :

```
CREATE TABLE Eleve (  
  idEleve INT AUTO_INCREMENT PRIMARY KEY,  
  nom VARCHAR(50) NOT NULL,  
  prenom VARCHAR(50) NOT NULL,  
  classe VARCHAR(20) NOT NULL,  
  solde DECIMAL(10,2) NOT NULL DEFAULT 0.00  
) ENGINE=InnoDB;  
  
CREATE TABLE Menu (  
  idMenu INT AUTO_INCREMENT PRIMARY KEY,  
  dateMenu DATE NOT NULL,  
  description VARCHAR(200) NOT NULL,  
  prix DECIMAL(8,2) NOT NULL  
) ENGINE=InnoDB;  
  
CREATE TABLE Reservation (  
  idReservation INT AUTO_INCREMENT PRIMARY KEY,  
  dateReservation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  statut ENUM('confirmée', 'annulée', 'consommée')  
    NOT NULL DEFAULT 'confirmée',  
  idEleve INT NOT NULL,  
  idMenu INT NOT NULL,  
  FOREIGN KEY (idEleve) REFERENCES Eleve(idEleve),  
  FOREIGN KEY (idMenu) REFERENCES Menu(idMenu)  
) ENGINE=InnoDB;  
  
CREATE TABLE Paiement (  
  idPaiement INT AUTO_INCREMENT PRIMARY KEY,  
  montant DECIMAL(10,2) NOT NULL,  
  datePaiement DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  mode ENUM('espèces', 'mtn_momo', 'orange_money')  
    NOT NULL DEFAULT 'espèces',  
  idEleve INT NOT NULL,  
  FOREIGN KEY (idEleve) REFERENCES Eleve(idEleve)  
) ENGINE=InnoDB;
```

Points de vigilance : les tables référencées doivent être créées avant celles qui portent les clés étrangères ; DECIMAL (et non FLOAT) est obligatoire pour les montants afin d'éviter les erreurs d'arrondi binaire ; InnoDB est requis pour l'intégrité référentielle.

CORRIGÉ**Exercice 7 — Réseau local et publication : Site PHP/MySQL sur serveur interne**

1. Plan d'adressage. Réseau 192.168.20.0/24 ; le serveur occupe .10 ; on attribue les clients à partir de .11. En l'absence de routeur mentionné vers Internet, on peut laisser la passerelle vide ou utiliser le serveur si celui-ci fait office de passerelle ; on retient ici le serveur comme passerelle et DNS local.

Poste	Adresse IP	Masque	Passerelle	DNS
Serveur	192.168.20.10	255.255.255.0	–	8.8.8.8
PC1	192.168.20.11	255.255.255.0	192.168.20.10	192.168.20.10
PC2	192.168.20.12	255.255.255.0	192.168.20.10	192.168.20.10
PC3	192.168.20.13	255.255.255.0	192.168.20.10	192.168.20.10
PC4	192.168.20.14	255.255.255.0	192.168.20.10	192.168.20.10
PC5	192.168.20.15	255.255.255.0	192.168.20.10	192.168.20.10

(DNS 8.8.8.8 accepté si le serveur n'héberge pas de service DNS.)

2. Étapes de publication :

1. Installer la stack LAMP : `sudo apt update && sudo apt install apache2 php libapache2-mod-php php-mysql mysql-server.`
2. Sécuriser MySQL (`sudo mysql_secure_installation`), créer la base et l'utilisateur applicatif, importer le script : `mysql -u user -p mabase < script.sql.`
3. Déployer le code : `git clone` dans `/var/www/elearning` ou transfert `scp -r ./site user@192.168.20.10:/var/www/elearning.`
4. Fixer les droits : `sudo chown -R www-data:www-data /var/www/elearning` puis `sudo chmod -R 755 /var/www/elearning.`
5. Créer le VirtualHost dans `/etc/apache2/sites-available/elearning.conf` (ServerName, DocumentRoot `/var/www/elearning`), l'activer : `sudo a2ensite elearning && sudo systemctl reload apache2.`
6. Tester en local sur le serveur : `curl http://localhost` doit renvoyer le HTML de la page d'accueil.
7. Tester depuis un poste client : navigateur sur `http://192.168.20.10`; vérifier la connexion à la BDD (page dynamique réelle).

3. Règle pare-feu (HTTP limité au LAN) :

```
sudo ufw allow from 192.168.20.0/24 to any port 80 proto tcp
```

Point de vigilance : `sudo ufw allow 80` seul ouvrirait le port à toutes les sources, y compris Internet si le serveur est un jour exposé — il faut donc restreindre avec `from 192.168.20.0/24.`

CORRIGÉ

Exercice 8 — Plan de tests : Calcul de moyenne pondérée

Rappel de la formule : $\bar{n} = \frac{\sum n_i \cdot c_i}{\sum c_i}$, arrondie à 2 décimales.

#	Description	Données entrées	Résultat attendu	Type
1	Cas nominal : deux notes, coefficients différents	[12, 16], [2, 3]	$(24 + 48)/5 = 14,4$	Nominal
2	Valeurs limites : notes 0 et 20	[0, 20], [1, 1]	10,00	Limite
3	Coefficients variés (dont grand coefficient)	[10, 15, 18], [1, 2, 5]	$(10 + 30 + 90)/8 = 16,25$	Nominal
4	Note hors intervalle (> 20)	[21, 15], [1, 1]	false	Erreur
5	Coefficient nul	[12, 14], [0, 2]	false	Erreur
6	Tableaux de tailles différentes ou vides	[12, 14], [1]; ou [], []	false	Robustesse

Vérifications des calculs : cas 1 : $12 \times 2 + 16 \times 3 = 24 + 48 = 72$; $72/5 = 14,4$. Cas 2 : $(0 + 20)/2 = 10$. Cas 3 : $10 + 30 + 90 = 130$; $130/8 = 16,25$. Correct.

Points de vigilance : le cas 5 (coefficient nul) doit être rejeté avant la division, sinon risque de division par zéro si tous les coefficients sont nuls; le cas 6 vérifie la robustesse face à des entrées incohérentes; chaque cas doit être exécutable indépendamment (idéalement via PHPUnit).

CORRIGÉ Exercice 9 — Sujet type Bac : Rapport de réalisation et livrables

1. Ordre logique standard du rapport :

1. Introduction (contexte, problématique, objectifs)
2. Cahier des charges
3. Planification (Gantt, répartition des rôles)
4. Conception (UML : cas d'utilisation, classes, séquence ; MCD/MLD)
5. Réalisation (arborescence, extraits de code clés, captures d'écran IHM)
6. Plan de tests et résultats de recette
7. Déploiement et guide d'utilisation
8. Conclusion et perspectives

Logique : on suit le cycle de vie du projet (lancement → planification → conception → réalisation → validation → exploitation).

2. Introduction rédigée (modèle attendu) :

Le cybercafé « Le Net Plus », situé au quartier Marché A à Bafoussam, accueille quotidiennement une centaine de clients. La gestion y est encore entièrement manuelle : les durées de connexion sont notées au crayon sur un cahier, les montants calculés de tête, et les réservations de postes gérées oralement. Cette organisation provoque des erreurs de facturation, des pertes financières difficiles à chiffrer, des litiges avec les clients et de longues attentes aux heures de pointe. Comment dès lors automatiser la gestion du cybercafé afin de garantir une facturation exacte, une traçabilité complète des sessions et un pilotage quotidien fiable de l'activité ? Pour répondre à cette problématique, le projet « GestionCyber » vise trois objectifs : automatiser la création des comptes clients et la facturation au temps réel, assurer la traçabilité des sessions et des réservations de postes, et produire automatiquement un rapport journalier de caisse et de fréquentation. La solution retenue est une application web dynamique développée en PHP avec une base de données MySQL, déployée en réseau local sur la stack WampServer, accessible depuis tous les postes du cybercafé via un simple navigateur.

Barème indicatif (sur 10) : ordre des rubriques : 4 pts (0,5 par position correcte) ; introduction : 6 pts (contexte 1,5 ; problématique 1,5 ; objectifs 1,5 ; solution retenue 1,5).

Points de vigilance : l'introduction doit obligatoirement contenir les quatre éléments demandés, dans cet ordre ; la problématique doit être formulée (question ou enjeu clair) ; citer explicitement la stack (PHP/MySQL, WampServer, hébergement local).

CORRIGÉ**Exercice 10 — Sujet type Bac : Projet intégrateur complet « Fête de la Musique Yaoundé »**

1. Analyse. Acteurs : Participant (visiteur qui s'inscrit), Administrateur (EventCM), Passe-relais de paiement (MTN MoMo / Orange Money, acteur secondaire système). Cas d'utilisation :

- Participant : S'inscrire, Choisir un type de billet, Payer en ligne (include : « Générer QR code »), Recevoir/Télécharger son billet.
- Administrateur : S'authentifier, Consulter le tableau de bord (stats par type), Gérer la liste des participants, Valider un billet à l'entrée.

2. Conception BDD. MCD : $\text{TypeBillet}(1,1) \rightarrow (0,n) \text{Billet}(1,1) \leftarrow (1,1) \text{Participant}$ (un participant possède un ou plusieurs billets : $(1,n)$) ; $\text{Paiement}(1,1) \rightarrow (1,1) \text{Billet}$ (un paiement règle un billet).

MLD :

```
TypeBillet(idType, libelle, prix)
Participant(idParticipant, nom, email, telephone)
Billet ( idBillet , reference, dateAchat, qrCode, #idType, #idParticipant)
Paiement(idPaiement, montant, operateur, statut, datePaiement, #idBillet)
```

Script SQL demandé :

```
CREATE TABLE Participant (
```

```

    idParticipant INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(80) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    telephone VARCHAR(20) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE Billet (
    idBillet INT AUTO_INCREMENT PRIMARY KEY,
    reference VARCHAR(20) NOT NULL UNIQUE,
    dateAchat DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    qrCode VARCHAR(255),
    idType INT NOT NULL,
    idParticipant INT NOT NULL,
    FOREIGN KEY (idType) REFERENCES TypeBillet(idType),
    FOREIGN KEY (idParticipant) REFERENCES Participant(idParticipant)
) ENGINE=InnoDB;

```

3. Backend inscription.php :

```

<?php
header('Content-Type : application/json; charset=utf-8');

try {
    $pdo = new PDO(
        'mysql:host=localhost;dbname=billetterie;charset=utf8mb4',
        'app_user', 'motdepasse',
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
    );

    // Validation basique des donnees POST
    $nom = trim($_POST['nom'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $tel = trim($_POST['telephone'] ?? '');
    $idType = (int)($_POST['idType'] ?? 0);

    if ($nom === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)
        || $tel === '' || $idType <= 0) {
        http_response_code(400);
        echo json_encode(['erreur' => 'Donnees invalides']);
        exit;
    }

    // Verification du type de billet
    $stmt = $pdo->prepare('SELECT idType FROM TypeBillet WHERE idType = ?');
    $stmt->execute([$idType]);
    if (!$stmt->fetch()) {
        http_response_code(400);
        echo json_encode(['erreur' => 'Type de billet inexistant']);
        exit;
    }

    // Transaction : Participant + Billet
    $pdo->beginTransaction();

    $stmt = $pdo->prepare(
        'INSERT INTO Participant (nom, email, telephone) VALUES (?, ?, ?)'
    );
}

```

```
$stmt->execute([$nom, $email, $tel]);
$idParticipant = (int)$pdo->lastInsertId();

$reference = 'TKT-' . strtoupper(bin2hex(random_bytes(3))); // TKT-XXXXXX

$stmt = $pdo->prepare(
    'INSERT INTO Billet (reference, idType, idParticipant)
    VALUES (?, ?, ?)'
);
$stmt->execute([$reference, $idType, $idParticipant]);

$pdo->commit();

http_response_code(201);
echo json_encode(['succes' => true, 'reference' => $reference]);

} catch (PDOException $e) {
    if ($pdo->inTransaction()) {
        $pdo->rollBack();
    }
    // 23000 = violation de contrainte (ex : email en doublon)
    if ($e->getCode() === '23000') {
        http_response_code(409);
        echo json_encode(['erreur' => 'Email déjà inscrit']);
    } else {
        http_response_code(500);
        echo json_encode(['erreur' => 'Erreur serveur']);
    }
}
```

4. Réseau & Déploiement. Commandes ufw :

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow from 192.168.1.100 to any port 22 proto tcp
```

Directives Apache essentielles : `ServerName billetterie.eventcm.cm` (identifie le site); `DocumentRoot /var/www/billetterie/public` (n'expose que le dossier public); `AllowOverride None` dans le bloc `<Directory>` (empêche un fichier `.htaccess` malveillant de modifier la configuration).

5. Plan de tests (3 cas) :

#	Description	Données entrées	Résultat attendu
1	Inscription nominale	nom=« Amina », email=« amina@mail.cm », tel=« 6XXXXXXX », idType=1 (VIP)	HTTP 201, JSON avec référence TKT-... , lignes insérées en BDD
2	Email invalide	email=« amina@ »	HTTP 400, message « Données invalides », aucune insertion
3	Type de billet inexistant	idType=99	HTTP 400, « Type de billet inexistant », aucune insertion

Barème indicatif (sur 20) : analyse UML 3 pts ; MCD/MLD 4 pts ; SQL 3 pts ; PHP/PDO 5 pts ; réseau/sécurité 3 pts ; tests 2 pts.

Points de vigilance : requêtes préparées obligatoires (anti-injection SQL) ; la transaction garantit qu'on ne crée jamais un billet sans participant ; `bin2hex(random_bytes(3))` produit 6 caractères hexadécimaux ; SSH doit être restreint à l'IP admin ; ne jamais afficher le message brut de la `PDOException` au client (fuite d'informations).

CORRIGÉ**Exercice 11 — Sujet type Bac : Synthèse — articulation des compétences par phase****1. Livrables par phase :**

Phase	Livrables principaux
1. Lancement	Cahier des charges ; diagramme de Gantt et fiche de répartition des rôles
2. Conception	Dossier UML (cas d'utilisation, classes, séquence) ; MCD/MLD et dictionnaire de données
3. Réalisation	Code source versionné (Git) ; script SQL de création et de peuplement de la BDD
4. Tests/Recette	Plan de tests exécuté ; procès-verbal de recette signé
5. Déploiement	Serveur configuré (VirtualHost, pare-feu, SSL) ; guide d'utilisation et rapport de réalisation

2. Articulation des trois modules sur l'application web déployée :

- **Programmation** : fournit la logique applicative — architecture MVC (séparation modèle/vue/contrôleur), code PHP/PDO côté serveur, HTML/CSS/JavaScript côté client, éventuellement une API REST JSON pour le tableau de bord.
- **Systèmes d'information / Bases de données** : fournit la persistance et la cohérence

des données — conception MCD/MLD, scripts `CREATE TABLE`, contraintes d'intégrité, index sur les colonnes de recherche, transactions pour les opérations critiques, vues et triggers pour les statistiques automatiques.

- Réseaux / Internet / Sécurité : fournit l'infrastructure d'exploitation — adressage IP et configuration de la VM, DNS, certificat SSL/TLS (HTTPS), règles de pare-feu `ufw`, durcissement du serveur (comptes, mises à jour), sauvegardes distantes de la BDD.

Aucun des trois modules ne suffit seul : sans programmation pas de fonctionnalités, sans BDD pas de données fiables, sans réseau/sécurité pas d'accès ni de protection.

3. Exemple de faille (OWASP Top 10) : l'injection SQL (A03 :2021 – Injection). Si le développeur concatène directement les saisies utilisateur dans une requête (`"SELECT * FROM users WHERE login = '$login'"`), un attaquant saisissant `' OR '1'='1` contourne l'authentification et peut vider la base. Parade : utiliser systématiquement des requêtes préparées PDO (paramètres liés), valider et filtrer les entrées (`filter_var`), appliquer le principe du moindre privilège sur le compte MySQL de l'application (pas de `root`), et désactiver l'affichage des erreurs SQL en production (`display_errors = Off`).

Barème indicatif (sur 10) : livrables 4 pts ; articulation des modules 4 pts ; faille et parade 2 pts.

Points de vigilance : les livrables doivent être concrets (documents, scripts, configurations) et non des activités ; pour la faille, exiger le nom de la faille, un scénario d'attaque et une parade technique précise — « mettre un antivirus » ne répond pas à une injection SQL.

12 FICHE BILAN

Projet Intégrateur TI

1. Lancement & Planification

- Cahier charges
- Planning Gantt
- Rôles équipe

2. Conception UML & BDD

- Cas utilisation
- Classes / MLD
- Séquence / BD

3. Développement Web Dynamique

- PHP / MVC
- HTML5 / CSS / JS
- PDO / Sécurité

4. Réseau & Hébergement

- Config IP / DNS
- Apache / VHost
- Pare-feu / SSL

5. Tests & Recette

- Unitaires / Int.
- Recette utilis.
- Correction bugs

6. Documentation

- Rapport réal.
- Guide utilis.
- Maintenance

Carte mentale du projet intégrateur : les 6 phases clés du cycle de vie en Terminale TI

FICHE BILAN L'essentiel à connaître pour l'examen

12.1 Définitions clés

- **Cahier des charges** : Document contractuel listant les besoins fonctionnels (cas d'utilisation) et non-fonctionnels (sécurité, performance, ergonomie) du client.
- **Diagramme de classes UML** : Modèle statique du système d'information (attributs, types, multiplicités) qui sert de base au **Modèle Logique de Données (MLD)** et aux classes PHP.
- **Architecture MVC** : Séparation Modèle (données/BDD), Vue (HTML/CSS), Contrôleur (logique PHP, routing). Garantit la maintenabilité.
- **PDO (PHP Data Objects)** : Interface standardisée pour accéder aux BDD (MySQL, PostgreSQL). Impose les **requêtes préparées** contre l'injection SQL.
- **VirtualHost (Apache)** : Configuration permettant d'héberger plusieurs sites (ex. `projet.local`) sur un même serveur via le fichier `hosts` local.
- **Recette** : Phase de validation formelle par le client (tests d'acceptation) avant la mise en production.

12.2 Méthodes express (à connaître par cœur)

Phase	Étapes clés / Livrables
1. Lancement	Analyse besoins → Cahier des charges (fonctionnel + technique) → Planning (Gantt/PERT) → Répartition rôles (Chef de projet, Dev, Admin Réseau, Testeur).
2. Conception	Diagramme cas d'utilisation → Diagramme de classes (avec multiplicités) → MLD (norme Merise) → Script SQL création tables (CREATE TABLE, FK, index).
3. Dev Web	Structure MVC : /public (index.php, assets), /app (Controllers, Models, Views), /config (PDO). Sécurité : password_hash(), htmlspecialchars(), CSRF token, HTTPS.
4. Réseau	IP fixe / DHCP → DNS local (/etc/hosts ou Bind9) → Apache VirtualHost (DocumentRoot, ServerName) → Pare-feu (UFW/ip-tables : ports 80, 443, 3306 local only).
5. Tests	Unitaires : fonctions Model (PHPUnit). Intégration : scénarios complets (inscription → paiement Mobile Money). Recette : Grille de test signée client. Bugs : Suivi (Mantis/Jira/Trello) → Correction → Régression.
6. Doc	Rapport : Contexte, Cahier charges, Choix techniques (UML, DB, Stack), Difficultés, Solutions, Bilan. Guide : Installation (XAMPP/-Wamp, SQL, Hosts), Manuel utilisateur (copies d'écran), FAQ, Contacts support.

12.3 Commandes & Syntaxes indispensables

- **SQL (MySQL)** : `CREATE DATABASE projet_ti CHARACTER SET utf8mb4;; CREATE USER 'dev'@'localhost' IDENTIFIED BY 'pwd'; GRANT ALL ON projet_ti.* TO 'dev'@'localhost';`
- **PDO (PHP)** : `$pdo = new PDO("mysql:host=localhost;dbname=projet_ti;charset=utf8mb4", "dev", "pwd", [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]);`
- **Apache (VirtualHost)** :

```
<VirtualHost *:80>
    ServerName projet-ti.local
    DocumentRoot /var/www/projet-ti/public
    <Directory /var/www/projet-ti/public>
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/projet-ti_error.log
    CustomLog ${APACHE_LOG_DIR}/projet-ti_access.log combined
</VirtualHost>
```

- **Linux (Debian/Ubuntu Server) :** `sudo a2ensite projet-ti.conf && sudo systemctl reload apache2, sudo ufw allow 'Apache Full'.`

À RETENIR Checklist avant le Bac

- ☐ Savoir rédiger un cahier des charges structuré (besoins fonctionnels numérotés + contraintes techniques).
- ☐ Maîtriser les 3 diagrammes UML obligatoires : Cas d'utilisation (acteurs/UC), Classes (attributs/types/multiplicités), Séquence (scénario nominal + alternative).
- ☐ Transformer un diagramme de classes en MLD puis en script SQL (PK, FK, NOT NULL, INDEX, ENGINE=InnoDB).
- ☐ Implémenter une architecture MVC en PHP natif : point d'entrée unique (`index.php`), routeur simple, séparation stricte M/V/C.
- ☐ Sécuriser l'accès BDD : PDO + requêtes préparées (`prepare/execute`), jamais de concaténation de variables dans le SQL.
- ☐ Gérer l'authentification : `password_hash()` (inscription), `password_verify()` (connexion), sessions PHP (`session_start()`, `session_regenerate_id()`), protection CSRF (token hidden input).
- ☐ Configurer un VirtualHost Apache local (fichier `.conf`, activation `a2ensite`, modification `/etc/hosts` client).
- ☐ Appliquer les bases réseau/sécurité : IP statique serveur, pare-feu (fermer port 3306 à l'extérieur), HTTPS auto-signé (OpenSSL) pour le local.
- ☐ Élaborer un plan de tests : cas de test (ID, action, données, résultat attendu, obtenu, statut), distinction tests unitaires / intégration / recette.
- ☐ Produire les 2 documents finaux : Rapport de réalisation (structure imposée : intro, analyse, conception, réalisation, tests, conclusion) et Guide d'utilisation (installation + mode d'emploi utilisateur final).
- ☐ Présenter le projet à l'oral : démo fluide (scénario nominal), justification des choix techniques (pourquoi MVC ? pourquoi PDO ?), gestion des difficultés (bug critique résolu, travail d'équipe).

**Signé • OnBuch+**Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC**OnBuch+**